Jedi 1.0

User manual

October 2026

Contents

Reading this documentation 2

The philosophy of Jedi 3

Using Jedi day to day 5

1. Jedi in a few pages 6

2. Installing and preparing Jedi 8

3. Choosing a security 16

4. Shortlisting with tools profile 18

5. Searching for a scale with tools score 22

6. Evaluating an opening with tools candidate 26

7. Opening a trade 30

8. Monitoring a trade daily 33

9. Adapting or ending a trade 37

10. Managing the market database 42

Understand Jedi’s mechanics 46

11. Round trips and trade flows 47

12. Positions, steps and levels 49

13. Read the structure of a scale 51

14. Understand a type I scale 54

15. Understand a type II scale 56

16. Maintain continuity between settings 58

17. From trade state to orders 61

18. Understand capital, risk and position sizing 63

19. Check with tools calc and tools play 67

20. Follow a complete trade example 70

Studying historical results 73

21. Preparing a historical study 74

22. Simulating a scale with tools backtest 76

23. Euronext Growth study example 79

24. Interpreting, comparing and monitoring results 82

Appendix 1. Command and option reference 84

Appendix 2. Configuring API and Web quote providers 89

Appendix 3. External selection script: JEDI ProScreener 93

Appendix 4. Jedi glossary and notation 96

Appendix 5. Default configuration and parameter usage 101

Reading this documentation

This edition accompanies **`next-version`, in preparation for Jedi 1.1**. It includes `split + reinit=reset` and `split + reinit=merge`, together with trade and scale naming conventions. These combinations require the updated executable; the published Jedi 1.0.0 release still rejects them. Chapter 02's installation examples retain the published 1.0.0 distribution: adapt the version for a later release.

Jedi is intended for investors who want to apply and monitor a mechanical trading method. On first reading, start with The philosophy of Jedi, just before Part I: the method's principle, discipline, the investor's decisions and the limits of historical results.

Three reading paths

Part I, chapters 01–10, covers practice: installing Jedi, choosing a security, studying a scale, evaluating an opening and monitoring the trade. On first use, read chapters 01 and 02, then reproduce the examples in the demonstration environment. Chapters 03–06 prepare a project; chapters 07–09 record operations. Chapter 10 explains market databases and prepares the Demo database used from chapter 04 onwards.

Part II, chapters 11–20, explains the rules: round trips, positions, scales, continuity, orders and capital. `tools calc` and `tools play` let you check the mechanics using chosen movements. Chapter 20 combines these ideas in a complete example.

Part III, chapters 21–24, organises historical studies: preparing data, backtesting, a chronological example and interpretation. It distinguishes searching for a scale, checking it on another period and monitoring dated decisions.

The appendices provide the command reference, provider configuration, the JEDI ProScreener script and a glossary. They let you find a detail without interrupting the main reading path.

Example conventions

Commands are written for Bash on Linux or Ubuntu in WSL. Command and option names remain English whatever the language of messages. Example prices are in euros. `10%` and `0.10` express the same coefficient when a field accepts decimals or percentages.

A backslash at the end of a line continues a command on the next line. Inside quoted text, line breaks belong to the text. Where `$` and `>` are shown with input, they represent shell prompts and should not be copied.

Examples use **`jedidemo`**, the launcher installed in chapter 02. It keeps experiments in their own environment. With the supplied launchers, **`jedi` always selects production**; **`jedidev`** is for development. Copy the files from `examples/` to the demonstration directory as described in that chapter.

EXA, EXB, EXC and GAP, their identifiers and their quotations are **fictional**. The files use weekdays, including some public holidays. They help reproduce commands and amounts without representing actual investments.

Required settings appear before the examples: fees, simulation capital and main database state. An existing committed portfolio changes `candidate` reserves in particular. A creation command assumes that the object or database does not already exist.

Finding information

Numbers identify files and cross-references. PDFs rebuilt from this edition include the three parts, 24 chapters and five appendices in their table of contents. French and English editions follow the same plan.

To discover the installed version's capabilities, run `jedi` without arguments, then request a subcommand's help, for example `jedi tools backtest --help`. `jedi version` shows the version; `jedi config --paths` shows the paths actually used.

Each study and trade remains understandable through its data, settings and operations. Keeping these elements lets you explain a result and resume work without having to guess earlier choices.

The philosophy of Jedi

General principle

Jedi is a method for managing a trade mechanically, based on predefined rules.

Once a trade has been opened and configured, the software determines the operations to consider according to:

- the scale in use;
- the current position of the trade;
- the operations already completed;
- any capital management rules that have been enabled.

Jedi does not try to predict the market's next movement to decide each day whether to buy or sell.

Its principle is to define rules for intervening in advance, then apply them consistently when the planned levels are reached.

Working with price fluctuations

Prices do not generally move continuously in a single direction. Even within a lasting trend, upward and downward movements follow one another.

Jedi seeks to use some of these fluctuations through successive purchases and sales.

For example, a round trip can involve selling a quantity of shares after a rise, then buying it back at a lower price.

The price difference on that round trip is positive. Its net result nevertheless depends on fees, any applicable tax and the actual execution conditions.

The method therefore does not require the user to predict the security's future trend precisely.

It still involves risk: a prolonged adverse movement, insufficient liquidity, large price gaps or insufficient capital can impair or prevent the expected functioning of the trade.

Decisions that remain with the user

Jedi automates calculations and applies rules, while the user retains responsibility for key decisions:

- choosing the security to examine;
- deciding whether to consider a new trade;
- making the final choice among the solutions proposed by the analysis tools;
- actually executing orders with their broker;
- deciding whether to continue or stop a trade when that decision is theirs to make.

Jedi's tools provide information and rules for managing a trade. They do not replace the investment decision.

Reproducible rules

One of Jedi's central aims is to make operational decisions reproducible.

For the same situation and the same settings, the rules should lead to the same result.

This reproducibility also addresses a major human difficulty in trading: the tendency to decide under the influence of fear, impatience or euphoria. The rules are defined before the situation arises; Jedi then applies them without changing them in response to the emotion of the moment.

This property helps users:

- limit improvisation during the life of the trade;
- keep operational decisions from being dictated by emotions;
- maintain a precise record of decisions and operations;
- test the rules on historical data;
- compare different configurations;
- study their behaviour using reproducible methods.

The user keeps the responsibilities described above. Jedi separates those decisions from immediate reactions to market fluctuations and helps maintain the chosen discipline.

Reproducibility does not mean that future financial results are themselves predictable.

What Jedi does not guarantee

Jedi does not guarantee:

- that a selected security will rise;
- that a scale will be profitable in every market situation;
- that a trade can always be funded;
- that a historical result will recur in the future;
- the absence of losses.

Statistical tools and backtests study how rules behave under defined conditions. They do not turn those observations into certainties about future results.

The first part of this documentation can now focus on a more practical question: **how do you use Jedi?**

Using Jedi day to day

This part follows the user's path from an initial security idea to trade monitoring. It is for an investor who wants to understand the decisions and apply regular rules without first learning Jedi's formulas.

A **security** is the instrument purchased and sold. A **scale** defines the mechanics' prices and quantities. A **trade** brings together that management, its settings and operations. These concepts are introduced before their details.

Security selection can come from personal research, an external screener or `profile`. `score` then helps search for a scale over the chosen history; `candidate` evaluates opening with the available price and resources. The user keeps the choices and executes orders with their broker.

Chapter 01 presents the path; chapter 02 prepares installation. Chapters 03 to 06 move from security selection to proposal evaluation. Chapters 07 to 09 cover opening, monitoring and adaptations, including a journal of exceptional events. Chapter 10 explains quotation files and market databases.

Examples use Jedi commands and clearly identified data. Demonstration files are fictional: they explain outputs. After this part, the reader will know where to find useful information, what to record and what to check before proceeding.

Part II explains the mechanics behind these commands. Part III shows how to examine results using historical data.

1. Jedi in a few pages

The examples below use the `jedidemo` launcher, installed in chapter 02. It keeps their data in a demonstration environment separate from your real portfolio.

Rules for managing a trade

Jedi is a command-line application for managing trades according to rules defined in advance. A **trade** brings together purchases and sales of one security, its scale, its settings and its history.

The **scale** connects price movements to quantities to buy or sell. From the current trade state, Jedi calculates the orders to consider. The user places them with their broker, checks their execution and records the operations actually completed.

This separation makes the process understandable: choose the rules, observe the levels reached, execute and record. A prolonged rise or fall, high fees or insufficient funds can nevertheless produce an unfavourable outcome.

Working with fluctuations

A round trip can involve selling some shares after a rise, then buying them back during a fall. Here is a **fictional** example calculated with Jedi:

```
jedidemo --border=markdown tools play -c o0,6-14,17 -- \
  scale=vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0,q0=100,c0=10,tick=0.01 1 -1
```

The created environment specifies a scale, 100 shares purchased at €10 and a tick size of €0.01. The arguments `1 -1` request one step upwards, then one step downwards. The double hyphen separates options from scenario arguments.

In the resulting output, the initial purchase is followed by:

Operation	Quantity	Price	Amount	Holdings afterwards
Sale	6	€11.00	€66.00	94
Purchase	6	€9.90	€59.40	100

Holdings return to 100 shares and the round trip adds €6.60 before fees. However, the final price is €9.90: the remaining shares must also be valued to determine the overall result. A profitable round trip and a profitable trade overall are separate findings.

`tools play` explains the mechanics here. It reads no historical prices and sends no orders to the broker. Type I and type II scales are developed in Part II.

Three decisions before opening

Stage	Question	Possible method
Choose a security	Which security should be examined?	Personal selection, external screener or `tools profile`
Choose a scale	Which rules should be examined?	An already chosen scale or a search with `tools score`
Evaluate the proposal	Can it be opened and continued with these quantities and funds?	`tools candidate`

The user chooses the security. `profile` helps establish a list based on trend, liquidity, volatility, oscillation and three additional criteria. Its statuses are **retained**, **caution** and **rejected**.

`score` simulates different scales on the selected security's history. Its standard ranking shows raw results, activity and maximum investment. The reader can understand the sorting criteria and choose a scale definition.

`candidate` examines that definition at the supplied price or the price recorded in Jedi. It checks useful quantities and funding, considering existing trades. An operable proposal can require too much capital; a proposal with available funds can produce orders that are too small.

A further backtest can examine a fixed scale over a historical period. None of these calculations turns a selection into an automatic purchase order.

The monitoring cycle

Once the trade is open, routine work consists of updating the price, consulting orders, monitoring their execution with the broker, recording operations and recalculating orders.

An **order** is a proposed or placed instruction; an **operation** is an executed purchase or sale. Recording an operation means describing what actually happened, including its quantity and price.

Exceptional events deserve a record: partial execution, cancellation of an order remainder, suspension, a quotation incident or a settings change. The `note` journal stores the date, security or trade name, facts and decision. Chapters 08 and 09 explain its use.

The data required

The main database stores securities in use, trades, settings, operations, notes and recorded prices. A separate market database stores historical quotations.

This second database is optional. `score` and `backtest` can read a prepared individual file; `profile` can also examine that file. A market database is used to compare a population of securities and manage its freshness.

An **OHLCV** history contains the open, high, low, close and volume for each session. A current price is a point-in-time value. Importing a history with `market` does not update the price in the main database.

`quote get` queries price providers; it updates no database. Recording and monitoring commands are explained in the practical chapters.

Where to continue

Chapter 03 helps choose a security. Chapters 04, 05 and 06 explain `profile`, `score` and `candidate`. Chapters 07 to 09 follow a real trade; chapter 10 explains files and market databases.

Part II develops the mechanics with `play` and `calc`. Part III explains backtests, period comparisons and result monitoring. The appendices provide references for commands, providers, the external screener and terminology.

2. Installing and preparing Jedi

Choose your environment

Jedi runs in a terminal. The distribution format chosen for Jedi 1.0 is an archive containing a statically compiled Linux executable for **x86-64** processors. Users do not need to install Crystal or compile the sources to use that executable.

Environment	Intended use for Jedi 1.0
Linux x86-64	Linux executable supplied in the release archive.
Windows x86-64	The same executable in a Linux distribution under WSL 2.
Native Windows, Linux ARM64, macOS	No executable distributed through this installation path.

SQLite is included in the static binary. **gnuplot** remains an external program needed for graphical functions. Management, selection and simulation commands can be used without producing a chart.

Install the executable on Linux

Check your computer's architecture:

```
uname -m
```

The executable described here requires `x86_64`. Download the binary archive and its checksum file from **Releases in the Jedi project repository**. The automatic "Source code" archive contains sources to compile; choose the executable archive.

The commands below use the convention `jedi-1.0.0-linux-x86_64.tar.gz`, with an extracted directory of the same name without `.tar.gz`. Adjust the version and names if your downloaded release specifies different ones. From the download directory:

```
jedi_version="1.0.0"
jedi_archive="jedi-${jedi_version}-linux-x86_64.tar.gz"
sha256sum -c "${jedi_archive}.sha256"
```

The check must report `OK`. If it fails, download again before proceeding.

Install the binary for your own account without administrator privileges:

```
tar -xzf "$jedi_archive"
mkdir -p "$HOME/.local/libexec" "$HOME/.local/bin"
install -m 0755 "jedi-${jedi_version}-linux-x86_64/jedi" \
  "$HOME/.local/libexec/jedi"
```

Then install the three launchers, supplied in examples/launchers/ in the documentation archive. From its `examples/` directory:

```
install -m 0755 launchers/jedi "$HOME/.local/bin/jedi"
install -m 0755 launchers/jedidemo "$HOME/.local/bin/jedidemo"
install -m 0755 launchers/jedidev "$HOME/.local/bin/jedidev"
export PATH="$HOME/.local/bin:$PATH"
hash -r
command -v jedi
command -v jedidemo
command -v jedidev
```

The displayed paths must identify the three files in `~/.local/bin/`. If `jedi` is already defined as an alias or function, remove that definition to use the installed launcher. `type -a jedi` shows the definitions found.

The compiled binary is in **`~/.local/libexec/jedi`**. The three small scripts in **`~/.local/bin/`** call it directly: `jedi` for your real portfolio, `jedidemo` for examples and `jedidev` for development. Run them as ordinary commands; they refuse to be sourced.

Launcher names, comments and messages are in English in both documentation editions. Two messages identify errors specific to these scripts:

Message	Action
`Jedi binary missing or not executable`	Check the binary's installation in `~/.local/libexec/jedi` and its executable permission.
`Run this launcher as a command; do not source it`	Call `jedi`, `jedidemo` or `jedidev` directly in the terminal.

To keep the search path across Bash sessions, add `export PATH="$HOME/.local/bin:$PATH"` to `~/.bashrc`. Run everyday Jedi commands under your user account.

For graphical functions, check:

```
command -v gnuplot
gnuplot --version
```

Install the `gnuplot` package with your distribution's package manager if needed.

On Windows, use the WSL Linux terminal

Jedi runs in Ubuntu under WSL 2. Keep the two terminals distinct:

Terminal	Commands in this chapter
PowerShell, on Windows	Installing and opening WSL; commands starting with `wsl`.
Ubuntu, on Linux	Installing Jedi, directories, configuration and every `jedi` command.

Install Ubuntu and open the right terminal

The following procedure applies to Windows 11 or Windows 10 version 2004 and later. It has two stages: enable the Windows components WSL needs, then install the Ubuntu Linux distribution.

In **PowerShell opened as administrator**, first enable WSL:

```
wsl --install
```

If Ubuntu opens directly, create the Linux account as explained below. If Windows requests a restart, restart before continuing.

After restarting, check the installation. Reopen PowerShell under the same Windows account:

```
wsl --list --verbose
```

If Ubuntu appears in the list, continue with account creation and the version check. If the message says that no distribution is installed, reopen PowerShell as administrator and **run the installation again**:

```
wsl --install -d Ubuntu
```

The first command may have stopped after enabling `VirtualMachinePlatform`, which requires a restart. Its success message then concerns that activation; it does not prove that Ubuntu is installed. Now wait for the distribution download and installation. If another restart is requested, restart and check the list again.

Until Ubuntu is installed, `wsl -d Ubuntu` cannot open it and returns `WSL_E_DISTRO_NOT_FOUND`. This command opens an existing distribution; it does not install one. The Microsoft procedure describes distribution installation.

Create the Linux account on first launch. Ubuntu may open automatically when installation finishes. Create a Linux username and password. This account is separate from the Windows account; the typed password is not displayed. Linux account setup details this step.

Once the account is created, Ubuntu's prompt may look like `user@computer:~$`. Commands entered there are Linux commands. To return to PowerShell and check the installation, enter this in Ubuntu:

```
exit
```

In PowerShell, check the list:

```
wsl --list --verbose
```

An **Ubuntu** row must now appear, with **2** in the `VERSION` column. Its state may be `Running` or `Stopped`; both indicate an installed distribution. For a newly installed Ubuntu distribution still on version 1, run `wsl --set-version Ubuntu 2` from PowerShell to convert it.

Then open Ubuntu from PowerShell:

```
wsl -d Ubuntu
```

If the Linux account has not yet been created, create it on this first launch. If the name shown in the list differs from Ubuntu, use that name in commands. The WSL reference describes listing and opening distributions.

All following steps take place **in Ubuntu**, using the Linux account created. Return to its home directory:

```
cd ~
echo "$HOME"
```

A path such as `/home/user` identifies this Linux home directory. It differs from `C:\Users\...` on Windows.

Prepare the language in Ubuntu

A fresh Ubuntu installation may use `LANG=C.UTF-8`. This locale is a neutral system setting with UTF-8. To choose French or English and prepare the examples, install the locales in **the Ubuntu terminal**:

```
sudo apt update
sudo apt install locales
sudo dpkg-reconfigure locales
```

In the displayed list, select these entries with **Space**:

Locale to generate	Use
`fr_FR.UTF-8 UTF-8`	French, France.
`en_US.UTF-8 UTF-8`	English, United States.

Use **Tab** to reach the confirmation button, then **Enter**. When asked for the default locale, choose **en_US.UTF-8** for the English reading path, or **fr_FR.UTF-8** for the French path. Wait for generation to finish.

Check the available locales:

```
locale -a
```

Their names may appear as `fr_FR.utf8` and `en_US.utf8`. Generation makes these locales available; it does not necessarily change the language of the terminal already open. To use English in this session:

```
export LANG=en_US.UTF-8
locale
```

For French, use `export LANG=fr_FR.UTF-8`. Check the `LANG` line in the `locale` output. Jedi uses this variable with its system language setting; installing locales and choosing `LANG` are separate operations.

The **Find the program and editor in future sessions** section below preserves this choice in `~/.bashrc`. Ubuntu's references on locale generation and inspection describe these commands.

Prepare Jedi's directories in Ubuntu

A fresh Ubuntu installation may not yet contain the XDG directories Jedi uses. Creating them explicitly makes its file locations clear:

```
mkdir -p "$HOME/.config/jedi"
mkdir -p "$HOME/.local/state/jedi"
mkdir -p "$HOME/.local/share/jedi/market"
mkdir -p "$HOME/.local/bin" "$HOME/.local/libexec"
mkdir -p "$HOME/jedi-examples"
```

Directory	Jedi file or use
`~/.config/jedi/`	Configuration `jedi.conf`.
`~/.local/state/jedi/`	Log `jedi.log`.
`~/.local/share/jedi/`	Main database `jedi.db`.
`~/.local/share/jedi/market/`	Historical market databases.
`~/.local/bin/`	Public launchers `jedi`, `jedidemo`, `jedidev`.
`~/.local/libexec/`	Compiled `jedi` binary called by the launchers.
`~/jedi-examples/`	Suggested working directory for tests and supplied files.

You do not need to define `XDG_CONFIG_HOME`, `XDG_STATE_HOME` or `XDG_DATA_HOME`. When absent, Jedi uses `~/.config`, `~/.local/state` and `~/.local/share` respectively, followed by their `jedi` subdirectory. It can also create missing parent directories during initialisation.

If XDG variables are already defined, paths may differ. The public `jedi` launcher uses these production paths and clears inherited `JEDI_*` path overrides. Check the effective locations with `jedi config --paths` after installation. `jedidemo` and `jedidev` select their own local environments, explained below.

Install the program and useful tools

Create a Linux directory for the downloaded binary archive and its checksum file:

```
mkdir -p "$HOME/telechargements-jedi"
cd "$HOME/telechargements-jedi"
explorer.exe .
```

Windows Explorer opens this Linux directory. Copy both files downloaded on Windows into it. Return to Ubuntu and follow **Install the executable on Linux**, above, starting with the `sha256sum` check. Keep databases and experiments in the Linux filesystem, following Microsoft's guidance.

Also install an editor and the graphics program in Ubuntu:

```
sudo apt update
sudo apt install gnuplot nano
```

The requested password is the Linux account's password. Run subsequent Jedi commands using that account.

Find the program and editor in future sessions

In Ubuntu, open the terminal's startup file:

```
nano "$HOME/.bashrc"
```

Append these four lines. The last one selects English for this terminal:

```
export PATH="$HOME/.local/bin:$PATH"
export EDITOR=nano
export VISUAL=nano
export LANG=en_US.UTF-8
```

In nano, save with **Ctrl+O**, confirm the filename with **Enter**, then exit with **Ctrl+X**. Reload the settings in the current terminal and check:

```
source "$HOME/.bashrc"
command -v jedi
command -v nano
jedi version
jedi config --paths
```

At Jedi's first launch, explicitly accept file creation as explained in the next section. The editor will serve `note create --edit` and `note update --edit`, described in chapter 08.

Find files from Windows

To open the configuration directory in Explorer:

```
cd "$HOME/.config/jedi"
explorer.exe .
```

Explorer also provides access through `\\wsl$`, then the distribution and its `home` directory. Paths displayed by `jedi config --paths` remain Linux paths for the Ubuntu terminal. Copy the documentation archive's `examples/` files to `~/jedi-examples/` for tests; the example guide identifies the databases to prepare.

Run Jedi for the first time

```
jedi version
```

This command displays the Jedi version and compilation information. At first launch it asks to create the configuration, log and main database in that order. **Explicitly answer `Y`** to each English prompt, or `O` in French.

The main database starts empty. It will hold securities, scales, trades, settings, operations and notes. Historical market databases are prepared separately in chapter 10.

Display the locations actually used:

```
jedi config --paths
```

Item	Usual Linux location
Configuration	`~/.config/jedi/jedi.conf`
Log	`~/.local/state/jedi/jedi.log`
Main database	`~/.local/share/jedi/jedi.db`
Market databases	`~/.local/share/jedi/market/`

System XDG variables can change these locations. Use the paths shown by `config --paths` for your backups.

Choose language and settings

With its default system language setting, Jedi uses French when `LANG` specifies French and English for other supported language values. To choose a command's language:

```
LANG=fr_FR.UTF-8 jedi version
LANG=en_US.UTF-8 jedi version
```

The `jedi.conf` file can remain empty: the program includes default settings. Add only the values you need to change. View the effective configuration with:

```
jedi config
```

It can contain access keys for providers you have configured. Check its contents before sharing it.

These **fictional** settings are used for the trade in chapters 07–09. Put them in the experiment environment's `jedi_test.conf`, described below, to reproduce the walkthrough:

```
fees:
  rate: 0.0035
  min: 0.0
trading:
  capital: 5000.0
market_data:
  default_universe: france
```

Here `fees.rate` means 0.35% and `fees.min` is the minimum per operation. `trading.capital` is the reference capital dedicated to trades. For actual use, enter your own amounts and fees. Simulation settings are separate from capital used for real trade management.

Command options override the settings they address: for example, `candidate --capital=...` uses that capital for the evaluation; it does not change `trading.capital`. File changes are read at the next launch.

Prepare a separate experiment environment

With the launchers installed above, **the command name selects the environment**:

Command	Use	Default location
`jedi`	Real portfolio.	`jedi.conf`, `jedi.log`, `jedi.db` and market databases at the XDG locations.
`jedidemo`	Documentation exercises.	Configuration, log, `jedi_test.db` and market under `~/jedi-examples/jedi/env_test/`.
`jedidev`	Development and contributor experiments.	`jedi_dev.*` files and market in the project's `env_dev` environment.

The first two commands are enough to use Jedi and learn with the examples. Commands behave as usual; their files are separate.

There is no terminal environment to switch. You can call `jedi`, then `jedidemo`, then `jedi` again. **`jedi` always uses production**; its launcher forces that choice and removes inherited `JEDI_*` path settings in its own process. XDG paths remain those of the system.

Copy the example files

From the documentation archive's `examples/` directory:

```
mkdir -p "$HOME/jedi-examples"
cp *.txt "$HOME/jedi-examples/"
jedidemo config --paths
```

At first launch, explicitly accept creation of the configuration, log and main database: `Y` in English, `O` in French. Check that `environment` shows **`test`** and record the paths:

Experiment item	Default location
Configuration	`~/jedi-examples/jedi/env_test/conf/jedi_test.conf`
Log	`~/jedi-examples/jedi/env_test/log/jedi_test.log`
Main database	`~/jedi-examples/jedi/env_test/data/jedi_test.db`
Market databases	`~/jedi-examples/jedi/env_test/market/`

Add the fictional settings from the previous section to **`jedi_test.conf`** at the displayed path. Example commands in the following chapters use **`jedidemo`**. For commands studying an actual source, first prepare its histories in this environment's market directory.

```
jedidemo trade list
```

The launcher opens `~/jedi-examples/` for Jedi, so filenames such as `cotations_EXA.txt` work from any terminal directory. Relative paths passed to `jedidemo` are read from that directory, including files to import or export. Use an absolute path for a file elsewhere. Redirections such as `> backup.txt` are performed by the terminal in its own directory; an absolute path avoids ambiguity.

The script's `cd` affects only the launcher and Jedi process. **The calling terminal keeps its directory and variables.** The launchers also retain its language, editor and time zone. To reproduce a calculation explicitly in UTC for one command:

```
TZ=UTC jedidemo version
```

Alternate between portfolio and demonstration

```
jedi trade list
jedidemo trade list
jedi config --paths
jedidemo config --paths
```

The first list concerns your real portfolio; the second concerns fictional trades. The last two commands show each environment's paths. Databases remain in place: `jedidemo` resumes the same examples at the next launch.

Keep several experiment environments

Each directory can have its own test database and market. For example, keep chapters 03–06's evaluations without trades in the usual environment, then create another one for chapters 07–09's openings:

```
mkdir -p "$HOME/jedi-examples-trades"
cp "$HOME/jedi-examples/"*.txt "$HOME/jedi-examples-trades/"
JEDI_DEMO_DIR="$HOME/jedi-examples-trades" jedidemo config --paths
```

This first Jedi command initialises the other environment after your confirmations. Use the same **`JEDI_DEMO_DIR=...`** prefix for every command using it:

```
JEDI_DEMO_DIR="$HOME/jedi-examples-trades" jedidemo trade list
jedidemo trade list
```

The first line uses the second environment; the next resumes the usual one. To choose another demonstration directory permanently, add `export JEDI_DEMO_DIR="..."` to `~/.bashrc`. It customises `jedidemo`; `jedi` remains production.

From an experiment directory you have already created, you can also run:

```
JEDI_DEMO_DIR="$PWD" jedidemo config --paths
```

In a directory called `experiment`, files are under `experiment/jedi/env_test/`. If the directory is already named `jedi`, they are under `jedi/env_test/`, without repeating that name.

Optionally choose another market

The demonstration market is local to the environment by default. To deliberately use an existing market directory:

```
JEDI_DEMO_MARKET_DIR="/path/to/market" \
  jedidemo market info abcbourse
```

Replace the path and source with those of your data. Only market databases are then shared; the main test database remains in the selected environment. `market import`, `update`, `replace` and `group` act on the selected market, so use a separate market for exercises that modify data.

A relative `JEDI_DEMO_MARKET_DIR` is read from the demonstration directory. Launchers clear the general `JEDI_MARKET_DIR` variable; deliberate selection uses `JEDI_DEMO_MARKET_DIR` or `JEDI_DEV_MARKET_DIR`.

For developers: use jedidev

This section is optional for users of the supplied binary. Contributors install **Crystal 1.21.1** and **Shards** following the official Crystal instructions, then work from the source root.

The `jedidev` launcher uses the current directory as its project, or the one supplied in `JEDI_DEV_DIR`. It selects `dev` mode and keeps configuration, log, main database and market databases separate:

```
jedidev config --paths
```

If the project is named `jedi`, its database is `<project>/env_dev/data/jedi_dev.db` and market `<project>/env_dev/market/`. With another directory name, they are under `<project>/jedi/env_dev/`.

To install dependencies, check sources and compile a working binary:

```
shards install
crystal tool format --check src spec
TZ=UTC crystal spec
mkdir -p bin
crystal build src/jedi_cli.cr --error-trace -o bin/jedi-dev
chmod +x bin/jedi-dev
JEDI_DEV_BINARY="./bin/jedi-dev" jedidev config --paths
```

Compilation also requires the dependencies' development libraries, including SQLite. Package names depend on the distribution. Tests run in UTC to reproduce the validation dates.

`JEDI_DEV_BINARY` selects the compiled binary for that command. Without it, `jedidev` calls the binary installed in `~/.local/libexec/jedi`. Its relative path is read from the selected project. To call from another directory, supply both paths:

```
JEDI_DEV_DIR="/path/to/project" \
  JEDI_DEV_BINARY="./bin/jedi-dev" jedidev version
```

`JEDI_DEV_MARKET_DIR` deliberately selects another market, as for demonstration. For real portfolio monitoring, use the `jedi` launcher.

Compile quickly during development

Compile without `--release` for everyday changes. For performance measurements and final checks, compile with `--release`. Both commands use `src/jedi_cli.cr`; the database environment is selected when running through `jedi`, `jedidemo` or `jedidev`.

```
mkdir -p bin
crystal build src/jedi_cli.cr --error-trace -o bin/jedi-dev
crystal build src/jedi_cli.cr --release --error-trace -o bin/jedi
```

Two small Bash scripts, `build-dev` and `build-release`, can wrap these commands. They produce separate files under `bin/`. Keep generated binaries outside versioned source files.

From the project root, select the development binary for this terminal:

```
export JEDI_DEV_BINARY=./bin/jedi-dev
jedidev config --paths
jedidev trade list
```

Rerun the quick compilation after a source change. Before final validation, also run `TZ=UTC crystal spec --release`, then build the optimised executable. Local compilation follows the host's libraries; the distributable static binary is built by the publication procedure.

Back up and recover data

Between Jedi commands, with none running, back up the configuration and databases listed by `config --paths` for every environment you need. Copying configuration and data directories retains the main database and market databases. Also retain original files and lists needed to rebuild them. The log can help investigate an incident.

Jedi additionally provides an export of the **main database**, including notes and recorded quotations. Here the production file is written to the chosen absolute path:

```
jedi database dump > "$HOME/jedi-sauvegarde.txt"
```

This export contains neither `jedi.conf` nor market databases. To test restoration, create a new directory and initialise its main database:

```
mkdir -p "$HOME/jedi-restauration"
JEDI_DEMO_DIR="$HOME/jedi-restauration" jedidemo config --paths
JEDI_DEMO_DIR="$HOME/jedi-restauration" \
  jedidemo database load "$HOME/jedi-sauvegarde.txt"
JEDI_DEMO_DIR="$HOME/jedi-restauration" jedidemo trade list
JEDI_DEMO_DIR="$HOME/jedi-restauration" jedidemo note list
```

`load` requires a completely empty database, including notes. It does not merge the export into an existing portfolio. Check trades, operations, notes and settings before resuming. The `jedi` launcher resumes your real portfolio; `jedidemo` without this prefix resumes the usual demonstration. To restore copied files, return each file to its intended location with no Jedi command running.

Under WSL, you can also create a complete distribution copy from PowerShell:

```
wsl --shutdown
wsl --export Ubuntu jedi-wsl-backup.tar
```

Adjust the distribution name. The Microsoft reference describes distribution export and import.

Update Jedi

Back up data, read release notes, then verify and install the new executable using the procedure above. Replacing the program retains your environment files. After installation, check `jedi version`, `jedi config --paths` and a few known records.

You can now choose a security, or go directly to EXA's fictional opening.

3. Choosing a security

Start with a working list

The security is the instrument to manage within a trade. Selecting it comes before choosing a scale and evaluating the amount to commit.

You can start with a personal watchlist, a market platform, an external screener or `tools profile`. In every case, the securities found form a list to examine. The user keeps the final choice.

To understand a result, record the population studied and the quotation date. An Euronext Growth list and a list spanning several markets compare different securities. The same security can change rank when the population changes.

Four practical questions

Main criterion	Question	Practical reading
Trend	Is the price broadly rising, falling or stable?	A persistent decline can lead to successive purchases and tie up capital.
Liquidity	What amounts are normally traded?	Compare this information with the proposed quantities and amounts.
Volatility	How large are daily price movements?	Small movements limit opportunities; large movements call for more attention.
Oscillation	Do movements regularly change direction?	Round trips depend on movements in both directions.

A large daily movement and regular oscillation are different. A security falling sharply every day can be volatile while providing few upward reversals.

Three criteria complete this reading: **maximum drawdown**, the decline from a previous peak; **gaps**, differences between a close and the following open; and the **suggested step**, a movement size proposed as a starting point for examining a scale.

Chapter 04 explains the measures, thresholds and seven ranking criteria. Starting with the four main criteria helps read the report; additional cautions still need examination.

Use an external selection

A screener filters a population according to conditions. The JEDI ProScreener in Appendix 03 uses daily bars and looks for a compatible trend, minimum liquidity and sufficient volatility. It also displays oscillation.

Explicitly choose the market or security list in the platform, then read the results for a completed session. Pick a few securities and examine their history, volumes and recent events.

External selection and `profile` should provide coherent lists on the main criteria. Differences can arise from the date, provider, price adjustments, trend calculation or available history. Perfect agreement between lists is not a selection objective.

The JEDI script and procedure are included in Appendix 03.

Use a Jedi market database

To try commands before preparing your own data, follow the start of chapter 10: the Demo database for the examples. It creates source `demo` and dataset `Demo`, then updates them. Chapter 04 uses them to explain `profile` results.

A market database organises histories into **datasets**, imported collections, and can bring securities together in **groups**, logical lists defined by the user.

```
jedidemo tools profile --database=abcbourse --dataset=Euronext_A --rank-criteria=4
```

This command examines the securities in `Euronext_A` and sums their ranks on the first four criteria. Replace `abcbourse` and the dataset with those of the database actually prepared.

A group can combine securities from several datasets:

```
jedidemo tools profile --database=abcbourse --group=PEA --rank-criteria=4
```

Combining `--dataset` and `--group` selects their intersection. `PEA` is a list stored in the database; its name does not automatically verify each security's eligibility.

Jedi's preparatory examples include French histories and the datasets `Euronext_A`, `Euronext_B`, `Euronext_C`, `Euronext_Access` and `Euronext_Growth`. These names do not prescribe which market the application must be used on.

Work with an individual file

A market database is unnecessary when studying a security already chosen. A **prepared file** contains its quotations in a format Jedi can read, with its identity and checked OHLCV rows. Chapters 05 and 10 explain preparation.

The fictional cotations_EXA.txt file is in the **examples/** directory supplied with this documentation. Data are provided as usable companion files, separately from the chapter text. The example guide explains terminal preparation and how to keep experiments separate.

After copying the files to `~/jedi-examples/` in chapter 02, the following commands use the file directly. `jedidemo` opens that directory; no market database import is needed:

```
jedidemo tools profile cotations_EXA.txt
jedidemo tools score cotations_EXA.txt --sort=fitness --budget=40
```

The first report describes the security; the second searches for scales. A budget of 40 provides a short demonstration. The application's standard budget is 500.

`profile` calculates recent criteria on 252 sessions, with a quality check over the long history. `score` retains 750 sessions by default to simulate more situations. Chapter 05 explains the difference and how to examine another period.

Move on to the trade proposal

After choosing a scale, evaluate the candidate with its ISIN, a scale definition and a checked current price:

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --amount=1000 --capital=5000
```

This example uses a **fictional** security and price. It evaluates an initial purchase of 100 shares and the proposal's requirements. It does not read prices from the `EXA` file: the €10 price is supplied explicitly to illustrate the calculation.

Before retaining a security, read its cautions, examine the scale, check useful orders and funding. If a difficulty remains, choose another proposal or clarify the data rather than automatically retaining the first ranking row.

4. Shortlisting with tools profile

What the command provides

`profile` describes a security's behaviour and compares a population to help establish a list of choices. It checks quotation quality, applies rejection and caution rules, then ranks the retained securities.

The ranking is transparent: it sums ranks without weighting. The reader can choose how many criteria contribute to the total and consult the values explaining those ranks.

Prepare the examples

The calculated examples in this chapter use three fictional securities. Their files are in **examples/** at the root of the documentation archive; the example guide explains settings and the working directory.

Before running `--database=demo --dataset=Demo`, complete the Demo database preparation at the start of chapter 10, through the update of all three securities to 2 October 2026. That procedure creates the database; `profile` does not.

The command using cotations_EXA.txt reads this file directly from `examples/` and requires no import. The `abcbourse` and `Euronext_A` examples below assume that the user has prepared those data.

Choose the population

```
jedidemo tools profile --database=abcbourse --dataset=Euronext_A --rank-criteria=4
```

`--database` selects a market database source. `--dataset` selects an imported collection. The universe used in the database name comes from the configuration, or from `--universe` when specified.

Other possible selections:

Requirement	Options
All members of a group	`--group=PEA`
Group members belonging to a dataset	`--dataset=Euronext_A --group=PEA`
One identified database security	`--security=RNO` or its ISIN
Several identified securities	Repeat `--security`

An individual file can also be examined:

```
jedidemo tools profile cotations_EXA.txt
```

With a file, the report describes one security; it establishes neither a population ranking nor a dataset freshness reference.

A recent window and a checking history

Trend, liquidity, oscillation, maximum drawdown, gaps and the suggested step use the security's **last 252 sessions**. Volatility uses a **20-session ATR**. ATR averages daily true ranges, including differences from the previous close; it is expressed as a percentage of the last close.

252 sessions roughly correspond to one trading year. This window describes recent behaviour. Jedi nevertheless keeps the longer history to check data quality and continuity.

The main quality checks require at least **three years of history** and a **minimum density averaging 180 quotations per year**, over that entire history. Jedi calculates density by dividing the number of available quotations by the time between the first and last dates, expressed in years of 365.25 days.

180 is a quality threshold set in Jedi, intended to reject overly sparse histories. It does not describe the usual number of sessions in a trading year or replace the recent 252-session window. The average covers the whole period; it does not ensure that every year is complete.

Inconsistent OHLC prices, too many sessions without volume or too many unchanged closes can also reject the security. These requirements explain why a file limited to 252 rows can be insufficient.

The individual report states the dates actually examined. Available sessions in each history determine the window: a lagging security can have a shifted window.

Read the statuses

Status	Practical meaning
Retained	No rejection or caution condition detected by these checks.
Caution	The security remains in the ranked list, with points to examine.
Rejected	At least one rejection rule is triggered.

The main rules are:

Measure	Rejection	Caution
Annualised trend	At or below −12% per year	Also examine the path and other measures.
Median daily close × volume	Below €100,000 for euro-based examples	More than 30% of sessions fall below this amount.
ATR20	Below 1.5%	Above 6%.
Oscillations of at least 5%	—	Fewer than four per year.
Maximum drawdown	—	Above 45%.
Gaps of at least 3%	—	Present in more than 3% of sessions.
Quotation lag	More than two sessions	One or two sessions.

The median is the central daily traded value: it reduces the influence of a few exceptional days. Trend is calculated from the whole window, rather than simply from its first and last prices.

Liquidity is measured in the monetary unit of the imported prices. The numeric threshold of 100,000 performs no currency conversion. For non-euro data, account for their unit.

The seven criteria, in ranking order

Order	Criterion	What to examine
1	Trend	A less unfavourable or more favourable direction receives a better rank.
2	Liquidity	A higher median traded value receives a better rank.
3	Volatility, ATR%	Jedi first ranks volatility bands, then values within the same band.
4	Oscillation	Number of oscillations and range; classes consider both measures.
5	Maximum drawdown	A smaller decline receives a better rank.
6	Gaps	A lower frequency receives a better rank.
7	Suggested step	A smaller step receives a better rank.

For volatility, the 2–5% band ranks ahead of the 1.5–below 2% and above 5–6% bands. Still higher volatility calls for caution. “More volatile” therefore does not systematically mean “better ranked”.

Oscillation favours sufficiently frequent movements of at least 5%, accompanied by a sufficient range. `Sw5/y` counts the first significant movement and subsequent reversals, annualised; it does not count complete round trips or Jedi operations. The individual report and help explain the bands used:

```
jedidemo tools profile --help
```

The movements-per-year column estimates activity associated with the suggested step. It is additional information, not an eighth ranking criterion. The proposed step is a starting point for examining a scale.

Start with the four main criteria

The `--rank-criteria=4` option sums the trend, liquidity, volatility and oscillation ranks. This provides a simple starting reading.

The application’s default remains **seven criteria**:

```
jedidemo tools profile --database=abcbourse --dataset=Euronext_A
```

`--rank-criteria=N` accepts 1 to 7. It uses the **first N criteria in the order above**. Subsequent criteria remain displayed and their cautions remain active.

In the ranks column, the vertical bar separates included ranks from informative ones. For example, `1/2/3/2 | 4/3/2` gives a total of 8 with four criteria. This sum is an illustration.

Identical values receive the same rank. The smallest total comes first; ties are resolved by comparing the included ranks in order, then by ticker. Changing the population can change ranks and totals.

Read a calculated example

After creating and updating the Demo database at the start of chapter 10, run from `examples/`:

```
jedidemo tools profile --database=demo --dataset=Demo \
  --rank-criteria=4 --include-rejected
```

Here is a selection of columns from the actual output, shortened for readability:

Ticker	Total	Trend/year	Median value	ATR20	Oscillations/year	Status
EXA	4	3.7%	€774k	3.4%	8.3	retained
EXB	—	−27.4%	€377k	3.1%	11.4	rejected
EXC	—	−0.5%	€12k	1.1%	0.0	rejected

EXB has liquidity and oscillations, but its trend alone rejects it. EXC is rejected for insufficient liquidity and volatility. EXA is the only ranked security, so it receives rank 1 on each of the four criteria.

`--include-rejected` displays rejected securities with a dash for total and ranks. Their presence does not change the ranking of the others. Securities marked “caution” remain in the ranking.

To understand the reasons, consult the security individually:

```
jedidemo tools profile --database=demo --security=EXA
```

Understand freshness

Jedi uses the dataset’s global freshness date: the latest date reached by **at least 95% of its securities**. This date is recalculated from stored quotations after import or update.

Lag is counted using the sessions present in the dataset database, up to this reference. It is zero when the security reaches or exceeds the reference, one or two sessions for caution, more than two for rejection.

This compares securities within the dataset. It does not measure distance from today’s date: if the whole dataset has stopped receiving quotations, check the date shown by `market info`.

The freshness check is unavailable with a file or a historical `--date` or `--select` selection. It is also unavailable if the dataset reference is missing, or if a security belongs to several datasets without an explicit `--dataset` resolving the ambiguity. A dash in the lag column means “not assessed”, not “up to date”.

Choose a few securities

Start with the four main criteria, then read cautions and the three additional criteria. Examine a large drawdown or frequent gaps before retaining a well-ranked row.

For a particular reading, use a named sort:

```
jedidemo tools profile --database=demo --dataset=Demo --sort=liquidity
jedidemo tools profile --database=demo --dataset=Demo --sort=oscillation
```

Option values remain English in both languages. `--limit=20` limits displayed rows; it does not reduce the population analysed to calculate ranks.

Retain a few understandable securities, then examine their scales with `score` and their proposals with `candidate`. Ranking helps organise the choice; entry price, quantities and funding are examined separately.

5. Searching for a scale with tools score

A search for an already chosen security

`score` simulates several scales on one security's history and ranks those it has evaluated. The security comes from the user's choice, `profile` or an external selection.

Standard use combines an **adaptive** search, **raw** ranking and **compact** display. "Raw" means results directly sorted using the three displayed measures. Adaptive search explores configurations, then examines promising areas more closely according to the selected sorting criterion.

Choose the quotations

From an individual file:

```
jedidemo tools score cotations_EXA.txt --sort=fitness --budget=40
```

From an existing market database:

```
jedidemo tools score --database=demo --security=EXA --sort=fitness --budget=40
```

The filename is a positional argument. For database use, `--database` identifies the source and `--security` the ticker or ISIN. `--universe` selects a universe other than the configured default.

What "prepared file" means

A prepared file is a history acquired from an identified source, converted into the format Jedi reads and checked before use. For one security, the recommended first row contains **ISIN; ticker; name**. Subsequent rows contain **ISIN; date; open; high; low; close; volume**.

Valid extract from the supplied fictional file `cotations_EXA.txt`:

```
FR4487601001;EXA;Exemple A
FR4487601001;20230630;10.0000;10.1800;9.8200;10.0000;60000
FR4487601001;20230703;10.1075;10.2895;9.9173;10.0990;61000
```

These two quotations illustrate the format; they are insufficient for a representative study. The supplied file contains 851 fictional sessions, from 30 June 2023 to 2 October 2026.

Every row must concern the same ISIN. Prices use a decimal point; the dates above are year, month, day. Chronological ordering makes checks easier. Avoid duplicates, check missing sessions and identify any price corrections.

Provider exports can require conversion before reading: keep the required columns, add the ISIN and adapt dates. `market` commands do not download these files. Chapter 10 distinguishes individual files from multi-security import files.

Why 750 sessions after a profile over 252?

`profile` examines recent security behaviour, mainly over 252 sessions, roughly one year. `score` evaluates scale paths over **750 sessions by default**, roughly three years. This longer period exposes the mechanics to more movements and historical conditions.

These settings answer different questions: "how has the security behaved recently?" and "how did these scales operate over the selected history?" They do not demonstrate that these durations are always best.

With the demonstration file, the earlier command reports:

```
History: 2023-11-20 → 2026-10-02 (750 sessions)
Requested opening amount: 1500.00 €
Search adaptive: 40 backtests, 11793 available candidates (budget: 40)
Partial search: the ranking covers the evaluated scales.
Ranking: transparent raw results (raw)
```

If the file contains fewer than 750 sessions, the report states the period actually available. The three-year check specific to `profile` is not an additional `score` rule.

Budget and search

Without `--budget`, adaptive search has a budget of **500 search backtests**. In this chapter, 40 gives a short demonstration. The budget counts scales actually simulated, not displayed rows.

The search does not necessarily test every available candidate. Its conclusion concerns evaluated configurations; the first row does not prove a global optimum.

A progress percentage is displayed during calculation. Time depends on the budget, history length and scales. Exhaustive `--search=grid` remains available for detailed use and can take much longer.

Read the three compact columns

Compact `raw` results are limited to the **first 20 scales**, or fewer when fewer were evaluated. The following three rows are a reference extract from the 1.0 edition's fictional study, with a budget of 40. After the maximum-investment correction in v1.1, rerun the command to obtain values and ranking for your version:

```
 Fitness   Op/mo   Max.inv  Typed builder
   0.433    0.644    1.000  I: kc:1.23,1.14,0.88,0.76, ki:0%, kb:1, kd:0.2
   0.416    0.532    1.000  I: kc:2.01,1.75,1.52,1.32,1.15,0.85,0.72,0.61,0.52,0.44, ki:0%, kb:1, kd:0.0
   0.414    1.960    1.050  I: kc:1.22,1.16,1.1,0.95,0.9,0.86, ki:0%, kb:1, kd:0.0
```

Column	Meaning
Fitness	Final gross gain divided by the first purchase amount. `0.433` means roughly 43.3% of that first purchase over the whole period.
Op/mo	All operations, including openings, expressed per 21 sessions from the first operation.
Max.inv	Maximum investment divided by the first purchase, excluding fees. `1.050` means a peak of 1.05 times that purchase.
Typed builder	Type I or II followed by a definition that recreates the scale.

Fitness is neither an annual rate nor a net return. Gross gain includes remaining holdings valued at the final close. A rise in the security can therefore contribute without coming from round trips.

Maximum investment is the largest of the successive trades' peaks of purchases minus sales, excluding fees, as in backtest. It can exceed the first purchase when the mechanics call for additional purchases. Fees must be examined in the economic display or a backtest.

Choose the sort

Option	Primary criterion	Tie-breaking
`--sort=fitness`, default	Fitness descending	Activity descending, then maximum investment ascending
`--sort=activity`	Activity descending	Fitness descending, then maximum investment ascending
`--sort=investment`	Maximum investment ascending	Activity descending, then fitness descending

Sorting compares measures rounded to three decimal places. The old numeric values `1`, `2`, `3` remain accepted; examples use explicit names.

In adaptive search, the criterion also guides refinement. Changing the sort can therefore change the scales evaluated as well as their order. To compare the paths of fixed scales, use their definitions in `backtest`.

High activity increases interventions and can lead to more fees. Low capital use and high fitness do not suffice to establish the real proposal's operability.

Examine the economic display

```
jedidemo tools score cotations_EXA.txt --sort=fitness --budget=40 --display=study
```

This display retains the ranking principle and adds net gain, the result relative to maximum investment, fees and operations. In the first row of our example, gross fitness of 0.433 becomes net fitness of 0.421; displayed fees are €19.

`Rnet%` and `Rann%` express net results relative to maximum investment. `Op/y` counts scale operations excluding openings on an annual basis; it does not exactly replace `Op/mo`. Rounded monetary amounts provide an overview.

`--ranking=jedi`, `--ranking=both` and `--display=technical` remain available for expert examination. Their details appear in appendix 01. Standard use starts with `raw`.

Examine another period

```
jedidemo tools score cotations_EXA.txt --sort=fitness --select=l252 --budget=40
```

This command uses the last 252 sessions. In our example, the first row over 750 sessions has fitness of 0.433; the first row over 252 sessions has fitness of 0.126 and a different builder.

These gains cover different durations and can concern different scales. Their simple comparison does not establish one window's superiority. Fixing a scale and simulating it over the intended periods allows its behaviour to be examined.

To end the search at a specific date:

```
jedidemo tools score cotations_EXA.txt --date=le:2025-12-31 \
  --select=l750 --sort=fitness --budget=40
```

The date filter is applied before row selection. Codes are `ge` for "from", `gt` for "after", `le` for "up to and including", and `lt` for "before".

Pass a scale to candidate

The `I:` or `II:` prefix identifies the type in the report; it is not part of the supplied value. Copy the text following the prefix:

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" \
  --amount=1000 --capital=5000
```

This definition comes from the first demonstration row. The €10 price and amounts are fictional. `candidate` checks the current proposal and displays the operational scale definition; it makes no purchase.

Check a price with calc

For the type I scale copied above, Jedi can check the first upward step's price:

```
jedidemo tools calc price \
  scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" \
  position=0:0:1 c0=10 tick=0.01 dir=up
```

The resulting output starts with:

```
price = round_up(cpe) = 11.4
  cpe = 11.4
```

The target position is the first upward step, the reference price is €10 and the tick size is €0.01. This checks one scale price rather than simulating the whole trade. For a type II scale or a sequence of operations, use `tools play`.

Retain an understandable solution

First read dates, evaluated scale counts and measures. Then examine a few configurations, their activity, fees and capital requirements. Similar fitness can accompany very different trading rhythms.

Simulation uses the `backtest` engine, fixed-amount openings, `conservative` execution and orders active from the next session after opening. The requested amount comes from the simulation configuration and appears in the report. These examples use €1,500 per opening with Jedi's default configuration.

Keep the selected definition and study settings. Check its path over another period, then evaluate the candidate at the current price. Changing dates or settings after seeing the gains must be considered a new attempt.

6. Evaluating an opening with tools candidate

Two questions before opening

`candidate` evaluates a trade proposal at a supplied or recorded price. It checks:

- **operability**: can the quantities produce the useful orders being checked?
- **funding**: do free funds cover the opening and projected additional purchases?

These results remain separate. The command does not find the security or select its scale. It requires no OHLCV history, creates no trade and sends no order.

Identify the security, price and scale

The candidate argument is an **ISIN**, not a ticker. A price explicitly supplied with `--price` overrides the main database price. It even allows evaluation of a security not yet recorded there.

Without `--price`, the security must have a recorded price in the main database. The report states its origin and, for a recorded price, its known date. A market database's last close is not used automatically.

A supplied price does not inherit the date of an old stored quotation. The user checks the date and suitability of the supplied price.

`--scale` accepts an existing scale name or a literal definition. After `score`, copy the builder without the `I:` or `II:` display prefix.

Choose the opening amount

Here is a **fictional** example, with no other committed trade in the main database:

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --amount=1000 --capital=5000
```

`--amount=1000` sets the desired first-purchase amount. At €10, it corresponds to 100 shares. When the amount is not exactly divisible by price, Jedi uses a whole quantity; the actual committed amount can be smaller than requested.

`--capital=5000` supplies portfolio capital for the funding check. This capital and the opening amount serve different purposes.

Actual output:

```
Opening quantity :               100
Actual opening amount :          1000.00 €

Operability :                    YES
Funding :                        YES
Capital free after reserve :     5000.00 €
Candidate maximum requirement :  1187.83 €
Margin after candidate :         3812.17 €
Candidate reserve calculated over 3 steps; funding excludes fees.
```

The proposal requires more than its initial purchase: an additional €187.83 is reserved for the projected path. The scale is also displayed through its complete operational definition.

Size using capital management

The second form uses `--money` instead of `--amount`:

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:1%,o:1,n:2" --capital=10000 --extended
```

The selected values explain the calculation here.

Parameter	Meaning
`r`	Capital proportion used to calculate monetary risk.
`n`	Number of downward steps used to calculate the StopLoss distance.

Parameter	Meaning
o	Multiplier of the monetary target used for gain protection.

With €10,000 and `r:1%`, calculated risk is €100. The scale gives a theoretical price of €8.10 two steps below €10, a distance of 19%. This gives a theoretical amount of €526.32 and 52 shares for €520.

Selected values from the extended output:

```
Risk amount R :                  100.00 €
Opening quantity :               52
Actual opening amount :          520.00 €
Minimum operable quantity :      32
Operability :                    YES
Candidate maximum requirement :  617.56 €
Margin after candidate :         9382.44 €
```

The calculation also considers the capital limit along the risk path and useful minimum quantities. It can refuse an opening when no whole quantity meets all constraints.

Without `--capital`, Jedi uses the capital configured for trades. If no positive capital is available, funding can be shown as “not evaluated”; that does not mean it has passed.

Choose r, o and n in practice

Start by expressing risk in euros, then relating it to the selected capital. With €10,000, 1% is €100 and 2% is €200. Increasing `r` increases accepted risk and can increase quantity; it is not a neutral way to remove a refusal.

Choose `n` after reading the scale: two steps do not represent the same percentage for different movement sizes. More steps generally place the StopLoss farther away and, for identical monetary risk, produce a smaller opening quantity.

This comparison uses three commands executed on our example, changing one parameter at a time:

Capital	r	n	Calculated distance	Quantity	Initial purchase	Maximum requirement excluding fees
€10,000	1%	2	19.00%	52	€520	€617.56
€10,000	2%	2	19.00%	105	€1,050	€1,253.22
€10,000	1%	3	27.10%	36	€360	€425.88

Here are the three complete commands, in the same order as the table rows:

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:1%,o:1,n:2" --capital=10000 --extended

jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:2%,o:1,n:2" --capital=10000 --extended

jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --money="r:1%,o:1,n:3" --capital=10000 --extended
```

`o` multiplies monetary risk to define the target used for gain protection. With €100 risk, `o:1` gives a €100 target and `o:2` gives €200. It does not directly set the opening quantity. Triggering depends on the result projected after `n` downward steps and the trade path; it is not simply “sell when current gain reaches €100”.

For initial learning exercises, keeping `o:1,n:2` and changing one parameter at a time makes comparisons readable. These example values do not by themselves determine suitable risk for every investor. Chapter 18 and `tools play` will show the effects on protective orders.

A StopLoss can execute beyond the calculated price, particularly during a gap. Calculated risk is therefore not a guaranteed loss limit. Also examine aggregate risk across committed trades.

Understand operability

Operability checks whether useful orders retain a sufficient quantity. The default is at least two shares per order and three upward steps checked, according to `diagnostic_steps` in the configuration.

`--minimum-quantity` and `--operability-steps` change this check. Reducing requirements changes the question asked; it does not automatically solve fees or liquidity.

For our scale, the calculated initial minimum is 32 shares. With €5,000 capital, `r:1%,o:1,n:2` allows only 26 shares under the risk constraint. The proposal is refused for insufficient quantity. The display gives zero opening quantity and the reason, although free capital is available.

Understand reserves and funding

Jedi takes portfolio capital, subtracts current net investment, then reserves projected additional purchases for trades still holding shares. The remainder is **capital free after reserve**.

For the candidate, maximum requirement is the initial purchase plus its own reserve. **Margin after candidate** is capital free after reserve minus this requirement.

Projection explores upward and downward paths over the chosen horizon. For reserves, this horizon is `diagnostic_steps`, increased to `n` when capital management uses a larger `n`. It is separate from the operability option.

Fees are excluded from this funding projection. Requirements beyond the horizon, new decisions and actual execution conditions can change the outcome. Leave room for these differences.

With the same €1,000 opening and only `--capital=1000`, the result is:

```
Operability :                    YES
Funding :                        NO
Candidate maximum requirement :  1187.83 €
Margin after candidate :         -187.83 €
```

The initial purchase fits the amount, but funds do not cover the projected reserve. A favourable price or historical result does not change this calculation.

When existing trades have exceeded R

A loss greater than calculated risk `R` can occur, for example after a gap or an interruption in monitoring. Capital management with `money` cannot retrospectively restore that limit. When calculating orders, Jedi can warn that a trade is out of control if its loss at the current price exceeds `R`.

`candidate` does not automatically block a new opening for this reason. It does not compare other trades' current losses with their `R`. It continues to include all trades still holding shares in reserves, including those that have exceeded this risk. Their reserve is projected from the last recorded state and scale over the chosen horizon; it does not assume that a StopLoss liquidated the trade.

Funding uses net investment: recorded purchases minus recorded sales in the main database. It does not replace this commitment with the shares' current market value. A recorded sale at a loss therefore affects calculated funds; an unrealised loss is not deducted as an additional risk budget. Capital used to calculate the candidate's `R` remains the value supplied by `--capital` or configuration. It is not automatically recalculated from other trades' losses.

With several trades in this situation, `candidate` can therefore display "funding: YES" when funds cover commitments and reserves. This result validates that funding calculation; it does not validate aggregate portfolio risk or continuation of troubled trades.

To examine existing trades' warnings, run:

```
jedidemo trade orders
```

The extended `candidate` report, requested with `--extended` in the commands above, details net investment, total reserve and each trade's reserve. Record exceptional incidents and decisions with `note`, identifying the trade concerned; separately record operations actually executed.

Read a refusal before changing the proposal

Result	What to check
No price available	Supply a valid price or record a quotation for the security.
Zero opening quantity	Amount, price and whole quantity.
Insufficient quantity	Scale, useful quantities and risk settings.
Capital is limiting	Initial purchase and projected investment on the risk path.
Negative margin	Free funds and reserves for existing trades and the candidate.

`--extended` supplies useful details. Prices, amounts and capital must be positive and finite; `--amount` and `--money` are incompatible.

Before opening, retain the ISIN, checked price, scale definition, settings and report. Chapter 07 then explains broker execution and actual recording in Jedi.

7. Opening a trade

Move from the proposal to actual operations

Chapter 06 evaluated a proposal. `candidate` creates no security, scale, trade or purchase in the main database. It sends no order to the broker.

Opening has three stages: prepare the trade in Jedi, execute the purchase through the broker, then record the actual execution. An operation merely contemplated must not be recorded.

This chapter uses **EXA, an entirely fictional security**: an initial purchase of 100 shares at €10, a scale with 10% movements and €5,000 reference capital. Use an empty experiment database prepared in chapter 02. Historical EXA files can exist in a market database; they do not create this security in the main database.

Check the proposal at the chosen price

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0" \
  --amount=1000 --capital=5000
```

In a database without other committed trades, the result gives:

Information	Value
Opening quantity	100
Initial purchase	€1,000.00
Maximum requirement, opening and reserve excluding fees	€1,187.83
Margin after candidate	€3,812.17

Operability and funding pass for this example. Recheck if price, amount, settings or portfolio commitments change before execution.

This example fixes an amount with `--amount` and leaves `money` disabled. If your proposal was sized with `--money`, use the same `r`, `o` and `n` when creating the trade. Capital used during management comes from `trading.capital`: the `--capital` supplied to `candidate` is not automatically stored for the trade.

Record the security and a current price

First check whether the security exists:

```
jedidemo security list EXA
```

Create it in our empty database:

```
jedidemo security create --isin=FR4487601001 --name="Exemple A" --ticker=EXA \
  --tick-buy=0.01 --tick-sell=0.01 --tax-rate=0
jedidemo quote set FR4487601001 --price=10
```

For an actual security, check its ISIN, name, ticker, quotation ticks and any tax. The €0.01 ticks and zero tax are fictional choices in this example.

`quote set` records the price with the entry date and time. This price supports monitoring; it does not replace the purchase's actual execution price. The price supplied to `candidate` had not been written to the database.

Choose readable names

For a trade, use **`<security_label>_<number>`**: `Exa_01`, then `Exa_02` if a separate new trade is created on the same security. A longer label such as `Airliquide_01` is also suitable. The number identifies the trade, not its settings. A split, scale change or reinitialisation retains the name.

For a scale, let Jedi assign **`B_kc_NN` for type I or `B_vr_NN` for type II** by omitting `--name`. The type comes from the builder and the number is computed within that family. In our empty database, the first type II scale will be `B_vr_01`. Chapter 13 explains naming and refusal of equivalent definitions.

Use unaccented letters, digits and underscores _, starting with a letter and using at least two characters. Names must be unique within their category and are case-insensitive. Jedi normalises their spelling: `EXA_01` becomes `Exa_01`.

This edition's examples adopt these names. A trade created under an older name remains usable; use its actual name in commands or correct it with `trade update --name`, as in chapter 09.

Retain the selected scale

Trade settings require a permanent scale in the main database. Store the candidate's definition under a name:

```
jedidemo scale create \
  --comment="Exemple avec mouvements de 10 %" \
  --builder="vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0"
jedidemo scale show B_vr_01
```

If the scale already exists, inspect and reuse it. When it comes from `score`, copy the builder without its display prefix `I:` or `II:`. The temporary name of a literal evaluation scale does not retain it permanently.

Create the trade and its initial settings

```
jedidemo trade create --name=Exa_01 --security=FR4487601001
jedidemo trade show Exa_01
```

The name identifies this management history and must be unique. `--security` accepts the security's ISIN or internal ID, not its ticker. If the proposal uses capital management, add the evaluated definition, such as `--money="r:1%,o:1,n:2"`, to the creation command.

Then associate the trade with the scale:

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-21 --time=09:00:00
```

Before the purchase, these initial settings have position `0:0:0`, with `q0` and `c0` at zero. The first operation fills the reference quantity and price. Adaptation options `--split`, `--sync` and `--reinit` apply to a trade already having settings.

Respect chronology across the whole main database. New settings must be strictly later than the latest operation and settings, even for another trade. An operation must follow the latest global settings and cannot precede the latest global operation. Entered dates and times are interpreted in the terminal's time zone.

At this point Jedi knows the proposal, but no purchase is recorded yet.

Buy through the broker and record execution

Place the purchase yourself, monitor execution and use the broker's confirmed values. Our example assumes 100 shares executed at €10, with €3.50 fees:

```
jedidemo operation create Exa_01 \
  --date=2026-09-21 --time=09:01:00 \
  --type=Buy --mode=Limit --quantity=100 --price=10 --fees=3.50
```

The first operation is a `Limit` or `Market` purchase according to the order actually executed. Its timestamp differs from the settings timestamp. Quantity and price are execution values; they can differ from the prior evaluation.

`--fees` supplies the operation's total fees, including any tax. If omitted, Jedi applies configured fees and the security's purchase tax. Enter definitive fees explicitly when provided by the execution statement. After recording, `operation fees ID AMOUNT` can correct this total on any operation; chapter 08 explains how to find its identifier and check the correction.

An unexecuted purchase creates no operation. If executed in several fragments, wait until you can record consolidated execution; chapter 08 explains how.

Check the opening and initial orders

```
jedidemo settings list Exa_01
jedidemo operation list Exa_01 -c 0,6-14,17
```

```
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Stock must be **100 shares**, `q0` 100, `c0` €10 and position `0:0:0`. Net investment excluding fees is €1,000. Compare stock and the operation with the broker's statement.

The two operating orders obtained are:

Type	Mode	Quantity	Price	Amount excluding fees
Buy	Limit	8	€9.00	€72.00
Sell	Limit	6	€11.00	€66.00

These rows are calculated proposals. You still have to place orders through the broker and monitor their execution. Do not record them as operations before they occur.

To examine an upward movement followed by a downward movement without changing database operations:

```
jedidemo tools play -c o0,6-14,17 -- \
  scale=vr:10%,np:3,nn:3,ki:10%,kb:1,kd:0,q0=100,c0=10,tick=0.01 1 -1
```

The calculated path sells 6 shares at €11, then buys 6 at €9.90: stock returns to 100. `play` helps understand the scale here; it describes no actual execution and does not guarantee that path.

Retain the scale definition and opening report. Chapter 08 continues this trade with monitoring and recording of an actual sale.

8. Monitoring a trade daily

A monitoring loop

Monitoring means obtaining a relevant price, reading status and orders, maintaining orders through the broker, then recording only actual executions. Recalculate orders after an operation.

Three kinds of information remain distinct:

Information	What it describes
Recorded price	A price observation used for monitoring.
Calculated orders	The next operations proposed from the trade's state.
Recorded operations	Purchases and sales actually executed.

A price change does not alter stock. Placing or cancelling a broker order does not change the Jedi database. A broker execution appears there only after you record it.

This chapter continues **fictional trade Exa_01 opened in chapter 07**: 100 shares bought at €10, scale `B_vr_01`. Recording commands are experiments for the separate database prepared in chapter 02.

Obtain a relevant price

For fictional experiments, enter the desired price:

```
jedidemo quote set FR4487601001 --price=11
```

`quote set` stores the price with the entry time, source `manual` and no retained variation. It cannot know when your observation was made: check that the price corresponds to the intended session and time.

With configured providers, usual commands are:

```
jedidemo quote update FR4487601001
jedidemo quote refresh
jedidemo quote refresh --mode=eod
```

`update` refreshes a registered security. By default, `refresh` updates securities linked to open trades. To collect data for an actual security, replace the fictional ISIN with your registered security's ISIN: EXA has no market-provider price.

`--mode=eod` requests end-of-day mode; `--mode=live` requests current-price mode. Content, delays and availability depend on the provider. A mode option can also be supplied to `quote update`.

For consultation without writing the result to the main database:

```
jedidemo quote get FR4487601001
```

Detailed provider configuration appears in appendix 02. Manual entry allows monitoring when collection is unavailable. Chapter 10's `market update` commands maintain histories; they do not replace these monitoring prices.

Monitoring commands can attempt to refresh a price whose configured retention time has expired. If collection fails, an old price can be reused with a warning. Read the warning and check the price's relevance before using orders.

Read status and orders

```
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Compare displayed stock with broker stock. Read each order's `Buy` or `Sell` type, mode, quantity, price and amount. At €11 and without another operation, Exa_01 still holds 100 shares and proposes the previous chapter's orders.

Mode	Practical meaning
Limit	Limit order.
Stop	Operating order triggered when a threshold is crossed.

Mode	Practical meaning
StopLoss	Protective order resulting from `money` capital management.

An operating Stop and a StopLoss have different purposes. Even with a scale described as optimised, read the mode actually proposed; some paths can produce an operating Stop.

The **READY** mark means the known price has crossed the order price in the direction corresponding to execution. **WATCH** means proximity under the configured threshold. An exactly equal price can still be marked “watch”. Neither mark proves broker execution.

If Jedi reports insufficient quantities, an old price or inapplicable capital management, examine the reason before continuing. Chapter 06 describes losses exceeding R.

Maintain orders through the broker

After calculation, compare Jedi’s rows with actual active orders. Cancel outdated orders, then modify or replace those that need it. Check the broker’s final summary: direction, mode, quantity, price and validity period.

A sale can reduce stock available for another sale order. The broker and Jedi do not automatically coordinate orders. After execution, check remaining orders before keeping them.

When a StopLoss and Limit sale must coexist, `trade orders --net-stop` can display a StopLoss quantity reduced by the upward sale’s quantity. This option creates no cancellation link between broker orders; you remain responsible for coordinating them.

Record an actual sale

Our example assumes a confirmed sale of 6 shares at €11 with €0.23 fees. Record it:

```
jedidemo operation create Exa_01 \
  --date=2026-09-22 --time=10:15:00 \
  --type=Sell --mode=Limit --quantity=6 --price=11 --fees=0.23
jedidemo operation list Exa_01 -c 0,6-14,17
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Stock becomes **94 shares**. Net investment excluding fees decreases from €1,000 to **€934**, since the sale received €66. Position becomes `0:0:+1`.

These are recalculated orders, including rounding to the example’s quotation tick:

Type	Mode	Quantity	Price	Amount excluding fees
Buy	Limit	6	€9.90	€59.40
Sell	Limit	6	€12.11	€72.66

Use these new values with the broker. An order that has only expired, been cancelled or remained unexecuted creates no Jedi operation.

Handle partial execution

Several fragments of one order must be grouped into **one consolidated operation**. Each operation record can advance the mechanical position; treating fragments as successive steps would distort that order’s path.

For our example sale, the broker might have executed:

Fragment	Quantity	Price	Time	Fees
First	2	€11.00	10:14:00	€0.10
Second	4	€11.00	10:15:00	€0.13
To record	6	€11.00	10:15:00	€0.23

The sale command above records this consolidation. If prices differ, use the quantity-weighted average price, total fees and time of the last fragment.

While the remainder can still execute, wait for the situation to stabilise before recording. During that wait, Jedi's state is temporarily behind: do not use it to launch another set of orders.

If you decide to retain and record only the executed part, **immediately cancel the remainder through the broker and confirm cancellation before replacing recalculated orders**. Record only the quantity actually executed. The trade continues with its actual stock.

The average price of several executions may not match the quotation tick. Jedi normally checks that tick. To record a broker-confirmed average without artificial rounding, prefixing the price with + disables this check for that entry: for example, `--price=+11.013333`. It does not increase the price. Use this form to report actual execution, rather than bypassing order-price checks.

Correct fees on a recorded operation

The broker's execution statement may show fees different from the entered or automatically calculated amount. `operation fees` replaces the **total fees** of a recorded operation, including any transaction tax.

First find its identifier by displaying operations with fees, column `16`:

```
jedidemo operation list Exa_01 -c 0,5-9,16,17
```

In our fictional database prepared without earlier operations, the sale of 6 shares has ID `2`. If its fees had incorrectly been recorded as €0.20, correct them to the confirmed €0.23:

```
jedidemo operation fees 2 0.23
jedidemo operation list Exa_01 -c 0,5-9,16,17
jedidemo trade results Exa_01
```

Use **the operation identifier actually displayed**, which may differ in your database. The amount uses the trade's currency and a decimal point, then is rounded to cents. `0` cancels the fees. The amount replaces the previous value; it is not a difference to add.

Any operation can be corrected, including an earlier operation or one in a closed trade. Quantities, prices, holdings, position, investment excluding fees and orders remain identical. Lists and exports show corrected fees; `trade results` recalculates total fees and net results. A new `tools play` starting from this operation or a later operation in the same trade copies the corrected historical fees. `backtest` and `score` calculate their own simulated operations using configuration.

If needed, retain the reason and previous amount in a note. For the remainder of our example, the sale's fees remain **€0.23**.

Retain context with note

Operations record stock and cash movements. `note` retains the context of exceptional events: partial execution, cancellation, interrupted monitoring, an incident or a particular decision.

To write several lines, open the content's quotation marks and close them only after the last line:

```
$ jedidemo note create 2026-09-22 "Exa_01 - partial sale" "Sale of 6 shares:
> 2 then 4 shares at EUR 11. Execution completed at 10:15.
> Total fees: EUR 0.23. One consolidated operation recorded."
```

$ and > represent terminal prompts: do not type them. While quotation marks remain open, the terminal continues the same command; line breaks are preserved in the note. This syntax also suits long text that would be cut off in print on a single line.

```
jedidemo note list Exa_01
```

In a database without previous notes, this note has ID 1. Use the ID actually shown to read or complete it:

```
jedidemo note show 1
jedidemo note update 1 --edit
```

`--contents` remains available and replaces the whole content, which can also span several lines. Retain existing text when adding information.

Write with an editor

Option **`--edit`** makes note creation and modification easier. Jedi uses the program in `VISUAL`, or `EDITOR` if `VISUAL` is absent. These variables must contain the executable's name or path, for example `nano`.

In the Ubuntu or Linux terminal, if chapter 02's configuration has not yet been made:

```
export EDITOR=nano
export VISUAL=nano
```

Then create a note:

```
jedidemo note create --edit
```

The editor opens a template with three markers, to keep in this order. Here is a completed example; marker names remain in English in every language:

```
# Date
2026-09-22

# Summary
Exa_01 - partial sale

# Contents
Sale of 6 shares: 2 then 4 shares at EUR 11.
Execution completed at 10:15; total fees EUR 0.23.
One consolidated operation recorded.
```

In nano, save with **Ctrl+O**, confirm the filename with **Enter**, then exit with **Ctrl+X**. Leaving the template unchanged cancels creation. To resume an existing note, `jedidemo note update 1 --edit` opens its current content: replace 1 with its identifier. Do not combine `--edit` with field options `--date`, `--summary` or `--contents`.

Notes form a general journal; including the trade name in the summary or text makes them searchable. There is no automatic trade link. A note replaces neither execution nor its operation record.

After this monitoring example, Exa_01 holds 94 shares. Chapter 09 starts from this state to present possible adaptations and closure.

9. Adapting or ending a trade

Start from a checked state

Adaptation changes rules or references used for subsequent operations. It must not replace actual past operations.

Before intervening, check stock, recent operations and active broker orders. Cancel orders made obsolete by the change and confirm cancellation. Back up the database as described in chapter 02.

Apart from the separate 1,790-share consolidation example below, the examples are **separate choices**, each starting from fictional trade Exa_01 at the end of chapter 08: 94 shares, €934 net investment, position `0:0:+1` and scale `B_vr_01`. Do not execute all variants successively on that same state. To compare them, retain a backup of this state and restore it into an empty experiment database before each variant.

Choose the required change

Need	Command or option
Continue with another scale	`settings create` with that scale, without a special option.
Give retained stock a new mechanical origin	`--reinit=reset,c0`.
Link a new origin to an actual transaction	`--reinit=merge,c0`, then the operation.
Account for a split or consolidation	`--split=NEW:OLD`.
Exceptionally resume a type I position	`--sync=x:y:e`.
Change the name or capital management	`trade update`.
Correct fees on any operation, including an earlier one	`operation fees ID AMOUNT`, as described in chapter 08.
End the trade	Record the actual sale of all remaining stock.

From this Jedi 1.1 change onwards, **`--split` may accompany `--reinit=reset,price` or `--reinit=merge,price` in one settings entry**. The split first adjusts the holding; the price supplied to `reinit` uses the security's new unit. `--sync` remains incompatible with `--split` and with `--reinit`. New settings must respect global chronology as explained in chapter 07.

The trade's latest settings must have been used by an operation before creating another set. This also applies to a simple change, split or reset. If unused settings resulted from an entry mistake, the end-of-history correction is explained below; do not invent an operation to satisfy the check.

Change scales while preserving continuity

In this variant, record a scale with 12% movements, then create the new settings:

```
jedidemo scale create --name=B_vr_02 \
  --comment="Exemple avec mouvements de 12 %" \
  --builder="vr:12%,np:3,nn:3,ki:10%,kb:1,kd:0"
jedidemo settings create Exa_01 --scale=B_vr_02 \
  --date=2026-09-23 --time=09:00:00
jedidemo settings list Exa_01
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Stock remains 94. Here the new settings retain `q0=100` and `c0=10`; those references are not current stock and price. They resume step `+1` with position `0:0:+1`.

During a simple change, Jedi resets jump counters `x` and `y` to zero and incorporates their effects into cumulative coefficients. Calculations therefore remain continuous without copying all counters. Chapters 12 and 16 explain the mechanics.

The new scale must accommodate the retained step; otherwise Jedi refuses the change. Read new orders before placing them.

Restart from current stock with reset

This variant retains 94 shares and chooses a fictional €11 reference price:

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-23 --time=09:00:00 --reinit=reset,11
jedidemo settings list Exa_01
jedidemo trade orders Exa_01
```

The new settings have `q0=94`, `c0=11` and position `0:0:0`. No purchase or sale is created. Earlier operations and existing investment remain in the history.

Choose the new price under an explicit rule, such as the observed price at intervention. A reset can change subsequent orders; it neither corrects an erroneous stock count nor erases previous losses. Retain the reason, selected price and cancelled orders in a note.

Link reinitialisation to an operation with merge

This other variant assumes an **actual** manual purchase of 6 shares at €11. Create settings, then record execution:

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-23 --time=09:00:00 --reinit=merge,11
jedidemo operation create Exa_01 \
  --date=2026-09-23 --time=09:01:00 \
  --type=Buy --mode=Market --quantity=6 --price=11 --fees=0.23
jedidemo settings list Exa_01
jedidemo trade status Exa_01
```

After settings creation, new references remain pending. The next operation establishes them from retained stock, the supplied price and the transaction. In this example, stock and `q0` become 100, `c0` is €11 and position is `0:0:0`.

Use `merge` when the transaction is part of the reinitialisation decision. Until it is recorded, `q0` and `c0` remain zero and **Jedi proposes no automatic order**. Record the actual purchase or sale to establish the new reference. The note should describe the reason and expected transaction.

Account for a split or consolidation

A split changes share count and price units without a purchase or sale. For two new shares per old share:

```
jedidemo settings create Exa_01 --scale=B_vr_01 \
  --date=2026-09-23 --time=09:00:00 --split=2:1
jedidemo quote set FR4487601001 --price=5.50
jedidemo trade status Exa_01
jedidemo trade orders Exa_01
```

Monitoring stock changes from 94 to **188 shares**. `q0` changes from 100 to 200 and `c0` from €10 to €5. Position retains step `+1`; net investment remains €934. The fictional €5.50 price is entered in the new price unit.

The Jedi ratio is always **new : old**:

Event	Jedi option
10 new shares per old share	`--split=10:1`
1 new share per 10 old shares	`--split=1:10`

Consolidating 101 old shares at ten for one leaves 10 whole new shares and a fractional entitlement of 0.1 share. Consult the event's terms for its treatment. Jedi handles neither this fraction nor any cash settlement. Compare whole stock with the broker's statement and note any amount received; this payment remains outside the trade results calculated by Jedi. Do not turn it into a fictional purchase or sale.

After the event, also check quotation ticks and every order. The ratio and effective date must match the actual event.

Combine a consolidation and reinitialisation

This case is **separate from the variants involving Exa_01's 94 shares**. Assume an already recorded fictional trade named `Exr_01`, retaining **1,790 shares** after its latest transaction. The security is consolidated at one new share for ten old shares, then resumed monitoring calls for a new reference. The chosen post-consolidation price is **€12**; do not divide it by ten in the command.

For illustration, the stored scale is `B_vr_03`, defined by `vr:10%,ks:0.8,1.2,ki:0%,kb:0`. Use the scale actually selected for your trade and dates respecting the whole database. Chapter 19 reproduces this case in memory without creating `Exr_01`.

With **reset**, create one settings entry:

```
jedidemo settings create Exr_01 --scale=B_vr_03 \
  --date=2026-10-12 --time=09:00:00 \
  --split=1:10 --reinit=reset,12
jedidemo settings list Exr_01
jedidemo trade status Exr_01
jedidemo trade orders Exr_01
```

The effective holding and `q0` become **179 shares**, `c0` is **€12**, position is `0:0:0`, and cumulative coefficients `kspc` and `kcpc` equal 1. Consolidation creates no transaction and does not change past financial results. Orders and stops use the 179 shares from these settings onwards; the reset price immediately supplies the first type II calculations.

With **merge**, choose this alternative from the same starting state:

```
jedidemo settings create Exr_01 --scale=B_vr_03 \
  --date=2026-10-12 --time=09:00:00 \
  --split=1:10 --reinit=merge,12
```

The effective holding is already 179 shares, but the reference is pending: `q0=0`, `c0=0`, no automatic order. If the investor actually purchases **21 new shares at €10**, record execution in the new units:

```
jedidemo operation create Exr_01 \
  --date=2026-10-12 --time=09:01:00 \
  --type=Buy --mode=Market --quantity=21 --price=10 --fees=0.74
jedidemo settings list Exr_01
jedidemo trade orders Exr_01
```

The holding and `q0` become **200**, and `c0=(179×12+21×10)/200`, or **€11.79**. An actual sale also works: selling 20 new shares at €14 would leave 159 shares and `c0=(179×12−20×14)/159`, or **€11.7484**. These references exclude fees; actual fees are recorded with the operation.

The reference price, execution price and traded quantity all use **post-split units**. Jedi does not convert these supplied prices again. Once a transaction belongs to the new settings, the split is not applied to its stock a second time.

The 1,790 shares create no fractional entitlement. With 1,793 shares, reset or merge would use the **179 whole shares**; fractional rights and any settlement remain outside Jedi's results, as explained above.

This combination does not allow consecutive unused settings. It brings two compatible changes into one settings entry, without a zero-quantity operation or manual conversion of orders.

Exceptionally resynchronise a position

Interrupted monitoring or a large gap can lead to resuming management at another theoretical position. Operations that did not occur must not be recorded.

`--sync` is restricted to **type I scales**. This fictional variant explicitly uses such a scale:

```
jedidemo scale create --name=B_kc_02 \
  --comment="Exemple de type I pour sync" \
  --builder="kc:1.4,1.2,0.8,0.6,ki:10%,kb:1,kd:0"
jedidemo settings create Exa_01 --scale=B_kc_02 \
  --date=2026-09-23 --time=09:00:00 --sync=1:0:+1
jedidemo settings list Exa_01
jedidemo trade orders Exa_01
```

Supplied position `1:0:+1` describes the requested connection. Jedi incorporates its effects and opens the active settings at `0:0:+1`. It does not correct stock or add missed operations.

Choosing a position requires understanding the scale and resumption price; part II covers this further. Before intervention, resolve stock discrepancies from actual executions. Note the interruption period or gap, previous position, price and selected position.

Change the name or capital management

Normally retain the name chosen in chapter 07 throughout the trade's life. To correct an unsuitable label, for example replace the short form `Exa_01` with `Exemplea_01`:

```
jedidemo trade update Exa_01 --name=Exemplea_01
```

To enable or change `money`:

```
jedidemo trade update Exemplea_01 --money="r:1%,o:1,n:2"
jedidemo trade orders Exemplea_01
```

These values use configured capital, here €5,000. They are illustrative choices; they change protective rules without altering past operations.

If a stored stop exists and `money` changes, Jedi requires explicit resetting:

```
jedidemo trade update Exemplea_01 --money="r:1.5%,o:1,n:2" --reset-stop
jedidemo trade orders Exemplea_01
```

Read the confirmation prompt. This option clears the stop stored in Jedi; it cancels no broker order. Check protective orders before and after the change. Increasing risk to make a loss acceptable on screen does not repair the trade.

Correct the end of the history

For an error affecting fees alone, use `operation fees ID AMOUNT`: any operation can be corrected, even in a closed trade, without stepping back through history. Chapter 08 describes the procedure.

For other operation data, there is no arbitrary editing of an earlier record. Correction steps backwards from the end of global history, then recreates correct records.

```
jedidemo operation delete
```

This command proposes deleting **the latest operation in the whole database**, rather than an operation chosen within Exa_01. Read its confirmation. If more recent settings exist, the operation cannot be deleted first.

If those latest settings have not been used by any operation, the sequence is:

```
jedidemo settings delete
jedidemo operation delete
```

After checking confirmations, recreate corrected records in their actual chronology and check orders. Deleting an operation can undo closure; use it to correct an entry rather than rewrite execution. Record the reason and corrected values in a note.

End the trade through its actual sale

This final variant starts from the 94 shares at the end of chapter 08, without the previous adaptations. Cancel orders incompatible with the exit, sell the stock through the broker and record confirmed execution:

```
jedidemo operation create Exa_01 \
  --date=2026-09-23 --time=15:00:00 \
  --type=Sell --mode=Market --quantity=94 --price=11.20 --fees=3.68
jedidemo operation list Exa_01 -c 0,6-14,17
jedidemo trade list
jedidemo trade results Exa_01
```

Stock becomes **zero**: the trade is closed. There is no administrative `trade close` command. Record the appropriate `Market`, `Limit`, `Stop` or `StopLoss` mode according to the order actually executed.

Useful values for our example are:

Information	Value
Recorded purchases excluding fees	€1,000.00
Recorded sales excluding fees	€1,118.80

Information	Value
Gains excluding fees	€118.80
Total fees	€7.41
Final stock	0

After fees, this example's gain is €111.39. The annualised rate displayed for such a short path is an extrapolation; it does not replace this monetary balance.

If the final sale remains partial, apply consolidation and remainder cancellation as explained in chapter 08. The trade remains open while holding shares.

Once stock is zero, Jedi refuses another operation on this trade. Create another trade for new management of the same security. Retain the completed trade: operations, settings, contextual notes and results support later checks. `trade delete` refuses to delete a trade with a history.

10. Managing the market database

The Demo database for the examples

This procedure prepares the data used from chapters 03 and 04 onwards. Files are supplied in examples/ at the root of the documentation archive. First follow chapter 02 to install `jedidemo` and copy the files to `~/jedi-examples/`. The launcher opens that directory for commands regardless of the terminal directory. The commands below assume the default universe `france` and a `demo_france.db` database that does not yet exist.

Run the steps in order, through the update of all three securities to 2 October. Chapter 04's `profile` examples will then use this `Demo` dataset. Individual files `cotations_EXA.txt`, `cotations_EXB.txt` and `cotations_EXC.txt` can also be studied directly without creating this database.

Import the dataset

This batch's `examples/` directory contains **entirely fictional** data with a simplified weekday calendar. With files copied to the demonstration directory, the following commands create a `demo_france` database that must not exist initially:

```
jedidemo market init demo
jedidemo market import demo Demo Demo_initial.txt labels_demo.txt
jedidemo market info demo --dataset=Demo
jedidemo market list demo --dataset=Demo --last-quote
```

`init` creates the database. `import` creates the dataset; an existing dataset must be updated or replaced with the appropriate command.

The initial demonstration import contains three securities and 2,550 quotations through 1 October 2026. The report states counts and anomalies. Some invalid quotations or quotations without labels can be reported and skipped: read the report and log.

To consult two sessions for one security:

```
jedidemo market quote demo FR4487601001 --select=l2
```

`market quote` displays this ISIN's history. It differs from `quote get`, which queries current-price providers.

Update the quotations

An update supplies a file containing new sessions with the required labels. It merges data into an existing dataset.

In the demonstration, first add only EXA's quotation for 2 October:

```
jedidemo market update demo Demo Demo_partiel.txt labels_demo.txt
jedidemo market info demo --dataset=Demo
```

EXA is recent; the other two securities remain at 1 October. The observed values distinguish:

Information	Value after the partial update
Last full import period	2023-06-30 → 2026-10-01
Currently available period	2023-06-30 → 2026-10-02
Global freshness date, 95%	2026-10-01

Then update all three securities:

```
jedidemo market update demo Demo Demo_update.txt labels_demo.txt
jedidemo market info demo --dataset=Demo
```

EXA's equivalent duplicate is skipped; the other two sessions are added. The global reference advances to **2 October 2026**, while the last full import period stays unchanged.

This illustrates normal dataset maintenance. Reimporting the whole history is unnecessary for advancing freshness.

Two databases with separate uses

Database	Contents	Use
Main Jedi database	Securities, trades, settings, operations, notes and recorded prices	Trade management
Market database	OHLCV histories, datasets and groups	Comparison, scale search and simulations

OHLCV means open, high, low, close and volume per session. These data describe a history; a current price describes one point in time.

The market database is optional. An individual file can supply `profile`, `score` or `backtest`. A database simplifies work on many securities and their updates.

`market` commands import files supplied by the user. They do not download them. Importing these histories updates neither the main database price nor trade operations.

Source, universe, dataset and group

Concept	Example	Purpose
Source	`abcbourse`, or `demo` in our examples	Logical name for the files' origin
Universe	`france`	Scope completing the database name
Dataset	`Euronext_A`, or `Demo`	Imported collection of securities and quotations
Group	`PEA`, or `EXEMPLES`	Logical list of securities

A source and universe identify a SQLite file, such as `abcbourse_france.db` or `demo_france.db`. The source name does not trigger data collection.

One security can belong to several datasets and groups. Quotations can be shared between datasets: per-dataset counts do not necessarily add up.

`Euronext_A`, `Euronext_B`, `Euronext_C`, `Euronext_Access` and `Euronext_Growth` belong to the French context used to prepare Jedi. Commands remain usable with other scopes, taking account of their data and monetary unit.

Check locations

```
jedidemo config --paths
```

With `jedidemo`, market databases normally live in `~/jedi-examples/jedi/env_test/market/`. `JEDI_DEMO_MARKET_DIR` selects another directory. With `jedi`, the production market normally lives in `~/.local/share/jedi/market/`, taking `XDG_DATA_HOME` into account. Chapter 02 explains these choices.

The default universe comes from `market_data.default_universe`, normally `france`. An explicit option overrides it:

```
jedidemo market info demo --universe=france
```

Use the same universe throughout a command sequence. Changing it identifies another physical database.

Prepare the files

"Prepare" means identify provenance, put data into the expected format and check consistency. An export obtained outside Jedi can need conversion. The software licence does not define usage rights for third-party quotations.

For a dataset import, prepare two files.

The quotation file contains **ISIN; date; open; high; low; close; volume**, without a three-field identity row:

```
FR4487601001;20230630;10.0000;10.1800;9.8200;10.0000;60000
FR4487601019;20230630;30.0000;30.4800;29.5200;30.0000;30000
```

The labels file contains **ISIN; name; ticker**:

```
FR4487601001;Exemple A;EXA
FR4487601019;Exemple B;EXB
FR4487601027;Exemple C;EXC
```

Watch the order: an individual file for the tools starts with **ISIN; ticker; name**, followed by its quotations:

```
FR4487601001;EXA;Exemple A
FR4487601001;20230630;10.0000;10.1800;9.8200;10.0000;60000
```

Do not confuse these two first rows. An individual file concerns one ISIN; a dataset file can contain several. Avoid a column-header row in the quotation file. The labels file accepts an `ISIN;Name;Ticker` header.

The quotation reader accepts semicolon or comma separators and several date formats. When preparing a new file, `YYYYMMDD` dates and decimal points avoid ambiguity. For ambiguous dates, `--date-order=dmy` or `ymd` specifies order; consult command help.

Before import, check ISINs, dates, columns, positive prices, highs and lows, non-negative volumes and duplicates. Identify raw or adjusted prices and use them consistently. A successful import does not prove that missing sessions, suspensions or provider corrections have been handled correctly.

Keep original files, period coverage, transformations, the security list and acquisition date. Provide enough history for the tools: `profile` examines 252 recent sessions but requires longer history for checks; `score` uses 750 by default.

Understand the three dates

Information	What it describes
Last full import and its period	The import operation and the quotation period supplied by that import
Currently available period	Oldest and newest quotations stored for the dataset
Global freshness date	Date reached by at least 95% of securities

The global reference is calculated from each dataset security's latest quotation. It can advance with some lagging securities when at least 95% have reached the new date. For 600 securities, at least 570 are required; 595 recent securities and five laggards satisfy that condition.

It does not depend simply on the time of the last `market update` call. It is also a relative reference: if all securities are old, check the date against the session you intend to examine.

`profile` uses this reference for individual lag: zero sessions, caution for one or two, rejection beyond two. Historical selection or an ambiguous dataset can make this check unavailable; chapter 04 explains these cases.

Read anomalies and gaps

An existing quotation with identical values is an equivalent duplicate. A conflicting value during `update` is reported and skipped; the stored value remains. Updating is therefore not a correction procedure for conflicting older quotations.

If the first new session is more than seven calendar days after the dataset's current end, the extension is refused unless `--allow-gap` is used:

```
jedidemo market update demo Demo Demo_update.txt labels_demo.txt --allow-gap
```

This option accepts the gap; it does not fill it. The check concerns the dataset's overall extension and does not detect every missing session in each security. Consult individual histories when a report or provider indicates a problem.

Replace a dataset and manage groups

`market replace` rebuilds a dataset from a new complete corpus, for example after data corrections. Check files and keep a backup before using it; data shared with other datasets are preserved.

A group is created or synchronised from a list of ISINs or tickers, one per line:

```
jedidemo market sync-group demo EXEMPLES groupe_exemples.txt \
  --as-of=2026-10-02 --reference-source=demonstration
jedidemo market list demo 'group=EXEMPLES' --last-quote
jedidemo tools profile --database=demo --dataset=Demo --group=EXEMPLES --rank-criteria=4
```

Synchronisation replaces current membership: a member missing from the file is removed. `--as-of` describes the list's reference date, not automatic reconstruction of past membership.

Maintain files, read `market info`, check latest quotations with `market list`, then run tools on the intended population. The main database and prices used for real monitoring continue to be managed separately.

Understand Jedi's mechanics

Part I showed how to prepare and monitor a trade. This part explains how a scale connects price movements with share quantities, orders and committed capital.

The prerequisite is familiarity with the practical path: chosen security, scale, settings, order and executed operation. More precise concepts are defined when first used. Formulas follow explanations in ordinary language.

Chapters 11 and 12 explain flows, positions, steps and levels. Chapters 13–15 present scale structure and its two types. Chapter 16 covers continuity across settings changes. Chapters 17 and 18 connect this state with orders, risk and sizing. Chapters 19 and 20 check calculations and follow a complete example.

Prepare the mechanics examples

Most examples start with **100 fictional shares at €10**, a type I scale and a €0.01 tick. This scale differs from Exa_01's type II in part I. Chapter 15 compares a type II at 20%. Chapters 18 and 20 also use 125 shares to illustrate sizing.

Paths are calculated with `tools play` and, for type I, `tools calc`. They require neither downloads nor quotation file imports. Movements are chosen to show the calculation, without claiming to represent a real market.

Use chapter 02's experiment environment. Fees are **0.35%, without a minimum**. For chapter 18's protections, add this to that environment's configuration:

```
simulation:
  capital: 20000
```

This capital applies to `play`. `candidate` capital is explicitly supplied by `--capital=20000`. Its evaluations assume a database with no open trades. Teaching scale creations and notes write to the experiment database; paths defined directly in `play` remain in memory.

The example guide recalls these conventions. Start with chapter 11's flows, then follow chapters in order.

By the end of this part, readers can explain a holding change, understand a Limit or Stop order, distinguish protective StopLoss and read the effects of `r`, `o` and `n`. Understanding these calculations helps check the process; its historical results will be studied in part III.

11. Round trips and trade flows

A rising price brings money into a trade only when a sale takes place. A falling price takes money out only when a purchase takes place. To read a trade, start with the unit price, the number of shares held and the purchases or sales actually executed.

This chapter uses **Example A, a fictional security**, with 100 shares bought at €10. It is a type I example, separate from Exa_01's scale in part I. The chosen prices explain the calculations; they describe no real market.

Sell and buy back, or buy and sell back

A round trip consists of two opposite transactions for the same quantity. After both transactions, the holding returns to its previous size.

Completed round trip	Money received	Money spent	Difference before fees
Sell 20 at €12, then buy back 20 at €10	€240	€200	+€40
Buy 20 at €8, then sell back 20 at €10	€200	€160	+€40

These differences become known when **both transactions have taken place**. After the sale alone, the repurchase is still pending; after the purchase alone, the resale is still pending. A proposed order is not a completed round trip.

See the path in Jedi

`tools play` simulates chosen movements in memory. Chapters 13 and 14 will explain the scale. First read the resulting operations:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 -1 1
```

The warning about a future sale of a very small quantity concerns a continued rise beyond this round trip. It is explained in chapter 12.

`1` requests an upward movement; `-1` a downward movement. `--` allows negative numbers to be passed as arguments. The backslashes continue the command, including inside the quotation marks.

Operation	Flow before fees	Shares held	Current investment
Opening purchase: 100 at €10	−€1,000	100	€1,000
Sale: 20 at €12	+€240	80	€760
Repurchase: 20 at €10	−€200	100	€960
Purchase: 20 at €8	−€160	120	€1,120
Resale: 20 at €10	+€200	100	€920

A sale reduces the holding and releases money; a purchase increases it and uses money. `play` executes the calculated orders at its scenario prices and records no operation in your portfolio.

Understand investment and its maximum

Current investment is total purchases minus total sales, before fees. It ends at €920: €1,360 of purchases less €440 of sales.

Historical maximum investment is the highest current investment already reached. It is €1,120, after the purchase at €8. The following resale does not erase that maximum.

These figures are not the broker account balance: they track this trade's flows only. Future purchases may require an additional reserve, discussed in chapter 18.

Add the value of the remaining shares

The two round trips earned €80 before fees and 100 shares remain. The result also depends on their value at the chosen price:

```
holding value = shares held × chosen price
gross result = holding value − current investment
```

Valuation of the same final holding	Value	Investment	Gross result
100 shares at €10	€1,000	€920	+€80
100 shares at €9	€900	€920	−€20

Profitable transactions therefore do not protect the remaining shares against a fall. The chosen price values the holding; it does not guarantee the price at which all shares could be sold.

Keep fees separate

Jedi tracks fees separately. At 0.35%, with no minimum fee or transaction tax on this fictional security:

Operation	Fees
Opening purchase	€3.50
Sale at €12	€0.84
Repurchase at €10	€0.70
Purchase at €8	€0.56
Resale at €10	€0.70
Total	**€6.30**

At the final price of €10, the result after fees is **€73.70**. Current and maximum investment remain before fees. `trade results` distinguishes gross result from fees. In `play`, financial output `f` is gross. To read fees in the operations too, add column `16` to the selector: `-c o0,6-14,16-17`.

When the return does not happen

After selling at €12, the price may keep rising without allowing a repurchase at €10. The trade then holds fewer shares for the next rise. After buying at €8, it may keep falling without allowing a resale: more shares remain exposed.

Immediately after that purchase, the holding is 120 and investment €1,120. At a price of €6, its value would be €720, a gross result of −€400. The first profitable round trip does not offset this fall.

A price change alone alters neither the holding nor recorded flows. Chapter 12 adds the mechanical position used to calculate what comes next.

12. Positions, steps and levels

The holding tells us how many shares are owned. The **mechanical position** tells us where the trade stands within its scale. It is neither a quantity nor a result in euros.

Read the three numbers

A position is written `x:y:e`:

Number	Meaning
x	Number of jumps to a higher level since the active settings began.
y	Number of jumps to a lower level since those settings began.
e	Current step within the level reached.

The counters `x` and `y` are nonnegative. Step `e` can be positive, negative or zero. `1:0:+1` means one upward jump, no downward jump and step +1.

A **step** is a local stage of the scale. A **level** is a reference from which another series of steps unfolds. The first purchase places the trade at `0:0:0`.

Some outputs put a number before the position: `1 0:0:+1`. That first `1` is the settings rank. The position still contains three numbers.

Reach a boundary and change level

Chapter 11's type I scale has boundaries `emax=+2` and `emin=-2`. Its interior steps are −1, `0` and +1.

Current position	Movement	Next position
`0:0:0`	Up	`0:0:+1`
`0:0:0`	Down	`0:0:-1`
`0:0:+1`	Up to the upper boundary	`1:0:0`
`0:0:-1`	Down to the lower boundary	`0:1:0`

On reaching a boundary, Jedi increments the corresponding counter and resets `e` to zero. The position does not stay at +2 or −2. Step zero after a jump does not automatically return to the first price or holding.

Observe both jumps with play

At the upward jump, this teaching scale generates a sale of only one share at €14. Jedi displays a warning after the opening purchase, then reports a degraded operating mode after the first sale: the next upward order is for just one share. These messages flag a quantity too small for satisfactory operation; the simulation continues so the positions below can be observed. Chapter 06 explains the check.

Two upward steps, then two downward steps:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2 -2
```

Moment	Position	Simulated price	Holding
Opening purchase	`0:0:0`	€10	100
First rise	`0:0:+1`	€12	80
Second rise, upward jump	`1:0:0`	€14	79
First fall from this level	`1:0:-1`	€11.20	94
Second fall, downward jump	`1:1:0`	€8.40	118

The final counters are `x=1` and `y=1`: they do not cancel each other. Prices and holdings come from the scale; the coordinates alone would not calculate them.

Current and target positions

A **target position** is the position a calculated operation would reach. From `0:0:+1`, the next rise would target `1:0:0`, and the next fall `0:0:0`. These are alternative possibilities.

Placing, cancelling or letting a broker order expire does not change the recorded position. Jedi updates the path when you record the actual execution. The mechanical direction depends on operation type and mode, as chapter 17 explains.

Once the active settings have been used, orders start from their last operation. Before their first operation, orders start from the position recorded in the new settings.

Carry the step into new settings

An ordinary settings change retains `e` and resets `x` and `y` to zero. A last operation at `1:1:-1` is thus continued by settings starting at `0:0:-1`.

Holdings, flows and historical positions are preserved. The effects of jumps carry into cumulative coefficients, discussed in chapter 16. Resetting the counters does not remove their effect on calculations.

The `reset` and `merge` reinitialisations establish a new origin `0:0:0`. Type I synchronisation resumes from a chosen position. Chapter 09 gives the practical commands.

Chapter 13 presents the scale associating this position with a target price and holding.

13. Read the structure of a scale

A scale associates price movements with changes in the target holding. It does not describe transactions already performed. Jedi calculates a target, then compares it with the actual holding to determine the next order.

Two references: c0 and q0

At the first purchase, `c0` is the unit price and `q0` the quantity. For 100 shares bought at €10, the references are `c0=10` and `q0=100`.

Later settings may retain them, adjust them for a split or replace them during reinitialisation. **`c0` is therefore not always the first operation's price, and `q0` is not always the initial holding.** Nor are they automatically the latest price and current holding.

The **theoretical price** is the price associated with a mechanical target. The **theoretical holding** is the number of shares the scale would like to hold at that target. The order then uses a tick-compliant price and an integer quantity. Actual executions can differ.

Read the scale elements

A coefficient is a multiplier: a €10 reference with a price coefficient of 1.20 gives €12. A 100-share reference with a holding coefficient of 0.80 gives 80 shares, before level effects.

Element	What it controls
`kc`	List of price coefficients for type I.
`vr`	Variation rate for the next price in type II.
`ks`	List of holding coefficients for either type.
`ki`	Adjustment to the theoretical capital reference on a jump.
`kb`	Convention for that adjustment; our usual examples use `kb:1`.
`kd`	Distribution of holding changes when Jedi builds the `ks` list.
`np`, `nn`	Upper and lower boundaries of a compact type II builder.

`ki` differs from capital management risk `r`. A 10% level adjustment is neither 10% portfolio risk nor a promised gain.

Name and number a scale

At creation, **Jedi derives the name from the operational type produced by the builder**. Omit `--name` in `scale create` to use automatic naming.

Name format	Operational type
`B_kc_NN`	Type I: prices determined by `kc`.
`B_vr_NN`	Type II: prices determined by `vr`.

All type I builders share `B_kc_NN`, whether they receive `ks` or construct holdings using `kd`. The three type II builders share `B_vr_NN`: an explicit `ks` list, construction using `kd`, or the Legacy form using `sr` and `mi`. In the current code, Legacy is **type II**.

`NN` is the highest number currently present in that family plus 1. Numbering is independent for the two types: `B_kc_01` and `B_vr_01` can coexist. At least two digits are displayed, followed by `100`, `101`… Explicitly assigned numbers are included; gaps are not filled while a higher number remains present.

An equivalent operational definition is refused, even with a different builder. Jedi compares type, price coefficients or `vr`, holding coefficients, `ki`, `kb` and boundaries. For example, `kc/kd` and `kc/ks` can produce identical rules: they then represent one scale to reuse. The refusal identifies the existing scale by name and ID. It precedes automatic name allocation and consumes no number.

The builder and comment remain available through `scale show`. They retain the construction method and context; the name supplies type and number. Scales sharing a few values such as `vr` and `ki` can still have different definitions.

`--name` remains available to fix a reference, but must match the builder type: `B_kc_NN` or `B_vr_NN`, with a positive number of at least two digits. Case and extra leading zeros are normalised. An assigned number remains reserved even when a historical name has more leading zeros. Examples sometimes fix a number to preserve references between chapters and independent variants; normal use lets Jedi choose it.

Existing stored names are retained. Reuse an older scale by its actual name, or explicitly rename it with `scale update --name` using the corresponding format. Pre-existing duplicate scales are retained too, including during backup restoration. New creations and definition changes cannot introduce further duplicates; `--force` does not bypass this check.

To change an already used rule, creating a new scale and new settings preserves earlier operations' reference definitions. If a numbered scale is updated to the other type under `scale update`'s protections, Jedi computes its new name in that family unless a conforming explicit name is supplied.

Expand a simple type I scale

Save this teaching scale in an experiment database:

```
jedidemo scale create --name=B_kc_01 \
  --comment="Teaching scale of type I" \
  --builder="kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1"
jedidemo scale show B_kc_01
```

Create this name once only. If it exists, inspect the scale before reusing it. Other examples often pass the definition directly to a tool without creating a permanent name.

Jedi adds coefficient 1 at step zero:

Step	Price coefficient kc	Holding coefficient ks
+2, upper boundary	1.40	Jump to the next level
+1	1.20	0.80
0	1.00	1.00
−1	0.80	1.20
−2, lower boundary	0.60	Jump to the next level

In the first level, +1 targets 80 shares at €12 and −1 targets 120 at €8. At the boundaries, Jedi changes level and calculates new references.

Jedi accepts boundaries from two to six movements in each direction, with one to five interior steps per side. In compact type II, `np:3,nn:3` means three movements to each boundary, with two interior steps per side.

Distinguish the two types

Question	Type I	Type II
Next price	From `kc` and the level references.	Applying `vr` to the relevant last execution price.
Target holding	From `ks` and holding references.	From `ks` and holding references.
Rise then fall	Within one level, coefficients identify the same prices.	Percentages restart from the last price; the return can target another price.

Chapters 14 and 15 calculate these differences. The type does not predict a financial result.

Builder and expanded definition

The **builder** is the text supplied to Jedi. The **canonical definition** is the expanded, normalised rule it produces.

Form	Main fields
Type I, holding supplied directly	`kc`, `ks`, `ki`, `kb`
Type I, holding built by Jedi	`kc`, `ki`, `kb`, `kd`
Type II, holding supplied directly	`vr`, `ks`, `ki`, `kb`
Type II, holding built by Jedi	`vr`, `np`, `nn`, `ki`, `kb`, `kd`
Type II Legacy	`vr`, `np`, `sr`, `nn`, `mi`, `ki`, `kb`

The expanded definition always exposes `ks`, even when the builder uses `kd`. Type I exposes `kc`, type II `vr`. `scale show` lets you read the result before using it.

Understand optimised

Jedi checks whether the structure reduces the holding at the upper boundary and increases it at the lower boundary. It calls this structural check **optimised**.

The label measures neither profitability, fees nor portfolio requirements. Rounding and some type II paths still require inspection of the actual calculated orders. Chapter 17 will show a functioning Stop order.

An optimised builder is a method of constructing `ks`. A directly supplied list can also pass the check, as `B_kc_01` does.

To change a rule already used, creating a new scale preserves the interpretation of past operations. Chapter 16 explains how to associate it with new dated settings.

14. Understand a type I scale

Type I associates each step with a price coefficient `kc` and a holding coefficient `ks`. Within one level, these coefficients give known targets. A jump changes the references to which they apply.

We use 100 shares at €10, a €0.01 tick and this scale:

```
kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1
```

Calculate the first targets

In the first level, multiply references by coefficients:

Position	Theoretical price	Theoretical holding
`0:0:0`	€10 × 1 = €10	100 × 1 = 100
`0:0:+1`	€10 × 1.20 = €12	100 × 0.80 = 80
`0:0:-1`	€10 × 0.80 = €8	100 × 1.20 = 120

From 100 shares, the rise proposes selling 20 and the fall buying 20. Returning to zero before any jump restores the €10 price and theoretical holding of 100.

Move to the next level

At boundary +2, the new price reference is €10 × 1.40 = €14. The theoretical capital reference is multiplied by `1+ki`, or 1.10. It rises from €1,000 to €1,100.

The theoretical holding at the new level's origin is €1,100 ÷ €14, approximately **78.571429 shares**. This is a calculation target, not a fractional holding actually owned.

At boundary −2, reached directly from the first level, the price reference is €6 and the capital reference €900 with `kb:1`. The theoretical holding becomes €900 ÷ €6 = **150 shares**.

These values are neither current investment, account balance nor trade profit.

Check the multipliers with calc

Jedi calls the level's price multiplier `kcp`, its capital reference multiplier `kcb`, and its holding multiplier `ksp`. Their relationship is `ksp = kcb / kcp`.

For the first upward jump:

```
jedidemo tools calc cpe \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 c0=10
jedidemo tools calc spe \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 q0=100
```

We obtain `cpe=14` and `spe≈78.571429`, the theoretical price and holding respectively.

After one jump in each direction, the price reference is multiplied by 1.40 × 0.60 = 0.84. The capital reference is multiplied by 1.10 × 0.90 = 0.99:

```
jedidemo tools calc kcb \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:1:0
```

The result is 0.99. These two jumps do not restore the first level's references.

Follow integer transactions with play

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2 -2
```

Operation	Position	Quantity	Price	Holding afterwards
Buy Market	`0:0:0`	100	€10	100
Sell Limit	`0:0:+1`	20	€12	80
Sell Limit	`1:0:0`	1	€14	79
Buy Limit	`1:0:-1`	15	€11.20	94
Buy Limit	`1:1:0`	24	€8.40	118

The target of 78.571429 rounds to 79, then is compared with the 80 shares held: one share is sold. The last fall starts from the upper level, rather than directly from the opening.

The optimised structural check therefore does not ensure sufficiently large orders after rounding. Operability and fees still need examination.

Let Jedi distribute holding changes

The compact builder replaces the `ks` list with `kd`:

This example reuses `B_kc_02`, also used for synchronisation in chapter 09. If that name already exists in your experiment database, inspect its definition with `scale show` and reuse the corresponding scale. The create command below records it once only.

```
jedidemo scale create --name=B_kc_02 \
  --comment="Teaching builder" \
  --builder="kc:1.4,1.2,0.8,0.6,ki:10%,kb:1,kd:0"
jedidemo scale show B_kc_02
```

Create this name once in the experiment database. With `kd:0`, changes are distributed evenly between intervals. Jedi produces **`ks:0.89,1.25`**: interior targets of 89 and 125 for `q0=100`.

This differs from `ks:0.8,1.2`. Positive `kd` weights later intervals more; negative `kd` weights earlier ones more. Inspect the expanded scale to examine quantities.

Place the historical Qdt scale in context

`Qdt` remains for compatibility. Its canonical definition is:

```
kc:1.61,1.38,1.23,1.14,0.88,0.76,0.64,0.52,0.4,
ks:0.4,0.68,0.87,1.9,2.9,4.1,5.5,ki:14%,kb:0
```

It uses `kb:0`: a downward jump uses the inverse of `1+ki`. Its boundaries are `+4` and `−5`. It fails the optimised check and can request a purchase on a rise or a sale on a fall.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.61,1.38,1.23,1.14,0.88,0.76,0.64,0.52,0.4,\
ks:0.4,0.68,0.87,1.9,2.9,4.1,5.5,ki:14%,kb:0,\
q0=100,c0=10,tick=0.01" 4
```

Immediately before the upward jump, 40 shares remain. The jump targets 71 at €16.10: **a Stop purchase of 31 shares**. On a separate five-step downward path, the holding falls from 550 to 219 at the boundary: a Stop sale of 331 at €4.

Qdt illustrates why the mode and holding change matter, not just the direction of price. It is not the reference scale for the remaining examples. Chapter 15 examines the other price calculation method.

15. Understand a type II scale

In ordinary operation, type II calculates the next price by applying rate `vr` to the **last executed operation's price**, adjusted for a split when necessary. It retains positions and holding coefficients, without a `kc` price coefficient list.

After `reinit=reset`, the price supplied to reset immediately replaces the latest operation's price for the first orders. If reset accompanies a split, this price already uses post-split units. Subsequent operations then resume using their actual execution price as the basis. With `merge`, Jedi first waits for the manual transaction establishing the new reference. Chapters 09 and 16 cover these cases.

Our fictional example uses `vr:20%` to make the differences easy to see, with 100 shares at €10 and a €0.01 tick:

```
vr:20%,ks:0.8,1.2,ki:10%,kb:1
```

Calculate the next price

From an execution at €10, the next theoretical price is €12 upwards and €8 downwards. After an execution at €12, a fall targets €12 × 0.80 = **€9.60**.

A 20% rise followed by a 20% fall does not return to €10. The percentages apply to different prices. A 20% fall followed by a 20% rise also gives €9.60 from €10.

The current market price locates the orders; it does not replace the last execution price in this formula.

Replay both round trips

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=vr:20%,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 -1 1
```

Operation	Position	Tick-compliant price	Holding
Opening purchase	`0:0:0`	€10	100
Sale of 20	`0:0:+1`	€12	80
Repurchase of 20	`0:0:0`	€9.60	100
Purchase of 20	`0:0:-1`	€7.68	120
Sale of 20	`0:0:0`	€9.22	100

The last theoretical price is €7.68 × 1.20 = €9.216. For this Limit sale, it rounds up to €9.22. The next calculation will start from the executed €9.22 price.

Purchases total €1,345.60, sales €424.40 and current investment €921.20. The 100 shares at €9.22 are worth €922: a gross result of **€0.80**. The two round trips nevertheless produced €78.80 in cash; the fall in holding value also matters.

Comparison with type I illustrates the mechanics. The same directions produce other prices; this is not a historical profitability ranking.

Understand adjustment on a jump

Boundaries remain +2 and −2. On a jump, Jedi adjusts the new level's price reference to the path's price. For an order, it uses the theoretical price; for a recorded operation, the actual execution price.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=vr:20%,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2 -2
```

Operation	Position	Quantity	Price	Holding
Buy Market	`0:0:0`	100	€10	100
Sell Limit	`0:0:+1`	20	€12	80

Operation	Position	Quantity	Price	Holding
Sell Limit	`1:0:0`	4	€14.40	76
Buy Limit	`1:0:-1`	16	€11.52	92
Buy Limit	`1:1:0`	15	€9.21	107

At the upward jump, the theoretical capital reference is €1,100. At €14.40, the theoretical holding is approximately 76.388889, rounded to 76 for the order. At the last jump, €9.216 rounds down to €9.21 for the purchase.

Price rounding affects subsequent calculations. An unrounded calculation and a tick-compliant executed path can therefore diverge after several movements.

Build the holding with np, nn and kd

```
jedidemo scale create --name=B_vr_04 \
  --comment="Teaching builder" \
  --builder="vr:20%,np:2,nn:2,ki:10%,kb:1,kd:0"
jedidemo scale show B_vr_04
```

Create this name once. `np:2,nn:2` sets two movements to each boundary, with one interior step per side. `kd:0` distributes changes evenly. The expanded definition contains **`ks:0.88,1.2`**.

It therefore differs from our first list, `ks:0.8,1.2`. Read `scale show` before comparing orders. Other `kd` values change the distribution as in type I.

The historical form using `sr` and `mi` remains available for compatibility, with its own rules. Appendix 01 describes that syntax; the usual reading path uses the forms above.

Read orders even when the scale is optimised

The type II optimised check uses a sequence of movements in the same direction. After oscillations, the jump price can differ from that reference sequence.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=vr:20%,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 1 -1 1 1
```

Immediately before the last jump, 80 shares remain after a sale at €11.06. The jump targets a theoretical price of €13.272 and an integer holding of 83. Jedi simulates **a Stop purchase of 3 shares at €13.28**, even though the scale passed the optimised check.

That label does not replace inspection of modes and quantities. Chapter 16 examines how a trade continues under new settings.

16. Maintain continuity between settings

A trade can change scale without restarting its history. Purchases, sales, fees and shares held remain those of the recorded operations. New settings must establish references consistent with the position reached.

Chapter 09 gives the monitoring commands. Here we explain what Jedi preserves and recalculates.

Change the local frame

An ordinary change retains step `e`. Level counters `x` and `y` restart at zero. Jumps already completed carry into cumulative coefficients:

Coefficient	What it preserves
`kcpc`	Price multiplier from previous settings.
`kspc`	Holding multiplier from previous settings.

Their initial value is 1. An ordinary change retains `c0` and `q0`, which do not automatically become the latest price and holding. It carries the multipliers needed for continuity.

Past positions are not rewritten. New settings use another local frame, identified by their rank in lists.

Check a change without changing the scale

Three rises of our type I scale take 100 shares at €10 to 63 at €16.80, at position `1:0:+1`. Reapplying the same scale shows continuity without changing the rules:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 3 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 1
```

Moment	Position in the frame used	Holding	Last execution price
Before change	Previous settings, `1:0:+1`	63	€16.80
New starting point	New frame, `0:0:+1`	63	€16.80
Next rise	New frame, `1:0:0`	62	€19.60

On changing, `kcpc=1.40` and `kspc=1.10/1.40≈0.785714`. With `c0=10` and `q0=100`, local references at zero correspond to €14 and 78.571429 theoretical shares. The retained step `+1` recovers €16.80 and a target close to the 63 shares held.

The next rise sells **1 share at €19.60**. Compare without a change:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 3 1
```

Price and quantity are identical. The final position is `2:0:0`: the first jump remains in the original counters.

Actually change the rules

Another scale uses the same carry-over. The past path is preserved; new rules determine the next targets and can change order prices, quantities or modes.

For type II, the last operation also affects the adjustment described in chapter 15. Rebuilding orders from the last price alone would lose some context.

`tools calc` does not take a trade's settings history or cumulative coefficients. To check a change, use `play` with the full path, or inspect the real trade's settings and orders.

Distinguish carry-over and reinitialisation

Action	Purpose
Ordinary change	Retain `q0`, `c0` and the step; carry jumps forward.
`reinit=reset,price`	New mechanical start from the holding and chosen price.
`reinit=merge,price`	New start incorporating the following additional operation.
`split=new:old`	Adapt references to a split or consolidation.
`sync=x:y:e`	Impose a justified position for type I.

Reinitialisation resets the starting position and cumulative coefficients. It does not erase recorded losses or gains. `sync` retains the imposed step in the new frame; it is not a transaction.

After selling 20 at €12, resetting at €12 uses the **80 shares held**:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 reinit=reset,12 -1
```

The next fall buys 16 at €9.60, taking the holding to 96. Without reset, the return would buy 20 at €10.

To incorporate an additional purchase of 20 at €12:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 reinit=merge,12 "20@12.00" -1
```

References become 100 shares at €12. The next fall buys 20 at €9.60, leaving 120. The additional purchase is an operation; changing the settings alone does not perform it.

Apply the split before reinitialisation

`split + reinit=reset` and `split + reinit=merge` are accepted in one settings entry from this Jedi 1.1 change onwards. The latest operation's holding is converted first. Prices supplied to `reinit` and to the following transaction already use new units: they are not converted again.

For 1,790 old shares consolidated at `1:10`, the effective holding is 179 shares. With a €12 resumption price:

State	Effective holding	q0	c0	Starting position
After `split=1:10,reinit=reset,12`	179	179	€12	`0:0:0`
After `split=1:10,reinit=merge,12`, before transaction	179	0	0	`0:0:0`
Merge, after buying 21 new shares at €10	200	200	€11.79	`0:0:0`

Cumulative coefficients restart at 1 with both reinitialisations. For merge, the €12 price remains in the `reinit` option while pending; it supplies the manual transaction's weighted calculation. Automatic orders become available only after that transaction.

Earlier operations retain their units and positions. Before the first transaction under new settings, the summary, orders and projections use the effective holding and new position. After that transaction, recorded stock is already converted; Jedi does not apply the split again.

`split + sync`, `sync + reinit` and all three options together remain rejected. Settings must still be used by an operation before another set is created. Practical examples are in chapter 09 and complete simulations in chapter 19.

Check the change

Keep the state before modification and inspect new proposals in a simulation. After recording, reread settings, operations and orders as in chapter 09.

A change needs an explicit reason: a studied new scale, corporate action, correction or monitoring decision. A note preserves the context. Broker orders must then be adapted to the actual executed state; continuity of calculations does not perform that adaptation.

17. From trade state to orders

Settings newer than the latest transaction can already change the effective holding or starting position. After `split + reset`, Jedi immediately uses converted stock, position `0:0:0` and the resumption price. After `split + merge`, it waits for the manual transaction before calculating automatic orders. See chapter 09.

An order compares the actual holding with the integer holding targeted by the scale. A positive difference is a purchase; a negative difference is a sale. Its price follows the mechanical direction and the security's tick size.

Separate direction, type and mode

A price rise does not always require a sale, nor a fall a purchase. The scale determines the holding change:

Mechanical movement	Transaction needed	Ordinary order mode
Up	Sell	Sell Limit
Down	Buy	Buy Limit
Up	Buy	Buy Stop
Down	Sell	Sell Stop

A functioning Stop can therefore come from the scale. It does not necessarily mean liquidation or capital protection.

At the first step of our type I scale, selling 20 at €12 or buying 20 at €8 are two Limit proposals.

Observe a functioning Stop

Another `ks` list can reverse the expected direction. Keep prices and replace the holding coefficients with `ks:0.4,1.8`:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.4,1.8,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 2
```

The first rise sells 60 at €12, leaving 40. The jump targets 79 at €14: **a Stop purchase of 39 shares**.

On a separate downward path:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.4,1.8,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" -2
```

The first fall buys 80 at €8, taking the holding to 180. The lower jump targets 150 at €6: **a Stop sale of 30 shares**. The trade is not liquidated; 150 shares remain.

The optimised check helps identify some reversals. It guarantees neither sufficient quantities after rounding nor absence of Stops on every type II path.

Round to the tick size

An upward target rounds up; a downward target rounds down, using the tick for the purchase or sale concerned.

A €10.03 reference multiplied by 1.20 gives €12.036. With a €0.05 tick:

```
jedidemo tools calc price \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=0:0:+1 c0=10.03 tick=0.05 dir=up
jedidemo tools calc price \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=0:0:+1 c0=10.03 tick=0.05 dir=down
```

The results are **€12.05** with `dir=up` and **€12.00** with `dir=down`. The second command shows the other rounding of the same number; it does not turn `+1` into a downward target. Declare the tick for the security and market concerned.

Examine small quantities

An apparently substantial initial holding can produce a small order:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 3
```

The three rises sell 20, **1**, then 16 shares. With the default minimum of two shares, quantity 1 indicates degraded functioning. A zero quantity blocks that direction and the order is not added to proposals. A warning can also identify a forthcoming small quantity.

Chapter 18 explains why the opening check does not remove the need to reread orders after sizing and during monitoring.

Distinguish StopLoss

With `money` capital management, Jedi can propose a StopLoss sale of the holding. Its state shows `CL` for loss control or `GP` for protection of a calculated gain. The net quantity option can account for the upward sale order.

This StopLoss comes from a financial projection and the trade state. It is not a third step. It can remove ordinary orders at or below its price. Chapter 18 explains its calculation and limits.

The price indicators in `trade orders` locate proposals relative to the known price. They do not prove execution or guarantee its price. After a partial execution, gap or cancellation, check the holding, orders still active and recorded operations.

18. Understand capital, risk and position sizing

A project must answer two questions: how many shares to buy initially, and how much capital to keep available for subsequent purchases. Capital management provides a risk framework for sizing. `tools candidate` also checks operability and funding, as in chapter 06.

Our figures describe simulations. The theoretical risk does not guarantee the maximum loss actually obtained: fees, rounding or adverse execution can increase it.

After a split, current gains and stop quantities use the **converted holding**, even before a new transaction has been recorded. With reset, the protection path starts from the new mechanical position. These adjustments do not zero the financial result: past purchases, sales and fees remain retained. With merge, automatic proposals wait for the manual transaction, as explained in chapter 09.

Give K, r, o and n a meaning

Parameter	Practical question	Effect
`K`	What capital is allocated to trading?	Monetary basis of calculations.
`r`	What fraction can be risked per trade?	Monetary risk `R = K × r`.
`n`	How far down the scale should we examine a fall?	Price at `n` steps and sizing distance.
`o`	What projected gain must be exceeded before protecting it?	Monetary objective `O = o × R`.

With `K=€20,000`, `r=2.5%`, `n=2` and `o=1`, risk and objective are each €500. `o=2` would raise the objective to €1,000 without changing the initial quantity.

With its original configuration, Jedi accepts `r` from 0.5% to 5%, `n` from 1 to 5, and `o` from 0.5 to 5. These bounds are configurable. Our values illustrate choices to examine; they are not recommendations for every portfolio.

A `money` definition requires all three fields `r`, `o` and `n`: none is implicitly completed. Without `money`, the scale works without the protections described here.

Use the right capital

For a real trade, capital comes from `trading.capital`. For **`tools play`**, it comes from **`simulation.capital`**. `candidate --capital=20000` sets the capital for that evaluation without modifying configuration.

In play, the risk rate is the required r field of money; simulation.risk_rate and simulation.distance are not used. They contribute to score's reference opening amount and backtest without –capital. Appendix 05 gives built-in values and each setting's use.

Before the protection examples, use chapter 02's experiment environment and check its configuration file:

```
simulation:
  capital: 20000
```

Fees remain 0.35%, without a minimum. Part I's €5,000 capital for Exa_01 belongs to another example. Changing only `trading.capital` does not change `play` capital.

Size an opening

Our type I enters at €10. Two downward steps lead to €8, then €6. The distance from entry is 40%.

The risk-derived amount is `R/d`, with `d` expressed as a fraction: €500 ÷ 0.40 = **€1,250**, or **125 shares** at €10.

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:2 --capital=20000 --extended
```

The ISIN identifies the fictional demonstration security. The command supplies the price; no downloaded quote is used.

Useful result	Value
Theoretical price after two steps	€6.00
Risk distance	40%
Monetary risk	€500.00
Initial quantity	125
Opening amount before fees	€1,250.00
Reported minimum operable quantity	82
Candidate maximum requirement before fees	€1,855.60
Margin after candidate before fees	€18,144.40

Jedi compares risk, capital and minimum operability constraints. Risk limits the quantity to 125 here. The maximum according to capital on the risk path is 1,492; this does not allow the risk ceiling to be ignored.

The theoretical €6 price serves sizing. It is not a permanent instruction to sell at €6: the StopLoss evolves with operations.

Keep a reserve for future purchases

The opening uses €1,250, but the downward path without protection requires further purchases:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -3
```

State	Holding	Net investment before fees
Opening at €10	125	€1,250.00
Purchase of 25 at €8	150	€1,450.00
Purchase of 38 at €6	188	€1,678.00
Purchase of 37 at €4.80	225	€1,855.60

`candidate` uses three steps here and a future reserve of **€605.60** beyond the opening. This horizon comes from the operational check and can exceed `n`. The projection examines the scale's full integer purchases; it does not assume that StopLoss will interrupt them.

Reserves for other open trades are deducted from free capital. Fees are excluded from this projection. Also allow for fees and examine requirements beyond the displayed horizon.

A fixed €1,250 opening with only €1,700 capital remains operable, but gives a margin of **−€155.60**:

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --amount=1250 --capital=1700 --extended
```

Being able to pay for the first purchase is therefore insufficient to fund the scenario.

Choose parameters

Start with the assigned capital and the accepted loss amount for the project; derive `r`. Then examine the prices corresponding to `n` for the chosen scale: two steps do not always represent 40%.

These comparisons change one parameter at a time. Each command is complete:

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:1%,o:1,n:2 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
```

```
  --money=r:5%,o:1,n:2 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:1 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:3 --capital=20000
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:2,n:2 --capital=20000
```

Settings	Risk R	Price after n steps	Retained quantity	Objective O
`r:1%,o:1,n:2`	€200	€6.00	0: rejected	€200
`r:2.5%,o:1,n:2`	€500	€6.00	125	€500
`r:5%,o:1,n:2`	€1,000	€6.00	250	€1,000
`r:2.5%,o:1,n:1`	€500	€8.00	250	€500
`r:2.5%,o:1,n:3`	€500	€4.80	96	€500
`r:2.5%,o:2,n:2`	€500	€6.00	125	€1,000

At `r:1%`, the risk constraint cannot reach the reported operable minimum. Jedi rejects the opening and retains zero shares. Increasing `r` to obtain "YES" also increases monetary risk: it must remain a risk decision.

Increasing `n` moves the risk price farther away here and reduces initial quantity. Reducing `n` increases quantity and funding needs. Choose `n` by examining prices, the security's usual movements and the required reserve.

Then inspect integer orders. The reported minimum does not ensure that every larger quantity produces large orders: rounding can make small differences vary irregularly. With `q0=100`, three rises sell 20, 1 and 16; with `q0=125`, they sell 25, 2 and 19. The current opening check does not revalidate every order for the final quantity; complement it with `play` and monitoring diagnostics.

Understand loss control

During the trade, Jedi projects the result after `n` downward steps, including the scale's purchases and sales. If this result is below `−R`, it calculates a `CL` protection targeting that monetary loss.

```
jedidemo tools play -c o0,6-14,17rf -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01,money=r:2.5%,o:1,n:2,loss=fallback" -2
```

At opening, a StopLoss for 125 at €6 coexists with the ordinary purchase at €8. After buying 25 at €8, the holding is 150 and investment €1,450. Protection is recalculated as `(1450−500)/150`, approximately €6.333333, rounded to **€6.33**.

The ordinary purchase at €6 is removed because it is below protection. The simulation sells 150 at €6.33: **€500.50 gross loss** and **€8.40 fees**, or **€508.90 loss after fees**.

`loss=fallback`, the `play` default, first follows an available ordinary downward order, then StopLoss. `loss=active` would prioritise StopLoss on the first fall: the simulated chronology differs.

A gap below protection can worsen the loss. If the known price has already passed protection or the accepted loss, Jedi flags the problem; it cannot guarantee to recover an unexecuted sale.

Understand gain protection

`GP` protection starts when the **projected gain after `n` falls** strictly exceeds `0`. A current gain of €500 is not necessarily enough for `o:1` and `R=€500`.

To observe `o`, simulate 25 round trips of 100 shares between €10 and €12. This deliberately repetitive scenario shows the threshold; it does not predict 25 real profit opportunities:

```
jedidemo tools play -c o0,6-14,17r -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01,money=r:2.5%,o:1,n:2" "25x1,-1"
jedidemo tools play -c o0,6-14,17r -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01,money=r:2.5%,o:2,n:2" "25x1,-1"
```

Both paths finish with 100 shares at €10 and €1,000 gross profit. With `o:1`, gain-protection StopLoss orders appear during the path. With `o:2`, none appears: the projected gain has not exceeded €1,000.

Once `GP` is established, Jedi retains the protected monetary gain or raises it if the projection improves. Its price is recalculated when the holding changes. Protection is calculated before fees and remains subject to actual execution. A larger `o` delays activation here; it promises no larger gain and sets no immediate selling price.

After these checks, `tools backtest` can observe the scale on historical quotations. Its assumptions are explained in chapter 22.

19. Check with tools calc and tools play

An isolated value helps explain a coefficient. A path shows how operations follow one another, including rounding, settings changes and protections. Jedi provides both tools: `calc` for a type I calculation, `play` for a type I or II simulation.

A simulation defined directly in the command creates no trade and records no operation in the main database. The `scale create` and `note create` commands used elsewhere in this part do write to that database: keep chapter 02's experiment environment.

Choose the useful calculation

`tools calc` receives a type I scale, a position and a function. The position is written `x:y:e`, as in chapter 12.

Function	Calculated value	Additional information
`kcb`	Base capital coefficient	Position
`kcp`	Price coefficient	Position
`ksp`	Holding coefficient	Position
`cpe`	Theoretical price	`c0`
`spe`	Theoretical holding	`q0`
`quantity`	Integer quantity to buy or sell	`q0` and current holding
`price`	Tick-rounded price	`c0`, `tick` and direction

At the first upper level, our type I theoretical holding is approximately 78.571429 shares:

```
jedidemo tools calc spe \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 q0=100
jedidemo tools calc quantity \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  position=1:0:0 q0=100 stock=80
```

Jedi rounds the target holding to 79. From 80 shares, the order quantity is therefore **−1**, a sale of one share. The decimal theoretical result is not an executable order quantity.

`calc` does not reproduce fees, protections or continuity across a settings history. It rejects type II scales, whose prices depend on previous executions. For these situations, use `play`.

Define a path with play

The first argument describes the scale, reference holding, reference price and tick size. Following arguments describe actions:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1
```

The `--` separator passes negative movements as arguments. Quotation marks keep the definition in one argument; the final `\` continues the line without adding a space before `q0`.

The opening buys 100 at €10. The rise sells 20 at €12, then the fall repurchases 20 at €10. The holding returns to 100 and net investment before fees becomes €960.

Action	Interpretation
`1, -1`	One upward or downward step
`2, -2`	Two successive steps in the chosen direction
`"2x1,-1"`	Repeat the rise, fall pair twice
`8.00`	Move towards a price of €8
`"20@12"`	Manual purchase of 20 at €12
`"-20@12"`	Manual sale of 20 at €12
`mark`	Set a mark after the previous operation

`8` means eight steps; `8.00` means a price. A movement towards a price can cross several orders. A manual operation normally ends the path; the operation following `merge` reinitialisation receives special treatment, explained in chapter 16.

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" "2x1,-1"
```

These two round trips finish with 100 shares and €920 investment: €80 gross profit at the final €10 price.

Simulate a consolidation with reset or merge

These simulations reproduce 1,790 old shares consolidated into 179 new shares. They use a literal definition: no permanent scale or trade needs to be created. The old €0.80 price describes the starting operation; the €12 resumption price already uses post-consolidation units.

For **reset**:

```
jedidemo tools play -c co0,6-14,17 -- \
  "scale=vr:10%,ks:0.8,1.2,ki:0%,kb:0,q0=1790,c0=0.8,tick=0.01" \
  "split=1:10,reinit=reset,12" 1 -1
```

Section `c` displays settings and section `o` displays operations:

Step	Holding	Price	New settings reference
After split + reset, before transaction	179	Resume at €12	`q0=179`, `c0=12`, position `0:0:0`
First rise: sell 36 new shares	143	€13.20	Same reference
Following fall: buy 36 new shares	179	€11.87	Same reference

The €11.87 price reproduces Jedi's actual rounding at a €0.01 tick. The split is not applied again after the first transaction.

For **merge**, start again in a separate simulation:

```
jedidemo tools play -c co0,6-14,17 -- \
  "scale=vr:10%,ks:0.8,1.2,ki:0%,kb:0,q0=1790,c0=0.8,tick=0.01" \
  "split=1:10,reinit=merge,12" "21@10.00" 1
```

Settings wait for the manual purchase. After `21@10.00`, stock and `q0` equal 200 and `c0` is €11.79. The following rise sells 40 shares at €11, leaving 160 shares. Type II resumes from that manual transaction's actual price for the next movement.

Quantity `21` and price `10.00` describe the **new shares**, after consolidation. A manual sale uses a negative quantity, for example `"-20@14.00"`. Jedi checks sales against converted stock: `"-180@12.00"` would be rejected with only 179 shares available.

Both options belong to the same settings change. `sync` cannot accompany this combination. After `merge`, supply a manual transaction before an automatic movement or target price; otherwise Jedi rejects the scenario. In actual management, record only the transaction really executed.

Choose displayed tables

`--command` letter	Reading
`o`	Operations and holding changes
`c`	Settings and references
`r`	Orders available before operations
`f`	Gross financial result

Letters can be combined, for example `--command=o0,6-14,17crf`. The selector `-c o0,6-14,17` keeps eleven operation columns: identifier, type, mode, quantity, price, position, purchases, sales,

gross result, trade investment and holding. Gross result values the holding at each operation price; it includes more than realised gains.

To display fees too, add column `16`: `-c o0,6-14,16-17`. To append the financial result to the selected operations, add `f`: `-c o0,6-14,17f`. With `operation list`, enter the indices alone, without the letter `o`: `-c 0,6-14,17`.

Table `r` shows orders **before** each operation; an arrow identifies the followed order. Its last row should not be read as recalculated orders after the very last operation.

Compare flows around a mark

A mark divides the financial reading without changing the scale or orders. Only one mark is allowed. For the financial result, place it after an operation following the opening: the base operation cannot serve as a mark.

```
jedidemo tools play -c o0,6-14,17f -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=100,c0=10,tick=0.01" 1 -1 mark -1 1
```

The upward round trip produces €40 gross profit. The second, downward one produces another €40. The total is €80 gross, **€6.30 fees** and **€73.70 after fees** with our 0.35% configuration and no minimum. The financial table separates periods before and after the mark; fees are read in operations when column `16` is added to the selector.

Scale changes and reinitialisations enter the path as in chapter 16. The definition can also contain `money`: then check `simulation.capital` and `loss` mode as in chapter 18.

Start from a recorded operation

Instead of a direct definition, `play` can receive an actual operation identifier. It reuses that context to examine a continuation without changing recorded operations:

```
jedidemo tools play -c o0,6-14,17 -- 3 1 -1
```

Here, **3 is an example identifier**, to replace with an operation that exists in the database used. It is neither a trade name nor a quantity.

Keep commands, configuration and results when comparing settings. `play` follows movements supplied by the user; it does not measure their historical frequency. Part III will explain how `backtest` follows a quotation file and what assumptions it adds.

20. Follow a complete trade example

Bring the calculations together in one path: size an opening, observe flows, change settings, compare two continuations and close the holding. This fictional security is separate from Exa_01 in part I. All paths below run in memory with `tools play`; they create no trade.

The example uses type I, an initial €10 price, a €0.01 tick and 0.35% fees with no minimum. Reproduce `candidate` commands in an experiment database with no open trades.

Size the project

By default, `candidate` checks three upward steps for operability. Help displays the configured value, which is 3 with the original configuration:

```
jedidemo tools candidate --help
```

The `--operability-steps=COUNT` line says “default: 3”. If the configuration of this demonstration space changes, the displayed value follows that setting.

```
jedidemo tools candidate FR4487601001 --price=10 \
  --scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  --money=r:2.5%,o:1,n:2 --capital=20000
```

With €20,000 capital and 2.5% risk, `R` is €500. Two downward steps lead to €6, 40% below entry. The retained quantity is **125**, for an opening of **€1,250**.

The projected maximum requirement before fees is **€1,855.60**, including a €605.60 future reserve. This covers three steps according to the horizon reported by `candidate`. Chapter 18 explains why that horizon differs from `n`.

The following paths **do not activate `money`**: they isolate the scale mechanics. A protected trade can follow other orders. These paths therefore do not imply that a protected trade would execute the same purchases and sales.

Follow operations

```
jedidemo tools play -c o0,6-14,17f -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1
```

Operation	Price	Holding afterwards	Net investment before fees
Opening purchase of 125	€10.00	125	€1,250.00
Purchase of 25	€8.00	150	€1,450.00
Sale of 25	€10.00	125	€1,200.00
Sale of 25	€12.00	100	€900.00
Sale of 2	€14.00	98	€872.00

The mark follows the return to €10. This first round trip produced €50 gross. At the final €14 price, the 98 shares are worth €1,372: overall gross profit is **€500**. It includes the remaining holding’s value; not all this profit has been received in cash.

Maximum investment actually encountered is €1,450. The `candidate` reserve examined another, downward path; it does not predict the purchases on every path.

Reapply the same scale in new settings

Reapply the scale after the sale at €14, then advance one step:

```
jedidemo tools play -c o0,6-14,17cf -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 1
```

New settings retain `q0=125` and `c0=10`, reset local counters to zero and carry cumulative coefficients forward: `kcpc=1.4` and `kspc≈0.785714`. Changing settings triggers no transaction.

The next rise sells **19 shares at €16.80**. The holding becomes 79, net investment €552.80 and holding value €1,327.20. Gross profit is **€774.40**. New coordinates belong to the new settings; past operations retain theirs.

Record an exceptional decision

For a trade actually monitored in the database, a note can explain a decision or check that cannot be read from operations. Here, the following optional write must stay in the experiment database:

```
$ jedidemo note create 2026-10-05 "PEDA20 - settings" "State checked:
> 98 shares held after selling 2 shares at EUR 14.
> Same scale reapplied to check continuity.
> Broker orders to check before continuing."
```

$ and > represent shell prompts and are not typed. The content contains real line breaks. The note belongs to the general journal; `PEDA20` in its summary is a reference, not a trade created by `play`.

```
jedidemo note list PEDA20
```

If this note receives identifier 1 in an empty database:

```
jedidemo note show 1
jedidemo note update 1 --edit
```

Otherwise, use the identifier displayed by `note list`. The editor can later add broker confirmation, including cancellation of old orders. Its setup is described in chapter 08.

Compare a fall instead of the rise

Restart from the same state after the sale at €14 and settings change. Replace only the last movement with a fall:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 -1
```

This fall buys **20 shares at €11.20**. The holding becomes 118 and net investment €1,096.

Continuation after the change	Holding	Final price	Gross profit
One rise	79	€16.80	€774.40
One fall	118	€11.20	€225.60

These branches show possible consequences of the same settings. They give no probability for either.

Close the holding and reconcile the result

In the upward branch, manually sell the remaining 79 shares at €16.80:

```
jedidemo tools play -c o0,6-14,17f -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,\
q0=125,c0=10,tick=0.01" -1 1 mark 1 1 \
  scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1 \
  1 "-79@16.80"
```

This sale deliberately closes the holding; it is not the target quantity of the next ordinary order.

Closed branch result	Amount
Total purchases	€1,450.00
Total sales	€2,224.40
Final holding	0 shares
Gross profit	€774.40
Total fees	€12.88
Profit after fees	€761.52

The financial result splits gross profit into €50 before the mark and €724.40 afterwards. In a real trade, record the broker execution as in chapter 09, with its actual charged fees.

At each stage, reconcile the holding, monetary flows, active settings and next orders. Notes explain exceptional events; they do not replace recording executions. Part III will confront these mechanics with historical quotations and the limits specific to simulation.

Studying historical results

After daily use and the mechanics, this part examines what simulations show on past quotations. It assumes familiarity with scales, operations and the main capital questions.

A **search** compares scales over a period to choose one. A **backtest** simulates a fixed scale on historical quotations under explicit execution conventions. A **check over another period** examines that choice on data not used to tune it. **Dated monitoring** records decisions before observing their consequences.

These stages use Jedi commands, explicit dates and annotated reports. Chapter 21 prepares the question, securities, files and checks. Chapter 22 explains `tools backtest`, its assumptions and compact report. Chapter 23 follows a study on `Euronext_Growth`. Chapter 24 organises comparisons, interpretive limits and the results journal.

Reading distinguishes gross gain, fees, net gain, maximum investment and the valuation of remaining holdings. Daily quotations do not reveal the exact order of intraday movements; simulation rules must therefore be understood before commenting on results.

By the end, the reader will know how to prepare a historical examination, retain its definition and commands, compare scenarios and identify what needs further checking. A favourable past result remains an observation on specific data and assumptions.

21. Preparing a historical study

Ask a question you can check

A study starts with a specific question: how does a given scale behave on a security and period? How much activity does it generate? What maximum investment does it require? How does it respond to a decline or gap?

Write the question before reading gains. Fix the securities, dates, fees and sizing mode. A `score` search chooses scales using its input data; a backtest with a fixed definition examines their path. Changing a setting after reading the result creates a new trial to keep.

Distinguish the periods

Chapter 23 uses this timetable:

Stage	Available data	Use
Preparation	History from early 2021	Checks and historical depth
Shortlisting at 31 December 2024	Quotations through that date	`profile` over the last 252 sessions, ATR over 20
Search at 31 December 2024	Quotations through that date	`score` over the last 750 sessions
Verification	2025 quotations	`backtest` with the already fixed scale

The 252-session window describes the security's recent behaviour. The 750-session window exposes scales to a longer path. Verification data must not help tune the choice they are intended to examine.

This separation does not erase what the author already knows. A study performed after 2025 remains retrospective even if its commands isolate dates correctly. For prospective monitoring, record the choice on its actual date before subsequent observations become available.

Choose securities and keep their list

Specify the scope: market, dataset, group or explicit list. For a study stopping at the end of 2024, seek the list of securities present on that date. A current list may omit companies that left the market and add later listings.

Keep that list with the files. Identify exclusions, short histories and delisted securities. Do not quietly remove a security because its result is unfavourable. A dataset describes memberships in a database; its name certifies neither data provenance nor historical composition.

In an existing database, inspect the scope:

```
jedidemo market info demo --dataset=Demo
jedidemo market list demo --dataset=Demo --last-quote
```

These commands require chapter 10's Demo database. They read the database without downloading quotations.

Check prepared files

Keep the provider, retrieval date, original format, conversions and conditions of use or redistribution. Individual files and dataset import files have different layouts, described in chapter 10.

Check the security identity, currency, date order, duplicates, volumes and consistency of `Low ≤ Open, Close ≤ High`. Check missing sessions against the market calendar rather than simply counting weekdays.

Identify splits, consolidations and other price corrections. Adjusted and unadjusted quotations yield different paths. Do not combine adjusted closes with unadjusted openings and extremes. OHLCV quotations alone are not a dividend or corporate-action cash-flow history handled by the simulation.

To examine an individual file:

```
jedidemo tools profile cotations_EXA.txt
jedidemo tools stats cotations_EXA.txt --date=le:2025-12-31
jedidemo tools plot cotations_EXA.txt --date=le:2025-12-31
```

The profile highlights the characteristics and checks it calculates; `stats` and `plot` supplement inspection. A smooth chart or successful import does not certify a provider's accuracy.

Isolate a year correctly

Jedi accepts **one `--date=OP:YYYY-MM-DD` filter per command**:

Code	Meaning
`ge`	Date greater than or equal to
`gt`	Date strictly greater than
`le`	Date less than or equal to
`lt`	Date strictly less than

The date filter runs before `--select`. Repeating `--date` does not create a range: the last value replaces the previous one. To isolate 2025 using `--date=ge:2025-01-01`, use a file or database **frozen at 31 December 2025**, without later quotations. Chapter 23's supplied file has this boundary.

For a search ending in 2024, an upper bound suffices:

```
jedidemo tools score cotations_EXA.txt --date=le:2024-12-31 \
  --select=l750 --sort=fitness --budget=40
```

This file does not contain 750 sessions before that date; the report states the period actually available. Chapter 23's Growth file supplies the necessary depth. Always read the reported dates and session count.

`--select=l252` keeps the last 252 rows after filtering; `f100` keeps the first 100; `1:100` selects an index range starting at 1. A discontinuous selection such as `f10,-12:-1` joins separate pieces of history and can create an artificial jump between two dates. Use it for inspection while recognising that effect.

`profile` does not assess current freshness with a historical `--date` or `--select` selection. This avoids comparing quotations ending in 2024 with a 2025 or 2026 dataset reference. Check yourself that each security reaches the intended historical boundary.

Freeze a study copy

A database maintained through `market update` changes. Prepare a separate demonstration database for the study and keep the imported files. The `jedidemo` launcher and `JEDI_DEMO_DIR` directory override separate experiments; chapter 02 describes the steps.

Also keep the version and effective configuration:

```
study_dir="$HOME/jedi-study-2025"
mkdir -p "$study_dir"
jedidemo version > "$study_dir/version.txt"
jedidemo config > "$study_dir/config.txt"
jedidemo config --paths > "$study_dir/paths.txt"
```

The directory receives shell output independently of the directory where the launcher runs Jedi. Before sharing a configuration, remove any provider keys it contains.

The study record combines the question, security list, files, periods, scale, fees and commands. Chapter 24 supplies a template.

22. Simulating a scale with tools backtest

A fixed scale on a history

`backtest` reads a security's quotations and simulates operations proposed by a fixed scale. It uses an in-memory simulation database: it adds no trade to your portfolio and does not update market quotations.

The first opening uses the **Open** price of the first selected session where sizing permits the purchase. After complete liquidation, a new opening is attempted on the following session. The report may therefore aggregate several successive trades on the same security.

From the supplied fictional file:

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2"
```

From the Demo database prepared in chapter 10:

```
jedidemo tools backtest --database=demo --security=EXA \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2"
```

`--scale` accepts a definition or the name of a scale already recorded in that environment. Remove the `I:` or `II:` prefix shown by `score` before passing it. Here the constructor produces a type I scale whose operational definition appears in the report.

Choose dates

Without an explicit selection, `backtest` uses **all available history**. It does not automatically reuse `score`'s 750-session window.

```
jedidemo tools backtest cotations_EXA.txt --select=l750 \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2"
```

`--date` and `--select` follow chapter 21's rules. The selected start changes the opening price and mechanical starting point. A test starting with a new trade on 1 January does not automatically continue a trade begun the previous year.

Two sizing modes

Without `--capital`, each opening requests a **fixed amount**, calculated from configuration:

```
amount = integer part of
         simulation.capital × simulation.risk_rate / simulation.distance
```

Default settings give `20000 × 0.015 / 0.2 = €1500`. The integer quantity actually bought may leave a small fraction unused. `--distance` changes this calculation; it does not mean the `n` step count in `money`.

Adding `--money` without `--capital` adds trade protections while retaining fixed-amount sizing.

With **`--capital`**, Jedi also requires **`--money`** and calculates the quantity at each opening from risk, the capital ceiling and operability requirements:

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" \
  --capital=20000 --money="r:1.5%,o:1,n:2"
```

Chapter 18 explains `r`, `o` and `n`. By default this mode requires at least **two shares per checked order** and examines **three upward steps**. `--minimum-quantity` and `--operability-steps` change these checks and require `--capital` mode.

An inoperable opening is postponed to the next session and counted in the report. `--distance` cannot be combined with `--capital`. The supplied capital remains the sizing reference: the backtest does not simulate simultaneous funding of a multi-security portfolio or automatically reinvest gains through increased capital. Use `candidate` to examine funding for an actual project with your portfolio's reserves.

Know the execution assumptions

Normal operation uses **`--execution=conservative`** and **`--opening-day-orders=next`**. Orders generated by the initial purchase become active on the next session. The engine retains **at most one**

proposed operation per session, in addition to the opening when it occurs; it does not execute every step that a day might cross.

Daily data provide Open, High, Low and Close, without revealing movement order between opening and closing. Jedi therefore applies explicit rules:

1. An order already executable at the opening price takes priority over an order only touched later in the session.
2. When several orders remain possible without known chronology, the policy chooses by order type. It generally prioritises protection, then the downward purchase, then the upward sale. For loss protection not yet crossed at the opening, the downward purchase may precede protection; gain protection takes priority over that purchase.
3. The opening price applies when a gap makes the order immediately executable; otherwise the order level applies.

Active order	Level	Open	Simulated price
Limit purchase	€8	€7	€7
Limit sale	€12	€13	€13
StopLoss sale	€6	€4	€4
Stop purchase	€12	€13	€13

A favourable gap can improve a Limit price; an adverse gap can exceed the Stop level and intended risk. `conservative` names this convention without guaranteeing the worst possible intraday path.

The simulation does not reconstruct the order book, partial fills or spread. Daily volume does not automatically cap executed quantities. Fees come from configuration; simulated prices do not replace actual tick rules and broker conditions.

`--execution=legacy` and `--opening-day-orders=same` remain available for specific investigations. `legacy` changes priorities and uses theoretical levels instead of gap prices. `same` permits orders on the opening day. State these explicitly in any comparison using them.

Read the compact report first

The first command in this chapter, using EXA's 851 sessions, produces:

```
History: 2023-06-30 → 2026-10-02 (851 sessions)
Requested amount per opening: 1500.00 €
Openings: 1; inoperable attempts: 0
First purchase: 2023-06-30, 150 shares for 1500.00 €
Operations including openings: 25
Cumulative gross gain: 647.11 €
Total fees: 19.14 €
Cumulative net gain: 627.97 €
Maximum investment excluding fees: 1500.00 €
Net gain / maximum investment: 41.86%
Shares held at the end: 122; value 1578.97 €
```

Gross gain adds sales, subtracts purchases and adds remaining shares valued at the last close. Net gain deducts fees. Maximum investment is the largest of the individual trades' peaks of purchases minus sales, excluding fees. Backtest trades run sequentially without overlap, so the global maximum must occur in at least one trade.

The 41.86% covers the whole period; it is not an annual rate. The final €1,578.97 value is part of the gain, not another gain to add to €627.97. The 122 shares are not artificially sold at the end: the report charges no final liquidation that did not occur.

Understand the maximum when a trade reopens

Another invented example illustrates the calculation: the first trade buys 100 shares at €10 and sells all of them at €8.50. Its gross loss is €150. The next trade buys 100 shares at €10 again.

Trade	Peak investment before fees	State
Trade_01	€1,000	Closed, with a €150 gross loss.

Trade	Peak investment before fees	State
Trade_02	€1,000	Opened after the previous trade closed.
Backtest maximum	**€1,000**	The larger of the two peaks.

Cumulative purchases minus sales then total €1,150: the €150 loss remains in the financial balance. This balance is used to calculate cumulative gain; it is not an individual trade's maximum. Detailed and technical reports label it the **buy-sell balance**. It can exceed the maximum, particularly after an earlier loss.

The financial table requested with `-c f` now follows **trade creation order**, first to last. Compact summary r, the technical summary and the f total use the same maximum. Returns and ratios based on that maximum are recalculated using this value.

Examine operations and gaps

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="kc:1.23,1.14,0.88,0.76,ki:0%,kb:1,kd:0.2" -c rfo
```

`r` retains the report, `f` adds financial results and `o` operations. `t` lists trades, `d` remaining active orders and `p` the chart. Detailed outputs may be wide: inspect them in the terminal or save them to a file. `play` column selectors are not `backtest` options.

The small `cotations_GAP.txt` file checks a loss beyond intended risk:

```
jedidemo tools backtest cotations_GAP.txt \
  --scale="kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1" \
  --capital=20000 --money="r:2.5%,o:1,n:2" -c ro
```

Jedi buys 125 shares at €10, then executes protection at the next opening at €4 despite its €6 level. Nominal risk of €500 becomes a €750 gross loss and a €756.13 net loss after fees. This fictional two-session path explains a rule; it is not a return study.

Compare fixed scenarios

Changing only the scale on the same data with identical options makes a readable comparison. For example:

```
jedidemo tools backtest cotations_EXA.txt \
  --scale="vr:5%,np:2,nn:2,ki:0%,kb:1,kd:0.0"
```

Keep losing scenarios and the assumptions explaining them too. `--display=technical` adds detailed compatibility diagnostics; the compact report provides the initial economic reading. Chapter 24 organises interpretation across scenarios.

23. Euronext Growth study example

Numerical captures in this chapter come from the 1.0 edition's reference study. With the corrected maximum investment in v1.1, rerun the commands to obtain updated investment values, returns and rankings.

A path to reproduce, then adapt

This chapter shows how to organise a study of Euronext Growth securities: shortlisting at the end of 2024, searching earlier data, then checking 2025. Supplied files contain **three fictional securities**, EXA, EXB and EXC; they are not an Euronext Growth export. The dataset name demonstrates study organisation.

Outputs are calculated on these demonstration data. They show neither the Growth market's return nor that of actual securities. Written after 2025, this example is **retrospective**: command dates constrain calculations without turning the exercise into a decision actually made in 2024.

Prepare a frozen historical database

After installing the launchers in chapter 02, copy TXT files from `examples/` to `~/jedi-examples/`. Use a main experiment database without committed trades and these settings in its configuration:

```
fees:
  rate: 0.0035
  min: 0.0
simulation:
  capital: 20000
  risk_rate: 0.015
  distance: 0.2
```

`Growth_demo_2021_2025.txt` contains 3,909 quotation rows: 1,303 weekdays per security from 4 January 2021 to 31 December 2025. `labels_growth_demo.txt` supplies identities. Membership stays fixed; the simplified calendar includes public holidays.

Create a database separate from Demo, once:

```
jedidemo market init growthdemo
jedidemo market import growthdemo Euronext_Growth \
  Growth_demo_2021_2025.txt labels_growth_demo.txt
jedidemo market info growthdemo --dataset=Euronext_Growth
jedidemo market list growthdemo --dataset=Euronext_Growth --last-quote
```

Check three securities, 3,909 quotations and a final quotation of 31 December 2025 for each. Do not extend this database through a 2026 update: it is this study's frozen input.

Shortlist using end-2024 data

```
jedidemo tools profile --database=growthdemo --dataset=Euronext_Growth \
  --date=le:2024-12-31 --rank-criteria=4 --include-rejected
```

The calculation has four years of historical depth and uses the last 252 sessions through that date for recent criteria. It gives:

Security	Total	Trend/year	Median traded value	ATR20	Sw5/year	Status
EXA	6	10.2%	€1.2M	3.5%	12.5	retained
EXC	6	1.8%	€737k	3.5%	20.8	retained
EXB	—	−19.9%	€741k	3.4%	9.4	rejected

The first four ranks form the total; the other three remain informative. EXB is rejected for trend. EXA and EXC have the same total: EXA appears first through rank tie-breaking in importance order. EXB does not contribute to these ranks.

Current freshness is not assessed because history is filtered. Check that all three histories reach the end of 2024; the end-2025 freshness date shown by `market info` does not measure a lag here.

Choose EXA for the rest of this exercise. In an actual study, also record why it was selected over EXC.

Search for a scale before 2025

```
jedidemo tools score --database=growthdemo --security=EXA \
  --date=le:2024-12-31 --select=l750 --sort=fitness --budget=40
```

The selected period is **2022-02-16 → 2024-12-31**, or 750 sessions. Adaptive search evaluates 40 scales among 11,793 available candidates. The first compact result is:

```
Fitness    Op/mo   Max.inv  Typed builder
  0.437    0.924    1.000  I: kc:1.5,1.38,1.23,1.14,0.52,0.4,
                             ki:0%, kb:1, kd:0.2
```

The definition is split over two lines here for readability; pass it as a single value in the command. Gross fitness of 0.437 covers the search period without deducting fees. It does not predict 2025 fitness. The limited budget means the first row is best under the selected sort among scales actually evaluated.

For this exercise, fix that definition, fees of 0.35%, €1,500 requested per opening and default execution conventions. An actual choice also requires understanding the mechanics and checking operability at the contemplated opening price.

Check the choice on 2025

```
jedidemo tools backtest --database=growthdemo --security=EXA \
  --date=ge:2025-01-01 \
  --scale="kc:1.5,1.38,1.23,1.14,0.52,0.4,ki:0%,kb:1,kd:0.2"
```

The database ends on 31 December 2025, so the lower date bound isolates that year. The test starts with a new trade; it does not carry positions from the search path.

```
History: 2025-01-01 → 2025-12-31 (261 sessions)
First purchase: 2025-01-01, 108 shares for 1498.56 €
Operations including openings: 7
Cumulative gross gain: -13.11 €
Total fees: 7.10 €
Cumulative net gain: -20.21 €
Maximum investment excluding fees: 1498.56 €
Net gain / maximum investment: -1.35%
Shares held at the end: 108; value 1450.47 €
```

The favourable search path is followed by a slightly negative net result. Sales and repurchases produce round trips, but final holdings also contribute to the result. Adding `-c rfo` displays the seven operations.

The 1 January start reflects the fictional calendar, not an actual market session. The result describes this file, scale and conventions. Keep it unchanged before trying other settings.

Keep commands and results

```
study_dir="$HOME/jedi-study-growth-2025"
mkdir -p "$study_dir"
jedidemo version > "$study_dir/version.txt"
jedidemo config > "$study_dir/config.txt"
jedidemo tools score --database=growthdemo --security=EXA \
  --date=le:2024-12-31 --select=l750 --sort=fitness --budget=40 \
  > "$study_dir/score-before-2025.txt"
jedidemo tools backtest --database=growthdemo --security=EXA \
  --date=ge:2025-01-01 \
  --scale="kc:1.5,1.38,1.23,1.14,0.52,0.4,ki:0%,kb:1,kd:0.2" \
  > "$study_dir/backtest-2025.txt"
```

These redirections keep standard reports; `score` progress may remain visible in the terminal. Attach both source files, the security list, question and selected definition. Chapter 24 explains the dated journal; a note written today must use its actual date.

Move to actual data

Prepare securities actually present on Euronext Growth at the selected date, with their histories and relevant corporate actions. Describe exclusions and data availability for companies that left the market. Check currency and volume units.

Import these files into **another study database**, with a distinct name and explicit final boundary. Adapt source and security names in the commands. Keep shortlisting, search and verification separately.

If several securities are simulated separately, their maximum investments and gains do not automatically describe a fundable portfolio: purchases may coincide. Then examine opening projects and reserves with `candidate` at the price and date of the actual decision.

24. Interpreting, comparing and monitoring results

Explain the euros before ranking

A result belongs with its dates, scale, first purchase and conventions. Compact measures answer different questions:

Measure	Question
Gross gain	What is sales − purchases + final holdings value?
Fees	What brokerage cost was charged to operations?
Net gain	What remains after those fees?
Maximum investment	What is the largest peak of purchases minus sales within any successive trade, excluding fees?
Net gain / maximum investment	How does the net result compare with the largest amount committed within a trade?
Activity	How many operations and interventions does the path entail?
Final holdings	How much of the result still depends on shares held?

Sales are not all profit, and a share still held is not cash received. The report values remaining holdings at the last close without adding a fictional final sale.

`score` fitness divides **gross** gain by the first purchase. `backtest` compact return divides **net** gain by maximum investment. Comparing the percentages without checking numerators, denominators and periods gives a misleading reading. Appendix 04 collects these definitions.

Compare on a common basis

Keep securities, quotations, dates, fees, sizing and execution rules identical. Fix scales before viewing the comparison period. Record net result, fees, peak investment, operations and final holdings for each.

A more active simulation may earn more gross and less net. Another may appear to use little capital because sales reduced investment while still holding a substantial stock. A large gain during a continuous rise does not by itself demonstrate the benefit of round trips.

A simple two-scale comparison is often enough. `score --sort=activity` and `--sort=investment` help explore trade-offs; with adaptive search, they also change refinement. To compare the same scales, reuse their definitions in `backtest`.

Read the path's limits

Daily data do not reveal each movement's chronology. One proposed order is retained per session. Gaps, actual execution costs, partial fills, trading interruptions and corporate actions can produce a different real path.

Chapter 22 shows how a gap exceeds nominal risk. A StopLoss triggers action subject to market possibilities; it guarantees neither a price nor a loss ceiling. Several individual backtests do not replace a check of common funding across trades.

Annualisation available in some expert outputs transforms a ratio according to duration. It adds no observations and does not establish that a return is repeatable. For a short period, euros, operations and final holdings remain the first information to read.

Check other dates

Keep the selected scale and examine another period, including a different market phase. State whether each test starts with a new purchase or follows an existing actual trade. Different start dates give different reference prices and integer quantities.

For a new period, prepare the final boundary as in chapter 21 and keep the choice before reading the result. If you subsequently adjust the scale, retain the first report and create a new study entry. Trial count matters: searching many settings makes a favourable best result less surprising.

Keep a dated journal

The journal preserves what was decided, why, with what data, then what was observed. `note` suits study choices and exceptional trade events: gaps, suspensions, broker incidents, partial fills, parameter changes or corporate actions.

An editor makes a complete note easier:

```
jedidemo note create --edit
```

Or enter multiline text:

```
jedidemo note create 2026-10-07 "Growth 2025 study: protocol" "Retrospective study using fictional data.
Choice calculated through 2024-12-31; verification on 2025.
Fixed scale: kc:1.5,1.38,1.23,1.14,0.52,0.4,ki:0%,kb:1,kd:0.2.
2025 net: -20.21 EUR; maximum investment: 1498.56 EUR."
```

The date is when this note is written; it does not claim a decision was made in 2024. Use the actual date for your own journal. Do not mix positional fields with `--date`, `--summary` or `--contents`.

A note describes an event; it changes neither operations nor orders. Record each actual execution separately through `operation`, then check the trade state. Keep study files outside the note and mention their location.

Reproduction record

Field	What to keep
Question and date	Objective, actual choice date, prospective or retrospective character
Software	`version` output, language and relevant configuration
Data	Provider, retrieval, files, conversions, adjustments and usage rights
Population	Dated security list, exclusions and reasons
Periods	Choice boundary, search window, verification start and end
Settings	Exact scale definition, amount or capital and `money`, fees
Execution	`conservative` or `legacy`, `next` or `same`
Commands	Complete commands and saved reports
Results	Gross, fees, net, maximum investment, operations and final holdings
Follow-up	Practical decision, later changes and exceptional events

This record lets you reread a study, explain differences between reports and distinguish the initial choice from later modifications. For an actual project, return to `candidate` with the current price, then to the monitoring of executed operations described in part I.

For actual operations, check fees against execution statements. Correct an error with `operation fees ID AMOUNT`, as described in chapter 08, then check `trade results`. This changes the recorded portfolio's net results; `backtest` and `score` recreate their operations and fees from their own settings.

Appendix 1. Command and option reference

Built-in values and the use of every configuration entry are detailed in appendix 05.

Find help for the installed version

Run `jedi` without arguments to see command families. A family alone, such as `jedi tools`, shows its subcommands. Detailed help belongs at subcommand level:

```
jedidemo tools backtest --help
jedidemo market import --help
jedidemo note create --help
```

Read `version` and `config` directly: `jedi version`, `jedi config` and `jedi config --paths`. They have no help subcommand. The supplied jedi-help-en.txt collects reference-version help; your executable's help describes the version actually in use.

Names may be abbreviated to an unambiguous prefix. Examples keep full names. Named and positional fields are alternative forms: do not mix them for the same fields. Quote names containing spaces and expressions containing shell-interpreted characters.

Global `--border=PRESET` precedes the command family. Styles are `ascii`, `reducedascii`, `modern`, `reducedmodern`, `fancy`, `markdown`, `empty`, `blank`. `--` ends option parsing and is useful before negative arguments, particularly in `play`.

Portfolio and journal

Family	Subcommands	Use
`security`	`create`, `update`, `delete`, `show`, `list`	Identity, ticker, buy/sell ticks and purchase tax
`quote`	`get`, `set`, `update`, `refresh`	External query, manual entry or recording current quotations
`scale`	`create`, `update`, `delete`, `show`, `list`	Recorded type I or II scales
`trade`	`create`, `update`, `delete`, `show`, `list`	Trade identity and capital management
`trade`	`orders`, `status`, `results`, `plot`	Orders, state, results and chart
`settings`	`create`, `delete`, `show`, `list`	Scale applying at a date/time, split, synchronisation and reset
`operation`	`create`, `fees`, `delete`, `list`	Actual executions; fee correction by operation identifier
`note`	`create`, `update`, `delete`, `show`, `list`	Journal and exceptional events; `--edit` for an editor
`database`	`dump`, `load`, `schema`	Main database backup, restore and schema

Each help page describes required fields. Deleting an entity in use may be refused; deleting an operation does not automatically replace it. Chapters 08 and 09 show corrections and adaptations.

`quote get ISIN` displays an external price without recording a security or quotation. `quote set` records a manual price. `quote update` updates one registered security; `quote refresh` handles a selection, defaulting to securities of running trades. `--mode=live|eod` applies to `update` and `refresh`, not `get`. `--force` belongs to `refresh` and forces updates according to its filtering.

Correct operation fees

```
jedidemo operation fees 42 3.50
```

```
jedidemo operation fees 42 0
jedidemo operation fees --help
```

Syntax is `jedi operation fees <operation_id> <amount>`. The numeric identifier belongs to the operation, found with `operation list`, rather than the trade. The amount replaces **total** fees, including any transaction tax; zero cancels them. Use a decimal point and a non-negative value, rounded to cents. An earlier operation or one in a closed trade can be corrected. Net results, lists and exports use the new value. Historical fees are also copied into a new `play` starting from an operation in the same trade. Chapter 08 gives the complete procedure.

Adapt settings: split, reinit and sync

New settings require `--scale`, `--date` and `--time`. Their timestamp must follow the latest settings and operations across the whole database; the trade's previous settings must already have been used by an operation.

Option or combination	Effect
`--split=NEW:OLD`	Convert units; `1:10` means one new share for ten old shares.
`--reinit=reset,PRICE`	Current stock as `q0`, chosen price as `c0`, position `0:0:0`.
`--reinit=merge,PRICE`	Pending reference; an actual manual transaction then defines `q0` and `c0`.
`--split` + `--reinit=reset,PRICE`	Accepted in next-version / v1.1; reset from converted whole stock.
`--split` + `--reinit=merge,PRICE`	Accepted in next-version / v1.1; merge from converted whole stock.
`--sync=POSITION`	Synchronisation restricted to type I scales.
`--sync` + `--split` or `--reinit`	Rejected.

With split + reinit, `PRICE` and values of the following operation use **post-split units**. While merge waits for its operation, Jedi proposes no automatic order. Fractional rights are not accounted for. Chapter 09 gives practical cases and chapter 19 gives simulations.

Market databases

Subcommand	Main arguments	Use
`market init`	`SOURCE`	Create a market database
`market import`	`SOURCE DATASET QUOTES LABELS`	Import a complete dataset
`market replace`	`SOURCE DATASET QUOTES LABELS`	Replace dataset contents
`market update`	`SOURCE DATASET QUOTES LABELS`	Merge without deletion; ignore duplicates and report conflicts
`market sync-group`	`SOURCE GROUP FILE`	Create or synchronise group membership
`market group`	`SOURCE GROUP ISIN`	Add/remove a member using `--add` or `--remove`
`market info`	`SOURCE`	Metadata and checks; optional `--dataset=CODE`
`market list`	`SOURCE [REGEX]`	List; `--dataset=CODE`, `--last-quote`
`market quote`	`SOURCE ISIN`	History; `--date`, `--select`

`--universe=NAME` chooses the universe, `france` in default configuration. QUOTES, LABELS and FILE are file paths. Chapter 10 explains formats and the effects of imports, replacements and updates.

`import`, `replace` and `update` accept `--date-order=auto|ymd|dmy|mdy`, `--year-pivot=NN` and `--from=DATE`. Updates refuse an extension more than seven calendar days beyond the current end unless `--allow-gap` is supplied. Update does not overwrite conflicting quotations; use `replace` with a corrected corpus. `sync-group` accepts `--as-of=DATE` and `--reference-source=NAME` to describe membership.

`market list` accepts field regex expressions, including `group` and `dataset`, with `&&` and `||`. This does not provide a multi-group option to `profile`. Jedi exposes one `--group=CODE` in `profile`; no dedicated command deletes a group. `market quote` displays an OHLCV table and offers no CSV export.

Common listing options

Regex filters in lists supporting them are case-insensitive. `field=pattern` targets a field; `&&` means AND, `||` means OR, with AND precedence. Parentheses are unsupported. Available fields vary by command: read its help.

`--running` limits some lists to running trades. `--select` chooses records after filtering; indices start at 1 and negative indices count from the end. `fN` and `lN` mean first and last rows, `I:J` is a range, and commas combine selections.

Lists offering `--csv` may add headings with `--headers`. `--extended` is not universal: for `operation list` CSV, it adds associated names. `--column-indexes` chooses display columns and cannot be combined with CSV.

For a readable operation list:

```
jedidemo operation list -c 0,6-14,17
```

The `o` in `play -c o0,6-14,17` selects a section; it does not precede indices in `operation list`.

Historical tools: source and selection

`profile`, `score`, `backtest`, `plot` and `stats` accept an individual file or `--database=SOURCE --security=IDENT`, with optional `--universe`. IDENT is an ISIN or ticker. `profile` adds collective `--dataset` and `--group` selections and repeated `--security` options.

One `--date=OP:YYYY-MM-DD` filter applies per command, using `ge`, `gt`, `le`, `lt`. Date filtering precedes `--select`. Repeating `--date` replaces its previous value. To isolate a year with a lower bound, freeze the source at its upper boundary as in chapter 21.

tools profile

Option	Purpose
`--dataset=CODE`	Select a dataset
`--group=CODE`	Filter by one group
`--include-rejected`	Display rejected securities without ranking them
`--rank-criteria=N`	Add the first N ranks, 1–7; default 7
`--sort=KEY`	`global`, `ticker`, `trend`, `liquidity`, `volatility`, `oscillation`, `drawdown`, `gap`, `step`
`--limit=COUNT`	Limit displayed rows

The seven criteria are trend, liquidity, volatility, oscillation, drawdown, gap and step, in that order. The first four guide shortlisting; chapter 04 explains practical choices and statuses. Recent criteria use 252 sessions and ATR uses 20; depth and freshness are separate checks.

tools score

Option	Values and default
`--sort`	`fitness` by default; `activity`, `investment`; aliases 1, 2, 3
`--ranking`	`raw` by default; `jedi`, `both`
`--search`	`adaptive` by default; `grid`
`--budget`	500 search backtests by default in adaptive mode
`--display`	`compact` by default; `study`, `technical`

Without an explicit constructor, search explores three internal families. Positional constructors `P1`, `P2`, `P3` select one family for expert use. Default selection keeps the last 750 sessions after date filtering.

`raw` directly sorts fitness, activity and maximum investment. `compact` displays its top 20 with typed builders. `study` adds net economic interpretation and fees. `technical` exposes diagnostics and parameter influence in adaptive search.

Expert `jedi` ranking adds a preference based on engine checks, including operability, activity and robustness diagnostics; `both` displays both readings. Robustness diagnostics examine variants or subdivisions and do not guarantee future behaviour. Keep ranking, display mode and search strategy with results. Chapter 05 introduces normal use in `raw` mode.

tools backtest

Option	Purpose and default
`--scale=DEF`	Literal definition or recorded name, required
`--money=DEF`	Capital management; required with `--capital`
`--capital=AMOUNT`	Sizing through capital, risk and operability
`--distance=COEFFICIENT`	Fixed-amount calculation; incompatible with `--capital`
`--minimum-quantity=COUNT`	Minimum per checked order, default 2; capital mode
`--operability-steps=COUNT`	Checked steps, default 3; capital mode
`--execution=MODE`	`conservative` by default; `legacy`
`--opening-day-orders=WHEN`	`next` by default; `same`
`--display=MODE`	`compact` by default; `technical`
`-c`, `--command=COMMANDS`	Sections `r`, `f`, `o`, `t`, `d`, `p`

Without `--capital`, the amount comes from simulation configuration; `backtest` has no `--amount` option. Compact output defaults to `r`, technical output to `rf`. `T`, `K`, `S` retain extended compatibility lists. See chapter 22 for gap prices, reopenings and final valuations.

tools candidate

The ISIN is positional and `--scale` is required. `--price` replaces the recorded price and allows evaluation of an unregistered security. Choose either `--amount` or `--money`; they cannot be combined. `--capital` supplies K for funding and, with `money`, sizing; otherwise K comes from configured dedicated capital.

`--minimum-quantity` defaults to 2; `--operability-steps` defaults to `diagnostic_steps`, **3 in the initial configuration**. Help shows the effective value. `--extended` exposes sizing detail. Chapter 06 explains operability and funding separately.

Scale definitions

Signatures recognised by `scale create --builder=...` and the tools are:

Fields	Form
`kc,ks,ki,kb`	Type I with explicit price and stock lists
`kc,ki,kb,kd`	Type I with optimised construction of `ks`
`vr,ks,ki,kb`	Type II with an explicit stock list
`vr,np,nn,ki,kb,kd`	Type II with optimised construction of `ks`
`vr,np,sr,nn,mi,ki,kb`	Type II compatibility builder

Without `--name`, `scale create --comment=... --builder=...` assigns `B_kc_NN` for type I or `B_vr_NN` for type II, including Legacy. The next number is computed separately within each family. An explicit name must match the type and use a positive number of at least two digits. Equivalent operational definitions are refused at creation and update, even with `--force`. Chapter 13 details these rules and preservation of historical data.

`kc` and `ks` are decimal lists; `np`, `nn`, `kb` are integers. `vr`, `sr`, `mi` and `ki` accept a decimal coefficient or percentage; `kd` is decimal. Use a decimal point: `kc:1.4,1.2,0.8,0.6`. Commas separate values and fields.

In the compatibility builder, `sr` sets the target stock fraction at the final internal upward step; `mi` sets the budget for internal downward purchases relative to the initial amount. `np` and `nn` include boundaries, leaving `np-1` and `nn-1` internal steps. This form has its own rules: examine its operational definition and orders through `scale show` and `play` before comparing it with others.

tools calc, play, plot and stats

`calc` accepts `kcb`, `kcp`, `ksp`, `cpe`, `spe`, `quantity`, `price` functions with the necessary `key=value` parameters. It calculates **type I** mechanics. `play` handles types I and II, using a created state or an existing operation identifier. Arguments describe movements, target prices, transactions or setting changes.

In `play`, `-c o0,6-14,17` selects operation columns; add 16 for fees. Sections are `t`, `k`, `s`, `c`, `o`, `r`, `f`, `d`; `--verbose` expands messages. Chapter 19 gives complete commands. Random arguments, such as movement `0`, need particular care for reproducibility.

`plot` draws prices and their regression; `--legend=tl|tr|bl|br` selects the legend corner, otherwise automatic. `stats` shows statistical indicators for selected history. Both supplement data inspection; their outputs do not choose an investment for you.

Appendix 2. Configuring API and Web quote providers

This appendix supplements chapter 08. It is for someone who needs to **create or change** a current-quote provider. This work requires the ability to read API documentation or HTML, write a regular expression, inspect an HTTP response and check the source's access conditions. A configured provider is not necessarily available or authorized. Addresses and responses in the examples below are **invented**: they explain configuration structure and do not fetch a real quote.

Appendix 05 lists every built-in value, including fields that are currently unused.

Built-in providers and separate databases

Default configuration tries ABC Bourse, Tradingsat and Boursorama Web pages, then the Alpha Vantage API. These are collection definitions whose availability depends on the provider and its current response structure. `live` identifies Web collection without promising real-time or undelayed prices. `eod` identifies the API path; check the session actually returned.

These calls handle the main database's current quotation. They do not populate OHLCV history or replace `market import` and `market update`, which read prepared files. The `quotlife` refresh interval defaults to 240 minutes; `quote refresh --force` forces updates according to its selection.

Examples use `jedidemo` in the experiment environment. The same subcommands with `jedi` apply to the production portfolio.

Locate the file and understand the order of attempts

`jedidemo config --paths` displays the actual configuration file path (`config_file`) and log path (`log_file`). In normal operation the file is named `jedi.conf`; development and test environments use other names. It uses **YAML**, whose indentation places keys under a parent. Jedi starts with built-in settings and merges the personal file into them: nested mappings are merged, while an overriding scalar or list replaces its built-in counterpart. A small override is possible, but supplying `website_order` replaces the entire built-in list.

```
jedidemo config --paths
```

Current-quote providers are configured under `market_data`. That YAML name does not make `quote get` read Jedi's historical market database. The built-in order attempts `live` and then `eod`. `live` reads Web pages; `eod` queries an end-of-day API. Within each mode, Jedi tries provider names in configured order and then candidate identifiers for that provider. The first accepted quote ends the search.

YAML key	Role in Jedi 1.0
`market_data.mode_order`	String or list of `live` and `eod`; missing or empty falls back to `live`.
`market_data.website_order`	Names to try from `websites`; missing or empty traverses the entries in `websites`.
`market_data.api_order`	Names to try from `api_providers`; missing or empty traverses their entries.
`market_data.websites`	Web page provider definitions.
`market_data.api_providers`	API provider definitions.

An ordered name without a matching definition is skipped. To try only a chosen provider, set its corresponding order; leaving a definition in `websites` or `api_providers` does not activate it when a nonempty order omits its name. Built-in Web definitions, including ABC Bourse and Boursorama, do not establish permission to collect data: check the provider's access conditions before use.

Configure a Web provider

This example describes an **invented page** whose HTML would contain `data-price="12,34"` and `data-change="-0,50"`. The `.invalid` address identifies no usable service. Replace it with details of a source whose access is permitted and whose response has been checked.

```
market_data:
  mode_order: ["live"]
  website_order: ["demo_web"]
  websites:
    demo_web:
      url: "https://quotes.example.invalid/security/{key}"
      key_strategy:
        candidates:
          - template: "{isin}"
      regex_price: 'data-price="(?<price>\d+(?:[.,]\d+)?)"'
      regex_variation: 'data-change="(?<variation>(?:&#x2B;|&#x2D;|[+-])?\d+(?:[.,]\d+)?)"'
```

Jedi can read this configuration when placed under `market_data` in the configuration file. It does not guarantee that real HTML matches or that the number is the correct quote. The named `price` capture is mandatory in `regex_price`. If `regex_variation` is configured, its named `variation` capture must produce a number; otherwise this provider attempt fails and Jedi tries its configured fallback. If variation is **missing, null or an empty list**, Jedi returns `0.0` for variation. That means "not extracted" here, rather than "the price did not change." `regex_price` cannot be missing, null or empty.

Key under `websites.<name>`	Meaning and shape
`url`	HTTP(S) address normally containing `{key}`; Jedi encodes the generated key in that path segment.
`key_strategy.candidates`	Ordered `template` list using `{isin}`, `{ticker}`, `{name}` or derived variables declared in `vars`.
`key_exceptions`	Optional `ISIN: key` mapping; one matching key replaces ordinary candidates for that ISIN.
`regex_price`	Required pattern containing `(?<price>...)`.
`regex_variation`	Optional pattern containing `(?<variation>...)` when provided.
`http_error_patterns`	Optional patterns recognizing an error page near its start; `null` disables this check, and a list replaces built-in patterns.
`min_body_chars`, `min_body_lines`	Minimum body length in characters or lines; defaults are 100 characters and no line minimum. Earlier `min_response_chars` and `min_response_lines` names are also read.

For each candidate Jedi expands known variables in its template. If an unresolved field remains, the candidate is omitted. `quote get <isin>` supplies only the ISIN; templates requiring ticker or name work only where those fields are available from a registered security, for example with `quote update`. Duplicate keys are removed. If no candidate can be formed, Jedi makes no page request.

For a derived variable, the code reads this form:

```
key_strategy:
  candidates:
    - template: "{slug}-{isin}"
      vars:
        slug:
          from: "name"
          ops:
            - downcase
            - {gsub: {pattern: "\\.", replace: ""}}
            - {gsub: {pattern: " ", replace: "-"}}
```

`from` chooses `isin`, `ticker` or `name`. Supported operations inside `vars.<variable>.ops` are `trim`, `upcase`, `downcase` and `gsub`, a regular-expression replacement. Here `"\\."` is YAML's way to write regex `\.`. **An `ops` field directly on a candidate is not applied by `ScrapingKeyBuilder` in Jedi 1.0**: put operations in `vars` as above. A built-in Boursorama candidate has such a field, but a new provider should not rely on that effect.

Accepted regex shapes

Both `regex_price` and `regex_variation` accept a string or a list. A string in the list is an alternative pattern; a nested list of strings is **joined into one pattern**, then tried. Alternatives are attempted in order until numeric extraction succeeds:

```
regex_price:
  - 'data-price="(?<price>\d+(?:[.,]\d+)?)"'
  - ['class="last">', '(?<price>\d+(?:[.,]\d+)?)', '<']
```

The second entry's three fragments are joined without spaces or newlines. YAML single quotes preserve regex backslashes. The capture must represent a number Crystal can parse: Jedi replaces ` `, trims surrounding whitespace, and converts decimal commas to periods, `+` to `+`, and `-` to `-`. It does not automatically strip all spaces inside numbers, currency signs or every HTML entity. Verify the captured value, security and date, rather than merely checking that a regex matches some number.

Configure an API provider

The built-in API illustrates Alpha Vantage's `GLOBAL_QUOTE` response shape. The following example retains **the structure of its keys** but uses an invented address and provider name. It describes a service compatible with this particular response structure; it is not a universal adapter for all JSON APIs.

```
market_data:
  mode_order: ["eod"]
  api_order: ["demo_api"]
  api_providers:
    demo_api:
      api_key: "REPLACE_WITH_YOUR_OWN_KEY"
      symbol_candidates:
        mode: "ticker_plus_suffixes"
        suffixes: [".PA", ""]
      quote:
        base_url: "https://api.example.invalid"
        path: "/query"
        query:
          function: "GLOBAL_QUOTE"
          symbol: "{symbol}"
          apikey: "{api_key}"
        price_path: ["Global Quote", "05. price"]
        prev_close_path: ["Global Quote", "08. previous close"]
        trading_day_path: ["Global Quote", "07. latest trading day"]
        change_percent_path: ["Global Quote", "10. change percent"]
        change_percent_strip_percent: true
```

Place a real key only in a protected personal configuration; do not share this file, full URLs or unredacted logs. In Jedi 1.0 diagnostic context may contain the key when it is part of the URL. The built-in Alpha Vantage key `"yyyyy"` is merely a placeholder.

Key under `api_providers.<name>`	Actual use
`api_key`	String substituted into the request through `{api_key}`.
`symbol_candidates.mode`, `suffixes`	`ticker_plus_suffixes` appends each suffix to a ticker, in order; a ticker already containing `.` or `:` is used unchanged. Without a ticker, Jedi tries the ISIN without suffixes.
`quote.base_url`, `quote.path`, `quote.query`	Build the URL and parameters; parameter values are strings with substitutions `{symbol}`, `{api_key}` or `{q}` (the available ISIN).
`quote.price_path`, `quote.prev_close_path`	JSON key paths required to read price and previous close; a numeric path element can index an array.
`quote.trading_day_path`	Optional `YYYY-MM-DD` date path in `eod` mode; otherwise Jedi uses the current local date.

Key under `api_providers.<name>`	Actual use
`quote.change_percent_path`, `quote.change_percent_strip_percent`	API-supplied percentage; without the path, the percentage is calculated from price and previous close. `true` strips `%` from the string.

The current adapter also treats responses containing `Note` or `Information` as throttling indicators and expects a nonempty `Global Quote` object. Although JSON paths can be configured, an API with a different root may therefore need **a code change**. `symbol_path` and `change_path` appear in built-in configuration but are not read by the current extraction method: do not assume they have an effect on a new provider. `prev_close_path` remains required by the code even if `change_percent_path` supplies the percentage.

Diagnose without inventing a quote

First check file location and syntax. For a security whose ISIN is known, a query that changes no database can show whether a provider replies:

```
jedidemo config --paths
jedidemo quote get FR0000120271
```

This example command appears in Jedi's help; **it predicts neither an actual response nor permission to query the configured service**. `quote get` has no `--mode` option and does not read the stored quote. To target one mode for a registered security, `quote update <id-or-isin> --mode=eod` is supported, but **stores** the result on success; choose a suitable security and test procedure before using it. `quote refresh --mode=eod` may update several securities and is not the first diagnostic step. `quote set` can record a checked quote without an external request.

Symptom or stage	Concrete check
Configuration not applied	`config_file` path, YAML indentation, an order list containing the provider's exact name; lists replace built-in lists.
No Web identifier	ISIN, ticker/name required by template, derived variables and transformations, ISIN exception.
HTTP `429`, another error, or very short body	Quota, URL, redirect, permitted access, consent page; HTTP `200` does not guarantee a quote.
No match or a nonnumeric capture	Actual received HTML, named `price`/`variation` group, decimal format and HTML entities; verify the right security.
Empty or limited API result, unexpected JSON	Symbol and suffixes tried, key and quota, `Global Quote` structure and JSON paths.
Variation `0.0`	Check whether `regex_variation` was omitted: zero may mean "not extracted."

Jedi attempts patterns, keys, providers and modes according to configuration; the context of the final failure does not necessarily describe **every** earlier attempt. The log may contain a response excerpt, URL, parsing error and possibly an API key: inspect it locally and redact these values before sharing. Configuration errors, network responses and extraction of the wrong number are distinct problems. Check price and date against an independent authorized source before recording or using them in a trade. Keep data provenance and conditions of use, as described in chapter 21.

Appendix 3. External selection script: JEDI ProScreener

Purpose of the screener

JEDI ProScreener provides a list of securities to examine in ProRealTime. Its criteria are consistent with `profile`'s four main questions: trend, liquidity, volatility and oscillation.

Trend, liquidity and volatility provide the entry filters. Oscillation supplies selection information; it is not a rejection filter in this script. The user then chooses the security, scale and proposal to evaluate.

Prepare execution

In ProRealTime, open ProScreener, create a programmed screener and paste the listing below. Explicitly choose the market or list and work with **daily bars**. The program specifies `TIMEFRAME(Daily)`.

Read results after the selected session has ended. A developing bar can still change its close, volume and range. For comparison with Jedi, check the dataset's latest quotation date with `market info` and `market list --last-quote`.

The official ProScreener manual, version 6.1.0 dated 1 December 2025, consulted on 3 October 2026, states that calculations use 256 candlesticks, or 1,024 on Premium platforms. The script's 252-session window fits this scope. The manual also specifies three or six output criteria depending on licence.

The listing has three active columns. A four-column alternative is kept in a comment: use it only when the platform supports it, replacing the final active instruction. Keep only one active `SCREENER` instruction.

What the script measures

Measure	Calculation and purpose
Trend	Slope over 252 sessions, annualised; filter strictly above −12%.
Liquidity	Median close × volume over 252 sessions; filter at or above 100,000.
ATR20	Simple average of 20 true ranges divided by the close; filter at or above 1.5%.
Oscillation	Movements and reversals of at least 5% over 252 sessions, annualised; displayed information.

With euro prices and share volumes, liquidity is expressed in euros per session. The threshold does not convert currencies. ATR is calculated directly with a simple average; retain this calculation for consistency with Jedi.

Active columns are `Trend %/y`, `ATR20 %` and `Sw5/y`. Liquidity is filtered even when not displayed. The program accepts at least 252 historical bars; it does not reproduce `profile`'s minimum three-year depth check.

Complete listing

```
// JEDI - ProScreener journalier
// Cible : ProRealTime / ProScreener
// Fenêtre principale : 252 séances ; ATR : 20 séances
//
// Filtres équivalents aux principaux critères éliminatoires de Jedi profile :
//   - tendance annualisée > -12 %
//   - montant quotidien médian >= 100 000
//   - ATR(20) simple >= 1,5 %
//
```

```
// L'oscillation à 5 % est calculée et affichée mais n'est pas éliminatoire.

TIMEFRAME(Daily)

// ---------------------------------------------------------------------
// 1. Historique minimal disponible dans ProScreener
// ---------------------------------------------------------------------
validHistory = BarIndex >= 251

// ---------------------------------------------------------------------
// 2. Tendance sur 252 séances
// Approximation de JEDI : régression de LOG(Close) par numéro de barre,
// annualisée sur 252 séances.
// ---------------------------------------------------------------------
trendSlope = LinearRegressionSlope[252](LOG(Close))
trendPct = (EXP(trendSlope * 252) - 1) * 100
trendOK = trendPct > -12

// ---------------------------------------------------------------------
// 3. Liquidité : médiane de Close * Volume sur 252 séances
// ---------------------------------------------------------------------
FOR i = 0 TO 251 DO
  $tradedValue[i] = Close[i] * Volume[i]
NEXT

ArraySort($tradedValue, ASCEND)
medianTradedValue = ($tradedValue[125] + $tradedValue[126]) / 2
liquidityOK = medianTradedValue >= 100000

// ---------------------------------------------------------------------
// 4. Volatilité : ATR(20) SIMPLE, comme Jedi
// Ne pas utiliser AverageTrueRange[20], qui applique le lissage de Wilder.
// ---------------------------------------------------------------------
atrSum = 0
FOR i = 0 TO 19 DO
  tr1 = High[i] - Low[i]
  tr2 = ABS(High[i] - Close[i + 1])
  tr3 = ABS(Low[i] - Close[i + 1])
  trueRange = MAX(tr1, MAX(tr2, tr3))
  atrSum = atrSum + trueRange
NEXT

atr20 = atrSum / 20
atrPct = 0
IF Close > 0 THEN
  atrPct = atr20 / Close * 100
ENDIF
volatilityOK = atrPct >= 1.5

// ---------------------------------------------------------------------
// 5. Oscillation : nombre de mouvements/reversals de 5 % sur 252 séances
// Algorithme calqué sur Jedi ; valeur annualisée d'après la durée calendaire.
// Ce critère est informatif, pas éliminatoire.
// ---------------------------------------------------------------------
pivot = Close[251]
direction = 0
swings5 = 0

FOR i = 250 DOWNTO 0 DO
  c = Close[i]

  IF direction = 0 THEN
    changePct = (c / pivot - 1) * 100

    IF changePct >= 5 THEN
      direction = 1
      swings5 = swings5 + 1
      pivot = c
    ELSIF changePct <= -5 THEN
      direction = -1
      swings5 = swings5 + 1
      pivot = c
    ENDIF
```

```
  ELSIF direction > 0 THEN

    IF c > pivot THEN
      pivot = c
    ELSE
      reversalPct = (pivot - c) / pivot * 100
      IF reversalPct >= 5 THEN
        direction = -1
        swings5 = swings5 + 1
        pivot = c
      ENDIF
    ENDIF

  ELSE

    IF c < pivot THEN
      pivot = c
    ELSE
      reversalPct = (c / pivot - 1) * 100
      IF reversalPct >= 5 THEN
        direction = 1
        swings5 = swings5 + 1
        pivot = c
      ENDIF
    ENDIF

  ENDIF
NEXT

spanYears = (Days - Days[251]) / 365.25
swingsPerYear = 0
IF spanYears > 0 THEN
  swingsPerYear = swings5 / spanYears
ENDIF

// ------------------------------------------------------------------
// 6. Sélection JEDI
// ------------------------------------------------------------------
selected = validHistory AND trendOK AND liquidityOK AND volatilityOK

// Version GRATUITE : ProScreener accepte 3 critères de sortie.
// La liquidité est bien filtrée mais n'est pas affichée afin de laisser
// visibles tendance, volatilité et oscillation.
SCREENER[selected](trendPct AS "Trend %/y", atrPct AS "ATR20 %", swingsPerYear AS "Sw5/y")

// Version PREMIUM : remplacer la ligne SCREENER précédente par celle-ci
// pour afficher aussi la liquidité :
// SCREENER[selected](trendPct AS "Trend %/y", medianTradedValue AS "Med value/d", atrPct AS "ATR20 %",
  swingsPerYear AS "Sw5/y")
```

Read and use the list

Start with a few securities whose trend, ATR and oscillation are understandable. ATR above 6% deserves the same attention as the corresponding `profile` caution, although this script retains the security. Weak oscillation also deserves examination.

Consult gaps, maximum drawdown, the suggested step and long-history checks in Jedi or the history. The screener does not provide a seven-criterion global ranking.

The script's trend measure uses bar numbers and annualisation over 252 sessions; Jedi uses historical dates. Price sources, adjustments, volumes and compared populations can therefore produce differences. Examine a large discrepancy against these elements before changing the rules.

A screener result sends no price to Jedi's main database. For the chosen security, examine its quotation file or database history with `profile` and `score`, then run `candidate` with a checked current price. Chapter 03 describes this connection; chapter 10 explains files and groups.

The objective remains a coherent list of choices on the main criteria, with a reading of cautions.

Appendix 4. Jedi glossary and notation

Objects and paths

Term	Meaning in Jedi
Security	Instrument identified by an ISIN, name and ticker.
Scale	Rules mapping a mechanical state to target prices and quantities.
Builder	Parameters creating a scale; `score`'s `I:` or `II:` prefix states its type.
Operational definition	Definition actually used after construction and any transformations, shown in reports.
Trade	A security's purchase/sale path with successive settings.
Settings	Scale applying from a date/time, with the information needed for continuity.
Operation	An executed and recorded purchase or sale. A proposed order is not an operation.
Stock, holdings	Number of shares held at a point in the path.
Round trips	Successive sales and purchases following scale movements.
Operability	Ability to produce checked orders with sufficient integer quantities.
Funding	Ability to cover the opening and projected needs while accounting for other trades.
Note	Dated journal entry describing a choice or event without changing the trade.

Positions and coefficients

Notation	Meaning
`x:y:e`	Position: upward jumps, downward jumps and step within active settings.
Step	Local stage within the scale.
Level	Reference for a series of steps; reaching a boundary causes a jump.
`q0`, `c0`	Trade reference quantity and price, initially those of the first purchase.
`kc`, `ks`	Price and stock coefficient lists for a type I scale.
`vr`	Relative variation used to construct scale spacing.
`np`, `nn`	Step counts towards positive and negative boundaries.
`sr`	Target stock fraction at the final internal upward step of the compatibility builder.
`mi`	Downward-purchase budget parameter of that builder, relative to the initial amount.
`ki`	Investment fraction applied at level changes.
`kb`	Coefficient moving level prices.
`kd`	Builder parameter transforming quantities; it is not a return.
`kcb`, `kcp`, `ksp`	Calculated capital, price and stock coefficients.
`cpe`, `spe`	Theoretical price and stock at a target position.

Notation	Meaning
Type I	Scale based on reference coefficients; `calc` checks its calculations.
Type II	Scale recentered from actual operations; examine it with `play`.
Optimised	Construction or continuity variant; the term does not prove a better gain.

Chapters 13–16 explain the parameters. A reset may redefine `q0` and `c0`; read active settings before interpreting them as the very first historical purchase.

Split and new origin

Notation	Meaning
`split=NEW:OLD`	Number of new shares for old ones; `1:10` is a ten-for-one consolidation.
Effective holding	Holding to use after conversion, even while the latest historical operation retains the old quantity.
`reinit=reset,price`	Immediate new mechanical origin; position `0:0:0`, effective stock as `q0`, chosen price as `c0`.
`reinit=merge,price`	New origin established by the following manual transaction; references remain pending until then.
`split + reinit`	Combination available in next-version / v1.1; convert stock before reset or merge, supply prices after the split.
Fractional entitlement	Fractional right resulting from consolidation; Jedi accounts for neither its handling nor its settlement.

Chapter 09 details the effects and restrictions. Reinitialisation changes mechanical references, not past financial results.

Orders and protection

Term	Meaning
Limit	Purchase at a maximum price or sale at a minimum price.
Stop	Order triggered at a threshold; execution price may differ.
StopLoss	Protection order: loss cutting or gain protection according to trade state.
Tick	Price increment used when rounding an order.
Gap	Difference between previous close and next opening; it may cross a level without execution at that level.
`conservative`	Normal OHLC policy, prioritising orders executable at the opening and handling gaps.
`next`	Orders generated by the first purchase active on the next session; backtest default.

The scale's operating Stop and protective StopLoss serve different purposes. Chapters 17 and 22 explain the rules.

Capital

Notation	Meaning
`K`	Capital allocated to trading; monetary reference for sizing and funding.
`r`	Fraction of K corresponding to a trade's nominal risk.
`R`	Monetary risk: `K × r`.
`o`	Target coefficient; monetary threshold `0 = o × R` before gain protection.
`n`	Downward step count used for the risk path.
`T`	Requested opening amount, excluding fees.
`money`	Definition `r:...,o:...,n:...`; all three fields are required.
Future reserve	Projected additional purchase need over the displayed horizon.
Margin after candidate	Capital remaining after committed-trade and candidate needs.
`diagnostic_steps`	Operational-check setting, initially 3; affected help pages show its effective value.

Nominal risk is not a guaranteed loss ceiling. `candidate` reserves are projections excluding fees over a horizon, not proof that every later path would be funded. See chapters 06 and 18.

Data and freshness

Term	Meaning
OHLCV	Open, High, Low, Close and Volume for a session.
Prepared file	Identified quotations converted to Jedi's format and checked before use.
Source	Market database name, such as `demo`.
Universe	General scope included in the database name; `france` by default.
Dataset	Imported collection with memberships, files and metadata.
Group	Named security list independent of dataset membership.
Last complete import	Date and scope of the last complete import/replacement; import history.
Latest available quotation	Most recent date present; it may concern very few securities.
Global freshness	Date reached by at least 95% of dataset securities, calculated from actual data after imports/updates.
Individual lag	Market-session difference from that reference: 0 normal, 1–2 watch, more than 2 rejected.

Freshness is not measured against today's date. A file, historical selection or ambiguous dataset reference may prevent its assessment. Chapters 04 and 10 explain these cases.

profile criteria

Criterion	Reading
Trend	Estimated closing-price direction over 252 sessions, expressed as an annual rate.
Liquidity	Median Close × Volume; traded value per session, in euros if prices are in euros.
ATR20	Mean true range over 20 sessions, including differences from previous close; expressed as a percentage of price.
Oscillation	Path amplitude and reversal movements; `Sw5/y` counts moves of at least 5% and annualises them.
Drawdown	Largest decline from an earlier peak over the examined window.
Gap%	Frequency of gaps of at least 3% over the recent window.
Step%	Suggested scale spacing based on security characteristics; not a mandatory order.
Total	Unweighted sum of the first N ranks; smaller means better ranked within the current selection.
Status	Retained, watch or rejected according to checks; rejected securities do not enter ranking.

Results and studies

Term	Definition
Gross gain	Sales − purchases + final holdings valued at the last close, excluding fees.
Net gain	Gross gain − charged fees.
Current investment	Cumulative purchases − sales, excluding fees.
Maximum investment	A trade’s peak purchases minus sales; in backtest and score, the largest such peak among successive trades, excluding fees.
Backtest buy-sell balance	Cumulative balance retaining closed trades’ gains and losses; used to calculate cumulative gain.
Fitness	In `score`, gross gain / first purchase amount, covering the entire period.
Max.inv	In `score`, maximum investment / first purchase, excluding fees.
Op/mo	All operations, including openings, normalised to 21 sessions from the first operation.
Rnet%	Net result relative to maximum investment in economic output.
Rann%	Annualisation of that ratio over its duration; not a forecast.
Adaptive search	Exploration followed by refinement of chosen regions; budget limits evaluated scales.
Grid search	Exhaustive evaluation of available candidates for the builder.
Raw ranking	Direct sorting of displayed measures.
Jedi ranking	Expert reading adding engine preferences and diagnostics.
Separate verification	Test of a fixed scale on data not used to choose it.

Term	Definition
Dated monitoring	Keeping a decision at its actual date, then subsequent observations.

See chapter 24 to interpret these results.

Appendix 5. Default configuration and parameter usage

This appendix describes every entry in Jedi's built-in configuration, the `App::YAML_CONFIG` constant in `src/jedi/app/application.cr`. It matches the sources used for this edition, preparing v1.1. A built-in value is a starting point; it may differ from your effective configuration.

The tables give original values and their actual use in the code. Fields that are present but not consulted are identified. The complete YAML block at the end reproduces all values, including technical quote-collection patterns.

Read the effective configuration

```
jedi config --paths
jedi config
```

The first command displays paths, including config_file. The second displays those paths and the effective configuration after merging the personal file. In the experiment environment, use the same commands with jedidemo; its file may differ. Chapter 02 explains these environments.

The personal file uses YAML: indentation places entries beneath their parent. Jedi merges nested mappings; a supplied scalar or list replaces the entire corresponding scalar or list. You do not need to copy the complete configuration.

To allocate **€50,000 to both production and simulation**, these entries alone suffice in the relevant environment's file:

```
trading:
  capital: 50000
simulation:
  capital: 50000
```

Both built-in capitals are **20,000**, rather than 50,000. A value of 50,000 displayed by config may therefore come from this personal override. Values take effect on the next Jedi invocation; existing data and recorded transactions are not recalculated.

Display, monitoring and checks

Entry	Built-in value	Actual use
`border`	`"ascii"`	Table border style. `--border=PRESET`, before the command family, overrides it for that invocation.
`watchrate`	`10.0`	Proximity threshold in the order list, in percent: `10.0` means 10%. An absolute distance strictly below it receives the watch marker if no ready marker applies. It changes display, not risk or order prices.
`diagnostic_steps`	`3`	Quantity-check horizon over subsequent upward steps. Also the default for `--operability-steps` in `candidate` and capital-sized backtests. Candidate funding reserves use the larger of this value and `money.n`.

Entry	Built-in value	Actual use
`quotlife`	`240.0`	Maximum age, in minutes, of a recorded quote before current-price resolution attempts an update: 240 minutes = 4 hours. If the attempt fails, a previously recorded price may be reused with a warning.
`tickcheck`	`true`	Checks the price recorded by `operation create` against the security's quotation tick. Prefixing the price with + bypasses the check for that operation. Play and backtest engines record simulated operations without this check, while order prices still use tick rounding.

Border styles are ascii, reducedascii, modern, reducedmodern, fancy, markdown, empty and blank. For example:

```
jedidemo --border=modern trade list
```

Watchrate compares the proposed order price with the recorded quotation; it triggers no transaction. Quotlife is read by current-price resolution, including for trade orders. It does not schedule periodic updates: quote get queries providers without this age limit and without changing the database. Play orders use the simulated price, without this Web refresh.

Transaction fees

The following entries are under fees.

Entry	Built-in value	Actual use
`rate`	`0.0035`	Proportional fee rate: `0.0035` = 0.35% of the transaction amount. Used without an explicit `--fees`, and for new transactions in play, backtest and score.
`min`	`0.0`	Minimum monetary charge per transaction, in the same currency as prices. The calculation takes the larger of the proportional charge and this minimum, then rounds to cents.

With built-in values, a €1,000 transaction incurs €3.50 in proportional fees. For a purchase, any security tax is added to the rate before comparison with the minimum. An explicit operation create –fees amount replaces automatic calculation. Candidate funding projections exclude fees. Simulated fees enter play, backtest and score results; they do not change scale coefficients.

Changing `fees.rate` or `fees.min` affects new operations; it does not recalculate recorded fees. For an existing operation, use `operation fees ID AMOUNT`, explained in chapter 08. The amount directly replaces the recorded total; configured fees and tax are not added to it.

Real capital and money limits

Under trading:

Entry	Built-in value	Actual use
`capital`	`20000`	Reference capital for real trades' money protections. Also used for candidate sizing and funding when `--capital` is absent. Does not set a real trade's initial purchase quantity.

Under money:

Entry	Built-in value	Actual use
`risk_rate_min`	`0.005`	Lower accepted bound for money's `r`: 0.5%.
`risk_rate_max`	`0.05`	Upper accepted bound for `r`: 5%.
`target_coeff_min`	`0.5`	Lower accepted bound for the target coefficient `o`.
`target_coeff_max`	`5.0`	Upper accepted bound for `o`.
`steps_down_min`	`1`	Lower accepted bound for the integer number of downward steps `n`.
`steps_down_max`	`5`	Upper accepted bound for `n`.

These bounds are **inclusive**. They validate input; they do not automatically assign money management to trades. All three fields r, o and n must be supplied, for example money=r:2%,o:2,n:2. R is the fraction at risk, o multiplies monetary risk to set the target, and n sets the number of downward steps projected. Risk_rate_min and risk_rate_max can also be percentage strings in YAML, such as "0.5%".

Simulation parameters

The following entries are under simulation. Rates here are numeric fractions: 0.015 means 1.5%, whereas watchrate: 10.0 directly means 10%.

Entry	Built-in value	Actual use
`capital`	`20000`	Reference capital for play's protections. Also enters the reference opening amount used by score and backtest without `--capital`.
`distance`	`0.2`	Fraction in the denominator of the reference opening amount: `0.2` = 20%. Used by score and backtest without `--capital`; `--distance` can override it in the latter. Not used by play.

Entry	Built-in value	Actual use
`risk_rate`	`0.015`	Fraction in the numerator of the reference opening amount: `0.015` = 1.5%. Used by score and backtest without `--capital`. Never implicitly supplies money's `r`, including in play.

The reference opening amount is calculated at startup:

```
opening amount = capital × risk_rate / distance
built-in values: 20,000 × 0.015 / 0.2 = €1,500
personal capital of 50,000: 50,000 × 0.015 / 0.2 = €3,750
```

The result is truncated to a whole monetary unit. Keep capital and distance strictly positive. For protections, the engine falls back to trading.capital if simulation capital is not positive. **Play does not use the reference opening amount**: it opens with q0 shares at c0, or starts from the selected recorded transaction.

Which settings does each tool use?

Use	Capital	Risk rate	Distance or horizon
Real trade orders with money	trading.capital	The trade's r	The trade's n and scale.
tools play with money	simulation.capital	r supplied in the simulation	The simulation's n and scale.
tools candidate –money=…	–capital, otherwise trading.capital	r from –money	Distance reached after n downward scale steps.
tools backtest without –capital	simulation.capital for the opening amount	simulation.risk_rate for that amount; money.r for protections when enabled	simulation.distance, overridden by –distance if supplied, for the amount; money.n for protections.
tools backtest –capital=…	Explicit capital	r from the required –money	Distance reached after n steps; –distance is incompatible with this mode.
tools score	simulation.capital in the opening amount	simulation.risk_rate in that amount	simulation.distance in that amount; score simulations run without money protections.

With candidate –amount, the opening amount is explicitly chosen: it is not sized using simulation.distance or simulation.risk_rate.

Play example with personal capital of €50,000

After the configuration override shown above, this invented simulation supplies its own r, o and n:

```
jedidemo tools play -c o0,6-14,17 -- \
  "scale=kc:1.4,1.2,0.8,0.6,ks:0.8,1.2,ki:10%,kb:1,q0=1000,c0=10,tick=0.01,money=r:2%,o:2,n:2,loss=active"
    -1
```

Risk is **€1,000** (50,000 × 2%) and the target threshold **€2,000** (1,000 × 2). Projection follows two scale steps. The first purchase is 1,000 shares at €10, or **€10,000 before fees**, despite the

reference opening amount of €3,750. In this scenario the downward step executes a protective sale at €9, a gross loss of €1,000 before fees. Changing only simulation.distance or simulation.risk_rate leaves this path unchanged; changing money.r can change the protection.

Chapter 18 explains risk and sizing; chapter 19 introduces play paths.

Universe and quote-collection order

The following entries are under market_data.

Entry	Built-in value	Actual use
`default_universe`	`"france"`	Default universe used to choose the historical market database, including by market commands and tools reading database history. `--universe` overrides it where offered. Does not select a dataset or the PEA group.
`mode_order`	`["live", "eod"]`	Collection-mode order when the command does not set a mode: live Web pages, then the eod API. The first accepted result ends the search.
`api_order`	`["alphavantage"]`	API provider names to try in this order in eod mode. Each name must match a definition in api_providers.
`website_order`	`["abcbourse", "tradingsat", "boursorama"]`	Web provider names to try in this order in live mode. Each name must match a definition in websites.

The universe concerns historical storage; the other entries in this section collect a **current quotation**. They do not populate OHLCV history. Live means Web collection without guaranteeing real-time prices; eod means the API path. For real trade orders, current-price resolution requests live by default: changing mode_order does not force this path to use eod.

An empty order list traverses all matching definitions; it does not disable providers. With a nonempty list, a definition omitted from that list is not attempted. Appendix 02 explains provider configuration and diagnosis.

Built-in API provider

The following entries are under market_data.api_providers.alphavantage.

Entry	Built-in value	Actual use
`api_key`	`"yyyyy"`	Value sent to the provider through `{api_key}`. yyyyy is a placeholder; supply your own key to use this API.
`symbol_candidates.mode`	`"ticker_plus_suffixes"`	Builds API symbols from the ticker and suffixes. A ticker already containing `.` or `:` is kept unchanged. Without a ticker, Jedi uses the available identifier, here the ISIN.

Entry	Built-in value	Actual use
`symbol_candidates.suffixes`	`[".PAR", ".PA", ""]`	Suffixes tried in order. The final empty suffix tries the ticker alone. No suffix is added to an ISIN used when the ticker is missing.

Under market_data.api_providers.alphavantage.quote:

Entry	Built-in value	Actual use
`base_url`	`"https://www.alphavantage.co"`	Provider base address, to which the path is appended.
`path`	`"/query"`	Request path under that base address.
`query.function`	`"GLOBAL_QUOTE"`	HTTP function parameter requesting the GLOBAL_QUOTE response.
`query.symbol`	`"{symbol}"`	HTTP parameter replaced with each symbol built above.
`query.apikey`	`"{api_key}"`	HTTP parameter replaced with api_key.
`symbol_path`	`["Global Quote", "01. symbol"]`	**Unused** by current extraction. Its presence does not cause the returned symbol to be checked.
`price_path`	`["Global Quote", "05. price"]`	Path in the JSON response to the price to read.
`trading_day_path`	`["Global Quote", "07. latest trading day"]`	Path to the trading date. In eod mode it is recorded at local midnight.
`prev_close_path`	`["Global Quote", "08. previous close"]`	Path to the previous close. The code reads it even when the percentage change is supplied directly.
`change_path`	`["Global Quote", "09. change"]`	**Unused** by current extraction: the absolute price change is not read at this path.
`change_percent_path`	`["Global Quote", "10. change percent"]`	Path to the percentage change. If not provided, Jedi computes it from the price and previous close.
`change_percent_strip_percent`	`true`	Removes the percent sign from the change text before converting it to a number.

A path such as [“Global Quote”, “05. price”] means: open the Global Quote object, then read 05. price within it. The current reader also requires a nonempty Global Quote object. Changing only paths may therefore be insufficient to use another API.

Built-in Web providers

Each site has its own address, identifier templates and capture patterns. Tables describe entries and their uses; addresses and regexes appear below each table across the full page width. In double-quoted YAML strings, a trailing backslash continues the same string on the next line without adding a character. Values are identical to those in the code. Their presence does not guarantee that the site will respond or that its HTML will remain compatible.

ABC Bourse

Under `market_data.websites.abcbourse` :

Entry	Built-in value	Actual use
`url`	Address below	Security page.
`regex_price`	Patterns below	Price extraction.
`regex_variation`	Patterns below	Change extraction.

In the address, `{key}` is replaced with each candidate identifier. Regexes locate numbers in HTML: the named `price` group supplies the price; `variation` supplies percentage change.

```
url: "https://www.abcbourse.com/cotation/{key}"
regex_price: "id=\"lastcx\">(?<price>\\d+(?:[.,]\\d+)*)\\s*&#x20AC;"
regex_variation: "id=\"varcx\"\\s+class=\"quote_(up|down)b\">(?<variati\
  on>(&#x2B;|&#x2D;|\\+|-)?\\d+(?:[.,]\\d+)*)%"
```

Under `key_strategy.candidates` :

Entry	Built-in value	Actual use
`[0].template`	`"{ticker}p"`	Ticker followed by `p`.
`[1].template`	`"{isin}"`	ISIN alone.

Templates are tried in this order. The first is skipped if the ticker is unavailable.

Tradingsat

Under `market_data.websites.tradingsat` :

Entry	Built-in value	Actual use
`url`	Address below	Security page.
`regex_price`	Patterns below	Price extraction.
`regex_variation`	Patterns below	Change extraction.

In the address, `{key}` is replaced with each candidate identifier. Regexes locate numbers in HTML: the named `price` group supplies the price; `variation` supplies percentage change.

```
url: "https://www.tradingsat.com/{key}"
regex_price: "class=\"price\">(?<price>\\d+(?:[,.]\\d+)*)\\s*€"
regex_variation: "class=\"variation var-(down|neutral|up)\">(?<variatio\
  n>[+-]?\\d+(?:[,.]\\d+)*)\\s*%"
```

Under `key_strategy.candidates` :

Entry	Built-in value	Actual use
`[0].template`	`"{name_slug}-{isin}"`	Transformed name and ISIN.

This template is skipped if the required name is unavailable. `name_slug` is built by the transformations below.

Under `key_strategy.candidates[0].vars.name_slug` :

Entry	Built-in value	Actual use
`from`	`"name"`	Security name.
`ops[0]`	`"downcase"`	Conversion to lowercase.
`ops[1].gsub.pattern`	`"\\."`	Period matching.
`ops[1].gsub.replace`	`""`	Removes the periods.
`ops[2].gsub.pattern`	`" "`	Space matching.

Entry	Built-in value	Actual use
`ops[2].gsub.replace`	`"-"`	Replaces spaces with hyphens.

Boursorama

Under `market_data.websites.boursorama` :

Entry	Built-in value	Actual use
`url`	Address below	Security page.
`regex_price`	Patterns below	Price extraction.
`regex_variation`	Patterns below	Change extraction.

In the address, `{key}` is replaced with each candidate identifier. Regexes locate numbers in HTML: the named `price` group supplies the price; `variation` supplies percentage change.

```
url: "https://www.boursorama.com/cours/{key}/"
regex_price: "c-faceplate__info.+?price.+?ist-last>(?<price>\\d+(?:[,.]\
  \\d+)*)<"
regex_variation: "c-faceplate__info.+?fluctuation.+?ist-variation>(?<va\
  riation>[+-]?\\d+(?:[.,]\\d+)*)%"
```

Under `key_strategy.candidates` :

Entry	Built-in value	Actual use
`[0].template`	`"1rP{ticker}"`	`1rP` prefix and ticker.
`[1].template`	`"1rEP{ticker}"`	`1rEP` prefix and ticker.
`[2].template`	`"{ticker}"`	Ticker alone.
`[2].ops.ticker`	`["upcase", "trim"]`	**Not applied.**

The three templates are tried in this order and skipped if the ticker is unavailable. **The `upcase` and `trim` transformations under `[2].ops.ticker` are unused at this location** by the current builder. Supported operations are under `vars.<variable>.ops`.

Complete built-in configuration

This block reproduces the built-in values as readable YAML. Source-code comments are not configuration entries. It is a reference; a personal file can contain only the desired overrides.

```
border: ascii
market_data:
  default_universe: france
  mode_order:
  - live
  - eod
  api_order:
  - alphavantage
  website_order:
  - abcbourse
  - tradingsat
  - boursorama
  api_providers:
    alphavantage:
      api_key: yyyyy
      symbol_candidates:
        mode: ticker_plus_suffixes
        suffixes:
        - .PAR
        - .PA
        - ""
      quote:
        base_url: https://www.alphavantage.co
        path: /query
```

```
        query:
          function: GLOBAL_QUOTE
          symbol: '{symbol}'
          apikey: '{api_key}'
        symbol_path:
        - Global Quote
        - 01. symbol
        price_path:
        - Global Quote
        - 05. price
        trading_day_path:
        - Global Quote
        - 07. latest trading day
        prev_close_path:
        - Global Quote
        - 08. previous close
        change_path:
        - Global Quote
        - 09. change
        change_percent_path:
        - Global Quote
        - 10. change percent
        change_percent_strip_percent: true
  websites:
    abcbourse:
      url: https://www.abcbourse.com/cotation/{key}
      key_strategy:
        candidates:
        - template: '{ticker}p'
        - template: '{isin}'
      regex_price: "id=\"lastcx\">(?<price>\\d+(?:[.,]\\d+)*)\\s*&#x20A\
        C;"
      regex_variation: "id=\"varcx\"\\s+class=\"quote_(up|down)b\">(?<v\
        ariation>(&#x2B;|&#x2D;|\\+|-)?\\d+(?:[.,]\\d+)*)%"
    tradingsat:
      url: https://www.tradingsat.com/{key}
      key_strategy:
        candidates:
        - template: '{name_slug}-{isin}'
          vars:
            name_slug:
              from: name
              ops:
              - downcase
              - gsub:
                  pattern: \.
                  replace: ""
              - gsub:
                  pattern: ' '
                  replace: '-'
      regex_price: "class=\"price\">(?<price>\\d+(?:[,.]\\d+)*)\\s*€"
      regex_variation: "class=\"variation var-(down|neutral|up)\">(?<va\
        riation>[+-]?\\d+(?:[,.]\\d+)*)\\s*%"
    boursorama:
      url: https://www.boursorama.com/cours/{key}/
      key_strategy:
        candidates:
        - template: 1rP{ticker}
        - template: 1rEP{ticker}
        - template: '{ticker}'
          ops:
            ticker:
            - upcase
            - trim
      regex_price: "c-faceplate__info.+?price.+?ist-last>(?<price>\\d+(\
        ?:[,.]\\d+)*)<"
      regex_variation: "c-faceplate__info.+?fluctuation.+?ist-variation\
        >(?<variation>[+-]?\\d+(?:[.,]\\d+)*)%"
fees:
  rate: 0.0035
  min: 0.0
watchrate: 10.0
diagnostic_steps: 3
```

```
quotlife: 240.0
tickcheck: true
trading:
  capital: 20000
money:
  risk_rate_min: 0.005
  risk_rate_max: 0.05
  target_coeff_min: 0.5
  target_coeff_max: 5.0
  steps_down_min: 1
  steps_down_max: 5
simulation:
  capital: 20000
  distance: 0.2
  risk_rate: 0.015
```